Engineering Reasoning Agents for ARC-AGI-2 and ARC-AGI-3

Yi Chia CHEN, Simon Frieder

November 2026

Abstract

We present a coding-agent system for the ARC-AGI-2 and ARC-AGI-3 tracks of ARC Prize 2026, using open-source models in a no-internet environment with limited hardware and time. Agents are tailored to the respective formal ARC-AGI environments: For ARC-AGI-2, independent agents construct and test transformation programs in separate workspaces, followed by a refiner that inspects their solutions and revises candidate outputs. For ARC-AGI-3, agents infer game rules through interaction and maintain scripts, observations, and hypotheses across context windows. Both pipelines use the Codex runtime with Qwen3.8-Flash-Next. To deploy this 176-billion-parameter backbone alongside concurrent agent sessions, we combine expert pruning, three-bit expert quantization, and quantization-aware distillation. We adapt SGLang to retain sparse-attention history in host memory and preserve session state across tool calls, reducing memory pressure and repeated prefill. A speculative drafter trained on agent trajectories improves decoding throughput. We further develop a server-side trajectory recorder that preserves exact token sequences and behavior probabilities, supporting distillation and reinforcement learning with trajectories collected through the deployed agent runtime. The selected submissions achieved public leaderboard scores of 87.22 on ARC-AGI-2 using four L4 GPUs within twelve hours and 75.56 on ARC-AGI-3 using one RTX PRO 6000 within nine hours.

Source repository: https://github.com/ycchen-tw/ARC-Forge-2026
SGLang fork: https://github.com/ycchen-tw/sglang-arc-2026
Training data: https://huggingface.co/datasets/ycchen/arc-2026-training-data
Deployment details: Selected submissions and reproducible configurations in Table 2 and Appendix A.

Contents

1 Introduction **4**

2 Evaluation Setup and Main Results **4**

3 Design Choices and Supporting Evidence **7**

4 ARC-AGI-2: Solvers, Refinement, and Scheduling **10**

5 ARC-AGI-3: Interaction and Memory **16**

6 Developing the Harnesses with a Human in the Loop **21**

7 Model Compression and Adaptation **25**

8 Inference under Hardware Constraints **30**

9 The Codex–SGLang Runtime Contract **34**

10 TITO: Exact Trajectory Capture **38**

11 Streaming RL for Long-Horizon Agents **42**

12 Lessons, Negative Results, and Silent Failures **53**

13 Prior and Related Work **57**

14 Discussion and Limitations **59**

15 Conclusion **60**

References **62**

A Deployment Manifest of the Selected Submissions **69**

B Training Recipes of the RL Checkpoints in the Finals **72**

C Reproducibility and Data Separation **73**

D Codex Runtime Patch Inventory **76**

E Designing and Training the Speculative Drafter **77**

Reading guide

What was submitted. Table 2 lists the four selected submissions and Table 3 summarizes what they run; Appendix A gives every setting.
The solution. Start with the overview and evaluation in Sections 1 and 2, then the design evidence, the two agent harnesses and how they were developed in Sections 3 to 6.
Deployment. Sections 7 to 9 explain how compression, serving and the agent runtime fit the hardware budget. Appendices A and D give the exact configurations and runtime patches, and Appendix E covers the design and training of the speculative drafter.

Training and lessons. Sections 10 and 11 cover trajectory capture and RL; Sections 12 and 14 cover failed approaches, measurement pitfalls and limitations, and Section 13 places the system among prior work. Checkpoint recipes are in Appendix B; data separation and the steps that rebuild a submission are in Appendix C.

1 Introduction

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) is designed to evaluate how efficiently systems acquire new skills from limited experience, using tasks grounded in basic concepts such as objects, number, and geometry [1]. ARC-AGI-2 presents static grid problems in which a solver must infer transformation rules from a few input–output examples and apply them to new inputs, emphasizing rule composition, sequential reasoning, and contextual rule application [2]. ARC-AGI-3 introduces interactive, turn-based environments in which an agent must explore to discover the rules and objectives, then plan actions to complete successive levels. Its score reflects both level completion and action efficiency relative to human baselines [3].

The ARC Prize 2026 Kaggle competitions require submissions to use open-source models in a no-internet environment with limited hardware and time. Although frontier models equipped with coding tools can already solve many ARC-AGI tasks when given ample inference time [4–6], achieving comparable performance under these constraints remains challenging. Performance therefore depends on both the model's reasoning ability and how efficiently the system supports sustained reasoning and tool use.

We present a coding-agent system for both ARC Prize 2026 tracks, using Qwen3.8-Flash-Next as its reasoning model. The agent writes and executes programs to investigate each task, retaining its code, observations, and intermediate results in a persistent workspace. For ARC-AGI-2, independent agents construct transformation programs and test them against the demonstrations; another agent reviews their work and revises the proposed answers. For ARC-AGI-3, an agent explores an unfamiliar game, develops hypotheses about its rules, and writes scripts to analyze observations and plan actions. Its workspace preserves what it has learned as play continues (Sections 4 and 5).

Supporting these workflows on competition hardware requires reducing both the memory occupied by the model and the cost of sustained interaction. We compress the model and adapt the inference engine to reuse previous computation across tool calls. These changes reduce the memory and execution costs of running many agents with long reasoning histories (Sections 7 to 9 and Appendix E).

The system also connects task execution to model training and agent development. We record the agents' interactions for distillation and reinforcement learning (Sections 10 and 11). In parallel, language models analyze unsuccessful runs and propose changes to tools and workflows, while the researcher directs experiments and decides which changes to adopt. We describe this human-guided development process and the experimental evidence behind the resulting design (Sections 3 and 6).

The selected submissions achieved public leaderboard scores of 87.22 on ARC-AGI-2 using four L4 GPUs within twelve hours and 75.56 on ARC-AGI-3 using a single RTX PRO 6000 within nine hours, ranking second and first at the close of submissions. Section 2 reports the selected configurations and repeated-run results, and Table 3 summarizes what the final system runs.

2 Evaluation Setup and Main Results

This section gives the competition rules, the four selected submissions with their repeated-run scores, and the measurement terms used later.

2.1 Competition setting

Both competition tracks evaluate submissions without internet access. Table 1 summarizes their task interfaces, hardware, and time limits. The B200 GPU machines used for development and training are separate from the competition hardware. Detailed hardware measurements and submission settings appear in Appendix A.

Table 1: Competition tasks and resource limits for the selected submissions.

	ARC-AGI-2	ARC-AGI-3
Task interface	Input–output grid demonstrations and test inputs	Visual observations and environment actions
Submission output	Two candidate grids per test input	Actions used to complete successive levels
GPU hardware	4×L4 (24 GB each)	1×RTX PRO 6000 (96 GB)
Time limit	12 h	9 h

2.2 Selected submissions and repeated runs

Table 2 lists the four selected submissions, two per track. Each score is a Kaggle public-leaderboard score: it covers part of the hidden tasks, not the public development set, and private results were not available at the time of writing. An ARC-AGI-2 score is the mean over tasks of the fraction of test outputs matched by either of the two candidates, in percent; this is the official partial-credit convention. An ARC-AGI-3 score is relative human action efficiency (RHAE), which depends on both level completion and action count (Section 5). The two tracks' numerical scores are not directly comparable. At the close of submissions the selected entries ranked second (ARC-AGI-2) and first (ARC-AGI-3) on the public leaderboards.

Each submission of a notebook version runs the whole system again on the hidden tasks, so resubmitting an identical version gives a repeat measurement. We submitted each selected version two to five times, including identical late resubmissions after the deadline, which we use only as repeat measurements. The 87.22 of the selected ARC-AGI-2 base submission is the highest of its five runs, which average 85.64 (range 84.31–87.22); the four runs of the RL variant average 85.03. On ARC-AGI-3 the base version averages 74.55 and the toolkit variant 75.32. Scores of a single version span up to 5.00 points on ARC-AGI-2 and 5.94 points on ARC-AGI-3, while the two means of a track differ by less than one point. On ARC-AGI-2 that difference lies inside the range of either configuration, so the repeated runs do not rank one above the other. The configurations also differ in harness settings; the submission comparison is not an experiment that changes only the RL weights.

Each summary includes every scored run of its script version recorded in the submission ledger; for the ARC-AGI-2 base submission these are the five runs of script version 362595216. The 69.58 result discussed in Section 14 belongs to a different, unselected notebook version and is not an observation excluded from this mean.

Table 3 summarizes what the four submissions run. Sections 4 and 5 describe the two harnesses. Tables 16 and 17 in Appendix A give every setting with the notebook script versions, and Appendix B gives the training recipes of the two RL checkpoints.

Table 2: The four selected submissions. "Public" is the public-leaderboard score of the selected submission. "Runs" counts scored submissions of the identical notebook version (including post-deadline resubmissions used only as repeats), and "Mean (range)" summarizes their scores. Each name links to the exact script version; the tag in parentheses is its internal version name. The names identify deployment variants, not RL-only comparison arms; Tables 16 and 17 give the full configuration.

Configuration	Submission	Public	Runs	Mean (range)
ARC-AGI-2				
Base (ext300-arcv6): compressed model, no RL weights; two independent solvers, a refiner, and a second refiner in spare capacity; no thinking cap	57747104	87.22	5	85.64 (84.31–87.22)
Scheduler + RL (x39): adds an RL weight patch, a 49,152-token thinking cap, extra solvers for tasks that still fail the demonstrations, later prompts and helper library, and backlog-aware admission of optional work	57848658	85.97	4	85.03 (82.64–87.64)
ARC-AGI-3				
Base (v35): compressed model, no RL weights; first-generation harness with a click report and a read-only reviewer	57678190	75.56	3	74.55 (71.07–77.01)
Toolkit + late RL (v40): adds a moving-camera toolkit, game cards for the reviewer, a final-call notice, and an in-place switch to RL weights after 6 h	57846476	75.44	2	75.32 (75.20–75.44)

2.3 How to read the measurements

The report also uses local experiments, which run on public data: ARC-AGI-2 experiments use public input–output tasks, and ARC-AGI-3 experiments use the 25 public games. Each comparison's task set, hardware, budget and repetition count are stated in its table or text.

For ARC-AGI-2, pass@1 evaluates one candidate and pass@2 allows two candidates. Results marked "partial credit" use the official convention defined above; "strict" results require every test output for a task to be correct.

"G4" denotes the RTX PRO 6000 development machine. For ARC-AGI-2 it can run a time-scaled simulation of the four-L4 deadline; for ARC-AGI-3 it can rehearse the nine-hour budget. B200 measurements concern development, training, or serving experiments.

Decoder throughput measures the serving loop; end-to-end throughput also includes prefill, tools and waiting. Unless stated otherwise, score differences are in percentage points, brackets are 95% intervals, and single runs are observations rather than estimates of run-to-run variation. Section 3 discusses that variation; Section 14 and Appendix C describe data separation and evaluation limitations.

Budget terminology. Five limits recur in the report and act at different levels. A *thinking cap* limits reasoning within one model response. The *served context limit* bounds the token sequence accepted by the model; the agent's *compaction threshold* determines when it shortens its active history. An *episode output budget* counts cumulative generated tokens, including reasoning, across one episode, that is, one agent run on a task (Section 9 defines the execution units). The *wall-clock*

Table 3: The final system at a glance. Each track has a base submission and a second submission that adds RL weights and later harness changes (Table 2). A setting applies to both submissions of a track unless the row says otherwise. Token counts for context are: limit served by the model / agent context window / compaction threshold.

	ARC-AGI-2 (4×L4, 12 h)	**ARC-AGI-3** (1×RTX PRO 6000, 9 h)
Model	Qwen3.8-Flash-Next with 320 of 512 experts kept per layer, 3-bit experts with FP8 activations, and quantization-aware distillation (Section 7)	Same package
Drafter	DFlash, 8 layers, four draft tokens per cycle (Appendix E)	Same
Agent runtime	Codex 0.154.0 with our patches (Section 9 and Appendix D)	Same
Serving	One SGLang server across four GPUs; attention in two data-parallel pairs; session history in host memory (Section 8)	One SGLang process; NVFP4 KV; session history in host memory
Concurrency	128 agent sessions	110 games
Agents	Per task: two independent solvers, then a refiner; a second refiner in spare capacity; the two answers are chosen by vote (Section 4)	Per game: one player with a persistent workspace and a click report; a read-only reviewer from 2 h on for games stuck at least 45 min (Section 5)
Context	153,600 / 150,000 / 100,000; the second submission serves 179,968	240,000 / 235,000 / 100k–150k, moving with the number of live games
Thinking cap	None (base); 49,152 tokens (second)	24,576 tokens
Output budget	200k tokens per solver, 68k more for the first solver if time allows; 100k per refiner	No per-game output budget
Second submission adds	RL weights loaded at start-up; up to two extra solvers for tasks that still fail the demonstrations at 80k tokens; the thinking cap; later prompts and helper library; second refiner skipped when all candidates agree	A toolkit for the player; game cards for the reviewer; a final-call notice 15 min before the end; RL weights swapped in place at 6 h

deadline limits total elapsed work. A long episode can therefore span several context windows and still have a finite output budget.

3 Design Choices and Supporting Evidence

The selected submissions rest on five choices: an off-the-shelf backbone compressed to fit the competition hardware; a coding agent that keeps its work in a sandbox and answers by running programs; two independent solvers and a refiner for each ARC-AGI-2 task; long but finite output budgets; and a serving stack built so that planned work finishes before the deadline. Each subsection gives the headline evidence and points to the section with the details. Most comparisons are single runs on public tasks, and several changed more than one component at once; the last subsection reports how much single runs vary. Trajectory capture and RL are covered in Sections 10 and 11.

3.1 An off-the-shelf backbone, compressed to fit

The backbone is Qwen3.8-Flash-Next [7]. We compared available open models and our fine-tuned students on the 120 public ARC-AGI-2 evaluation tasks using the same tool-integrated reasoning (TIR) protocol (pass@2 with official partial credit). Without task-specific fine-tuning, Qwen3.5-35B reached 12.92%, Qwen3.8-27B about 59%, GLM-5.3-Flash 71.25%, and Qwen3.8-Flash-Next 88.61% at its highest reasoning effort (two runs: 90.28 and 86.94). Our best fine-tuned 35B student reached 42.78%. In this comparison, the off-the-shelf Flash-Next model outperformed the smaller models we had fine-tuned, so we focused on deploying it within the competition limits. Zeroing its 51B-parameter n-gram embedding tables reduced the score to 71.11%, so we retained them in host memory. These public-set comparisons guided model selection; they do not isolate the effects of architecture, pretraining data or task-specific training.

We compressed the model mainly to fit it and the session state in memory: the deployed package keeps 320 of the 512 experts in each layer, with three-bit expert weights and eight-bit activations, followed by quantization-aware distillation (Section 7). Pruning has a measured cost: a 256-expert model scored 82.36 on the same protocol (Table 9).

3.2 A coding agent that answers by running programs

On both tracks the agent is Codex working in a sandbox: it writes programs, runs them and keeps its task state in files (Section 9). Two ARC-AGI-2 observations motivated this design. First, when the model stated both a rule and an answer, executing its own `transform()` was more accurate than the reported answer in our comparison: +5.4 points (27 wins to 9, $p = 0.0039$). Second, an earlier harness of ours, three explorer sessions followed by a refiner, rose from 72.78% to 81.11% in pass@2 at the same generated-token budget when work was handed on as sandbox files rather than as a written summary. An image-only variant of the Codex harness scored 51.94 in pass@1 against 85.28 with readable grid values, so the agent always receives machine-readable grids.

On ARC-AGI-3 a tuned Duck harness led at 150k tokens per game, and the Codex arms led at 800k (64.69–69.99 against 57.80). We estimated a nine-hour run at on the order of 1M tokens per game, so we deployed Codex (Section 5).

3.3 Two independent solvers and a refiner

Each ARC-AGI-2 task gets two solvers that work independently, then a refiner that reads both workspaces and can run code (Section 4). In paired comparisons on the 120 public tasks, an independent second solver added 6.11 points of pass@2 over the first solver alone (95% CI $[+2.5, +10.4]$), and a refiner over the two workspaces added a further 2.92 points (Table 5).

3.4 Long but finite output budgets

In the selected submissions each solver may generate 200k tokens per task (the first up to 300k when the time gate allows) and the refiner 100k (Table 16). With the Codex harness (Section 4), a variant with a 1M-token per-task cumulative output budget scored 85–86 in pass@1 (86.11 and 85.28 in two revisions), against about 77 on average for the earlier 100k-token variants. These revisions did not

Table 4: Deployment observations about throughput, budgets and completed work. These are separate comparisons on different workloads, some with several changed components, not a controlled throughput–score curve. Single runs unless stated.

Observation	Measurement
ARC-AGI-2, early L4×4 build	A 9 h rehearsal finished 155 of 240 tasks at a median 618 tok/s; the other 85 never started.
ARC-AGI-3, harness generations	A second-generation harness that rebuilt each context window instead of compacting it scored 21.71 against 38.68 for the first generation in a 2 h run. Both cleared 16.9 levels per million generated tokens; the second generation lost on re-prefill (46.3M vs. 12.9M tokens) and decode speed.
ARC-AGI-2, extra solvers	Extra solver sessions recovered 2.29 points in a run where the first two solvers did poorly and 0.21 where they did well, about 0.13 points per million tokens over three runs.
ARC-AGI-3, context length	Halving the served context (262k→120k) cost 1.1 points while using 65% of the actions: the shorter-context run produced more thinking per action (Section 5).
ARC-AGI-3, further decode speed	With the HiSparse cache path already removing the cache bottleneck, a further +6.5% decode throughput (1,980→2,109 tok/s) moved the public score by +0.17.

isolate the effect of the budget. The allowance included reasoning tokens; it was not a per-response thinking cap. A four-hour per-task wall limit and compaction at 140k context tokens still applied. Output length had a long tail (mean 93k, median about 45k tokens per task), which motivated larger budgets for selected episodes rather than an unlimited deployment.

3.5 Completing work within the deadline

Serving throughput limited the number and length of attempts we could complete within the competition deadlines. After establishing the solver pipeline, we focused on reducing serving overhead so that more of the planned work could finish. Table 4 collects the observations that motivated this work. In ARC-AGI-3, completed levels and action efficiency must also be considered alongside generated tokens.

Section 8 and Appendix E describe the serving work that followed: a HiSparse-based cache path [8] that keeps session history in host memory, and speculative decoding with a DFlash block drafter [9]. On the L4×4 build the drafter made decoding 1.70–2.04× faster than decoding without speculation, in a sweep at 8–13k context (Table 10).

3.6 Interpreting variation across runs

Repeated baselines show how large a difference a local comparison can detect. In one ARC-AGI-2 benchmark, 18 baseline rounds on 120 public tasks had a mean pass@1 of 82.66 and a between-round standard deviation of 2.90 points. Comparing 1, 3 or 5 treatment rounds against all 18 control rounds gave estimated minimum detectable effects of 8.4, 5.1 and 4.1 points at two-sided $\alpha = 0.05$

and 80% power. Shared-server load varied across those rounds, so these are planning estimates for that setting, not thresholds for every experiment in the report.

In a separate ARC-AGI-3 check, two identical configurations scored 78.6 and 79.4 across 25 games; the paired per-game differences had a standard deviation of 19.2 points. That statistic measures variation across games within a paired comparison, not the standard deviation of an aggregate score across many runs. It puts the standard error of the mean difference between two single runs at about 3.8 points.

A small estimated effect with a wide interval leaves its direction uncertain; a large effect still requires checking for confounding changes. When a trace exposed a specific fault, such as a cache miss or dropped frame, we could justify fixing that fault without claiming a corresponding score gain. Section 12 records cases where further checks changed our interpretation.

4 ARC-AGI-2: Solvers, Refinement, and Scheduling

Both selected ARC-AGI-2 submissions run the pipeline of Figure 1 on every task. Two independent solvers each write and test a transformation program. A refiner reads both solver workspaces and writes two answers per test input. A second refiner runs when capacity is left, and a vote or a fallback rule picks the two submitted grids. A scheduler keeps 128 sessions running and harvests whatever exists at the deadline. The section describes what shipped, then the measurements behind the design, then the solver changes we did not adopt.

Every episode works in a Codex workspace built from the task: the demonstration pairs and test inputs as JSON, one text file and one image per grid, a short brief (`AGENTS.md`) and a checker. A solver maintains `transform()` in `solution.py` and runs the checker against the demonstrations as often as it likes. When the episode ends, or is killed by budget or deadline, the harness runs its own copy of the checker under the task's uid and harvests test predictions from whatever program exists (Figure 2). Solver candidates therefore come from executed code. Refiners can write final grids directly.

4.1 Solvers, refiners and budgets

The first and second solver work on the same task in separate workspaces and do not see each other's work. The second solver's prompt also invites an alternative program, `solution2.py`, for a second reading of the task; the harness counts it only when it passes every demonstration and its output differs from that solver's primary answer.

Each solver has an episode output budget of 200k tokens, including reasoning (Section 2 defines the budget terms). At the budget the harness inserts a wrap-up message asking the solver to write its best rule into `solution.py` and stop, and it stops the session 32k tokens later. The first solver can receive 68k more tokens before the wrap-up. The harness grants this extension when, at the episode's own output rate, the tokens fit before the 9-hour episode limit and before the deadline minus a 75-minute refiner reserve; Appendix A lists the resulting thresholds. We added it because, in three earlier public-evaluation runs, 56–73% of the points the final submission missed were on tasks where a solver had hit its budget. It is limited to the first solver, which starts first: a schedule simulation put the cost of the longer first-solver budget near zero. Codex compacts the history at

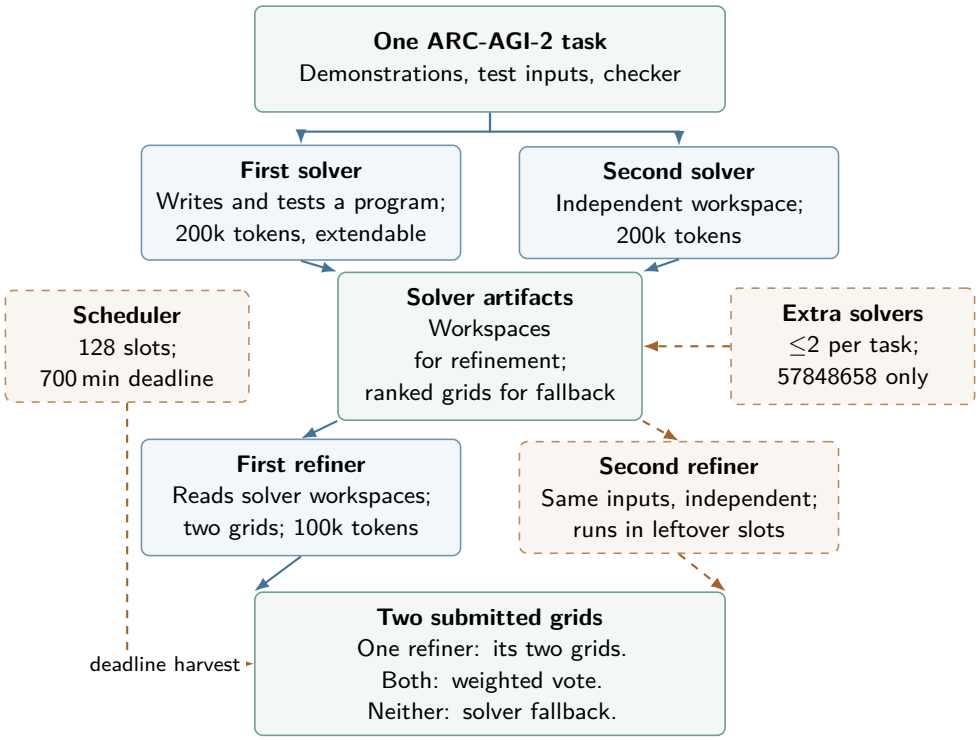


Figure 1: The ARC-AGI-2 pipeline for one task, as run by both selected submissions. Two independent solvers write and test programs; the first refiner reads both workspaces and writes two answers per test input. The harness submits one refiner's two grids, a weighted vote when both refiners finish, or ranked solver outputs when neither does (Section 4.2). The second refiner and the extra solvers are optional: the scheduler admits them when slots and time allow, and only submission 57848658 has extra solvers. Arrows show artifact flow, not launch order; token counts are episode output budgets.

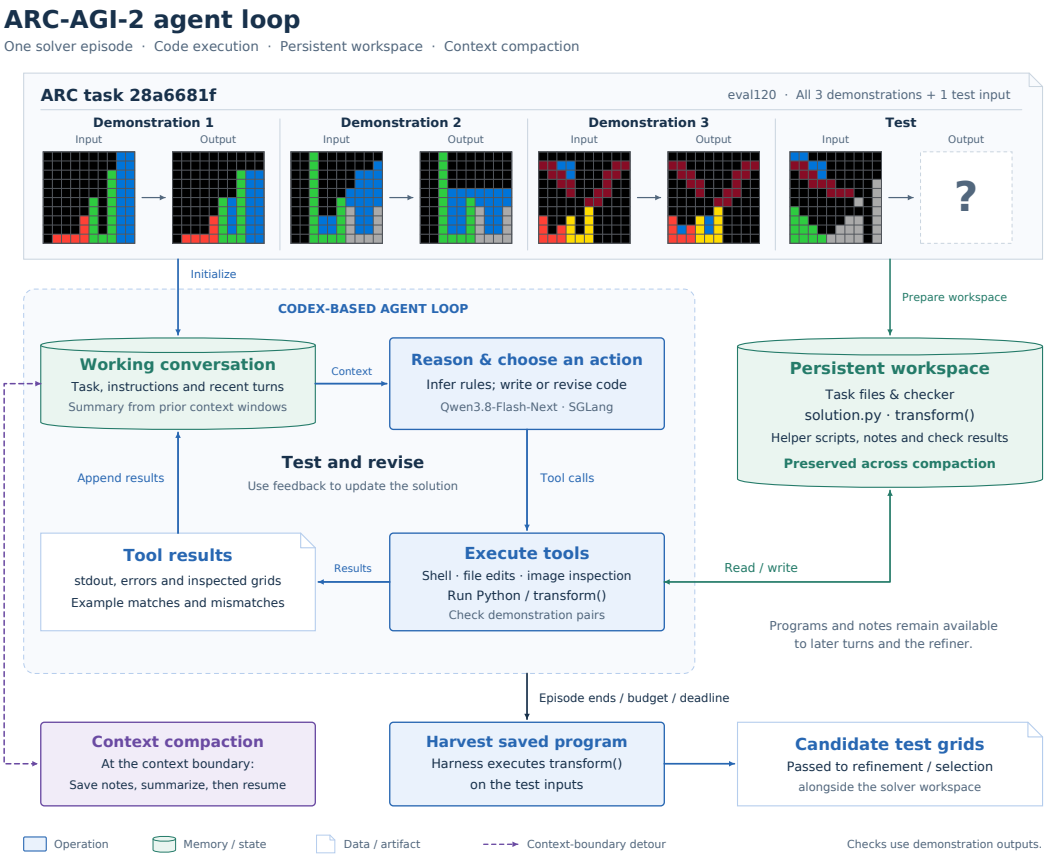


Figure 2: One ARC-AGI-2 solver episode, shown on public evaluation task `28a6681f`. The agent reasons over the task, then calls tools that edit and run its program against the demonstration pairs, and the results return to the conversation. The program, helper scripts, notes and check results are files in the workspace, so they survive context compaction and are later read by the refiner. When the episode ends, the harness runs the saved `transform()` on the test inputs to obtain the candidate grids.

100k tokens of its 150k-token window.

A task's refiner starts after its first and second solver finish. It reads both solver workspaces, including check results, notes and alternative-program outputs. It can execute code, and it writes up to two final grids per test input, in order of preference, within a 100k-token output budget. A second refiner, when the scheduler admits one, reads the same solver workspaces and does not see the first refiner's answers.

4.2 Selecting the final two answers

For each test input, the fallback pool first merges identical primary solver outputs. It ranks the distinct grids by whether any producing solver passed all demonstrations, then by the largest number of passed demonstration pairs, the number of producing solvers, and solver order. Distinct alternative-program outputs are appended after this primary pool. If a refiner leaves a slot missing or invalid, the harness fills it from that pool, choosing a different grid for a missing second slot where possible. Without a usable refiner result, it submits the first two grids in the pool.

When both refiners return results, each first-ranked answer receives two votes and each second-ranked answer one. Identical grids share their votes, and the two highest-scoring distinct grids are submitted. Ties follow the first refiner's first and second answers, then the second refiner's first and second answers. For example, outputs (A, B) and (C, B) give two votes each to A, B and C; the tie rule selects (A, B).

At the deadline, the harness harvests available solver programs and first-refiner outputs. It discards an interrupted second refiner and uses the fallback rules above; any slot still empty receives the test input as a valid-grid placeholder.

4.3 Scheduling against a 12-hour deadline

On four L4s each session decodes at roughly 12 tokens/s, so a long solver runs for hours and the deadline decides how much of the pipeline finishes. The scheduler keeps 128 session slots full, launching ready episodes in stage order (first solvers, second solvers, refiners) without waiting for a stage to finish across tasks. Second refiners use only slots that these required episodes cannot fill, and start only with at least two hours left while required episodes remain, or one hour once all are done. A valid submission file exists from the start and is rewritten atomically as episodes finish. At the deadline, 700 minutes after the start, the harness stops the running episodes and harvests them.

One of the two submissions (57848658, harness version x39) adds extra solvers and a backlog-aware admission rule. An extra solver is a further independent solver for a task that is not yet *decided*: a decided task has two independent solvers that pass every demonstration and give the same answer. The harness requests one when a solver has produced 80k output tokens and the task still has no program that passes the demonstrations; when both solvers pass but give different answers or a differing alternative program, after at least 100k output tokens together, in which case the extra solver reads both solvers' reports and is asked for a different reading; and when an earlier extra solver, or the first refiner with nothing queued, finishes and the task is still undecided. A decided task also receives one extra solver, capped at 100k tokens, because two agreeing solvers can both be wrong; the refiner does not wait for it. A task receives at most two extra solvers, and at most 48 run at once.

The admission rule keeps this optional work from displacing refiners: an extra solver starts only when its budget plus the estimated remaining output of the required episodes fits in the capacity left before the deadline. The estimate scales per-stage episode-length priors by a run-specific difficulty multiplier, the Kaplan–Meier restricted mean of this run's first- and second-solver lengths relative to the prior, with running solvers right-censored.

4.4 The selected configurations

The two selected submissions share everything described above except the extra solvers and the admission rule. Both serve the same compressed model and drafter (Sections 7 and 8) and run Codex at the xhigh reasoning-effort setting.

They differ as follows.

- *Weights.* Submission 57747104 (public score 87.22) uses the base weights. Submission 57848658 (85.97) loads an RL weight patch (Section 11).

- *Thinking cap.* 57747104 has none. 57848658 caps reasoning at 49,152 tokens per response and serves a 179,968-token context instead of 153,600. The cap keeps one response from pushing the history past what a compaction request can hold.
- *Optional work.* 57747104 has no extra solvers. 57848658 has the extra solvers and the admission rule, and it skips the second refiner when both solvers pass the demonstrations and agree with the first refiner's first answer (Table 14).
- *Budgets.* 57848658 adds one idle extension: a solver that reaches its budget without passing the demonstrations, while no episode is queued, receives 68k more tokens once.
- *Prompts and helper library.* 57848658 uses a later prompt set and helper library. Its solvers must write a hand-off report for the refiner, and a wrap-up message also arrives 15 minutes before a wall-clock limit.

The two submissions therefore differ in the model and in how time is allocated, and their scores do not isolate either change. Table 2 reports the repeated runs, and Table 16 in Appendix A records the exact budgets, prompts and versions.

4.5 Evidence for two solvers and a refiner

The pipeline grew from a single-session harness. Its first version scored 70.69 pass@1 on the 120 public evaluation tasks (G4 hardware). A helper library, a compact task file and a reminder to check the test outputs before stopping raised this to 78.33. A later version raised the cumulative per-task output budget, including reasoning, from 100k to 1M tokens, with a four-hour task deadline, and scored 86.11 (Section 3). This was a sequence of harness revisions, not an isolated comparison of output caps. Passing every demonstration still did not guarantee a correct test prediction: in a later run, 114 of 120 first solvers passed all demonstrations, yet their single-candidate score was 82.50. This motivated attempts that could propose another reading of the task.

We compared ways to produce a second attempt on one server (INT3 model, 64 concurrent sessions), sharing the first solver within each paired comparison (Table 5). An *independent* second solver added 6.11 points. A sequential solver that saw the first solution and was told to differ cost more tokens for an uncertain score difference (+0.83, CI spanning zero), and a verifier that reviewed the first solution instead of solving scored 5.97 points below an independent second solver.

A refiner over the two independent workspaces added 2.92 points, and 4.58 in a fresh confirmation run. Neither comparison lost a test pair, although this did not hold in every later deployment run (Table 6). Some gains came from proposing a new interpretation even when the solvers agreed. A third candidate for the refiner gave the same aggregate score at 37% more tokens. These comparisons led us to use two independent solvers followed by an execution-capable refiner.

4.6 Evidence from schedule rehearsals

To develop the scheduler without spending Kaggle runs, we time-scaled a 240-task public rehearsal roster (two copies of the public evaluation tasks) onto a faster single GPU. The same configuration scored 88.82 in a Kaggle Save & Run rehearsal and 88.12 in simulation (Section 14); the measured wall-time ratio was 2.93, with stage progress times within 2–4%.

Table 6 reports the score after each stage. The second solver adds 5.0–8.5 points in every row; the

Table 5: Ways to produce the second attempt (ARC-AGI-2 public evaluation, 120 tasks, pass@2 with partial credit; task-level bootstrap 95% CI). The initial experiment's first solver supplies one candidate and scores 82.50 (pass@1; unchanged when scored with an empty second slot). The verifier comparison and confirmation run use separate first-solver samples, scoring 81.53 and 79.17 respectively. Each difference is paired within its experiment.

Comparison	Δ points [95% CI]	Output tokens
Independent second solver vs. first only	+6.11 [+2.5, +10.4]	16.1M vs. 8.1M
Sequential (sees first) vs. independent	+0.83 [−2.9, +4.6]	17.6M vs. 16.1M
Verifier session vs. independent	−5.97 [−10.6, −2.1]	3.8M fewer
Refiner over two independent, round 1	+2.92 [+0.4, +6.3]	21.5M total
Refiner over two independent, round 2	+4.58 [+1.7, +8.3]	–
Three-candidate vs. two-candidate refiner	0.0 [−3.3, +2.9]	29.6M vs. 21.5M

Table 6: Score after each stage on a public rehearsal roster of 240 tasks: two copies of the public evaluation tasks (pass@2, partial credit). All rows use this roster, including the Kaggle Save & Run rehearsal; none is a hidden-task leaderboard score. "s0" is the first solver alone; "s0+s2" submits the two independent solvers' answers; "Final" adds extra solvers and refiners. G4 rows are time-scaled simulations of the 12 h budget; single runs. Harness versions: x34 has extra solvers for tasks that still fail the demonstrations at 80k tokens; x36 adds the solver hand-off report and the extra solver that reads both reports; x39 (submission 57848658) adds further extra-solver cases and the admission rule. No row uses the harness of 57747104, which has no extra solvers.

Run	Weights	s0	s0+s2	Final
Kaggle public rehearsal, x34	RL	83.26	89.24	88.82
G4 simulation, x34	RL	80.56	85.56	88.12
G4, x36 run 1	base	85.49	92.64	94.93
G4, x36 run 2	base	83.54	89.31	92.85
G4, x36 at 0.8× time	base	87.85	92.99	93.40
G4, x39	RL	80.83	89.31	93.54
G4, x39 at 0.8× time	RL	80.97	87.57	89.72

later stages add up to 4.23 points in simulation but lost 0.42 in the Kaggle rehearsal. Two runs of one configuration differed by 2.1 final-score points and 3.3 points after the first two solvers, limiting what we could infer from small differences in single runs. Extra solvers launched in otherwise idle capacity recovered 2.29 points in one run and 0.21 in another, about 0.13 points per million tokens across the comparison.

In replay on six simulated runs, the backlog estimate of x39 was 0.77–1.25 times the observed remaining work, compared with 0.74–1.63 for the fixed constants it replaced. The first-solver extension had an estimated gain of +4.63 points on 18 budget-limited hard tasks, but the confidence interval [−7.4, +17.6] was too wide to determine whether the extension helped or hurt. The RL weight patch changed a scheduling input: first and second solvers were about 25% shorter, including on tasks outside its training set. Section 11 reports the weight-only comparison on 513 held-out tasks, where output fell by 9–13% and single-attempt strict accuracy also fell.

4.7 Solver changes tested and not adopted

Changes inside the solver episode brought limited gains, so the solver stayed close to its early form. Against seven baseline rounds (mean 84.64 pass@1, round SD 3.1), a workspace helper library mined from the model's repeated code cut tokens by 5.3% [1.9, 8.8]; its score difference was uncertain (-0.66 [-3.09, $+1.63$]). Two successor libraries likewise showed no clear accuracy change ($+0.57$ and -0.04) and saved 3.1% and 5.1% of tokens; both submissions ship one of these libraries. Four tested changes all had lower point estimates (-1.03 to -5.20) and were not adopted: a 16k response thinking cap, a mid-episode "write the program now" reminder, permission to submit a second answer early, and a transposed task view.

Other controls changed measured behavior little. The xhigh, medium and low effort settings gave scores of 84.03, 83.79 and 82.55, using 68.8k, 67.5k and 67.8k output tokens per task. Test-time augmentation and a chunked grid format did not help (Table 14): recoloring introduced failures on otherwise easy tasks, and the chunked format helped only on constructed large-grid families, with no clear score effect under Codex.

Trace inspection suggested possible inefficiencies. An LLM annotation of 17 solver traces (1.33M tokens), selected with a bias toward long traces and short controls and with no independent human validation reported, classified 39% of reasoning as working out visually apparent properties such as shapes, line paths and symmetric copies; only one of eight wrong answers was attributed to a miss of this kind. In 14 of 22 reviewed sessions that passed the demonstrations but failed the test, the model had proposed the correct interpretation before rejecting it on unsupported grounds (Section 6). In a larger scan, long hand-computation passages occupied about a tenth of output. In offline replay at selected hand-computation points, adding rules to the prompt, brief or tool output left the heavy-computation rate similar (0.92 baseline vs. 0.88–0.94); interrupting reasoning with a user reminder reduced it to 0.15, measured on the next response only. Given the weak score evidence for solver changes, we concentrated subsequent work on scheduling and serving.

5 ARC-AGI-3: Interaction and Memory

In ARC-AGI-3 the agent must discover a game's rules and goal by acting. Score is relative human action efficiency (RHAE). A cleared level ℓ earns $s_\ell = \min\{115, 100(b_\ell/a_\ell)^2\}$, where a_ℓ is the agent's action count and b_ℓ the human baseline; an uncleared level has $s_\ell = 0$ [3]. For a game with L levels, the score weights levels by their index and applies a completion-based ceiling:

$$S_{\text{game}} = \min\left\{ \frac{\sum_{\ell=1}^{L} \ell s_\ell}{\sum_{\ell=1}^{L} \ell}, \quad \frac{100 \sum_{\ell:\, s_\ell > 0} \ell}{\sum_{\ell=1}^{L} \ell} \right\}.$$

Overall RHAE averages games equally; each game scores at most 100, despite the per-level cap of 115. Reasoning tokens do not enter the score but consume the nine-hour wall-clock budget, so we track tokens, actions and level completions separately.

Both selected submissions use the first-generation workspace harness and run 110 games concurrently with a 240k served context, dynamic compaction, a read-only reviewer and a click report. The second submission adds a toolkit, revised animation instructions, a final-call notice, game cards for the reviewer, and a switch to RL weights after six hours. Sections 5.1 to 5.4 describe

this deployment (Figure 3). Sections 5.5 and 5.6 report the development comparisons collected in Table 7; their budgets and context settings differ from the final deployment.

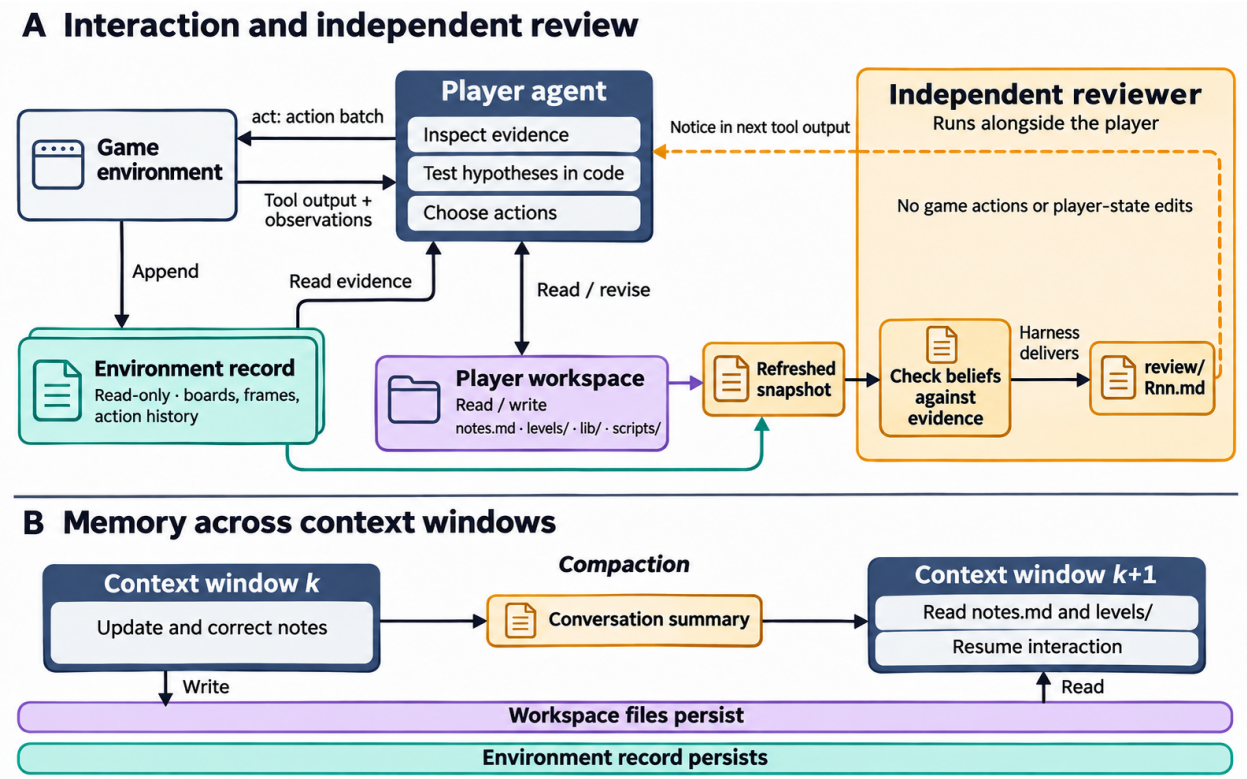


Figure 3: The ARC-AGI-3 interaction and memory loop. (A) The player acts through the game interface, reads the environment record, and maintains revisable notes and code. On stalled levels, an independent reviewer checks a refreshed snapshot while the player continues. The harness delivers its report and appends a notice to a subsequent tool output; the reviewer cannot act in the game or edit the player state. (B) Compaction summarizes the conversation while the record and workspace files persist. The player rereads its notes and level documents before resuming interaction.

5.1 The deployed agent

The harness has no built-in solver. One Codex agent (Section 9) plays each game from a workspace directory and reaches it only through an `act` command, which sends actions to the official environment gateway. `act do` plays a batch of actions in order and stops as soon as one changes the score. The actions are `ACTION1`–`ACTION7` and `RESET`; `ACTION6` is a click with board coordinates. The prompt advises batches of one or two actions to test a hypothesis and 10–20 once a rule is known.

After each batch the command prints the resulting 64×64 board as hexadecimal digits. The environment side also writes a `record/` directory that the agent cannot alter: the current board as text and as a 640 px PNG, one history line per action, and the final frame of every step. For a step with several frames it adds every frame as a PNG and a 512 px video that shows each frame twice. The prompt directs the agent to view the current image after every batch and to use code on the text record for exact positions. Under each click, `act do` also prints a click report: the color of the clicked cell and the extent of its same-colored region or, for a background cell, the nearest other cell.

The report exposes a click that lands one cell off its target and changes nothing.

A separate perception test favored per-frame PNGs at 768 px over the mp4 path (Table 7); the model still rarely identified a death animation as death.

The agent keeps its own state in `notes.md` (current beliefs, ruled-out hypotheses and the current attempt), `lib/` (reusable code), `scripts/` (one-off experiments) and `levels/` (one document per cleared level: goal, rules, clearing action sequence and what did not work).

Each run plays its games until the nine-hour wall-clock deadline, with a served context of 240,000 tokens, a Codex window of 235,000 and a thinking cap of 24,576 (Table 17). Codex runs at xhigh reasoning effort, and the configuration sets no output budget per game.

5.2 Memory across context windows

A game outlives many context windows. The deployed harness keeps Codex's compaction but lets the harness decide when it happens: the compaction point is $0.9P/n$ tokens, clamped to 100k–150k, for KV pool size P and n live games, with a note-writing reminder at 85k–125k (patches 0004/0005). With the deployed pool of 11,714,560 tokens, the 100k floor applies while all 110 games are live and the 150k ceiling once 70 or fewer remain, so survivors get longer windows as games finish.

The reminder asks the agent to update `notes.md` and correct wrong notes before its next action. Compaction preserves a summary in the ongoing conversation, and the prompt requires the agent to reread `notes.md` and every `levels/` document in full before it acts again. The notes remain revisable hypotheses.

An experimental second-generation harness instead rebuilt each window from a state packet and the agent's notes. In a two-hour comparison it cleared the same 16.9 levels per million generated tokens but generated fewer tokens and scored 21.71 against 38.68 (Table 4). Equal levels per token in this run does not establish equal capability at larger budgets. One source of its serving overhead was the handover: window messages sent as developer messages were folded into the leading system block and invalidated the cached prefix, and handing over through tool output restored the hit rate (96.7% vs. 88% in a two-game test).

5.3 The read-only reviewer

Both submissions launch a read-only reviewer on stalled games: a second agent that reads a live snapshot of the game's record and of the player's notes and code but cannot act or edit. Its report is written into the player's workspace, and a notice reaches the player with its next tool output.

Reviews begin two hours into the run for games stuck on one level for at least 45 minutes, with at most 24 reviewers at once and six reviews per game (Table 17). Players plus reviewers may not exceed 64, which delayed the first review to 4.27 h in the run measured below.

One 9 h run of the full harness measured the reviewer against a run without it: 124 reviews were launched, 96 delivered and all 96 opened. Six LLM annotators assessed 197 findings; of the 126 they could verify, 40% were classified as key, 34% correct but minor and 26% wrong. The annotations associated reports with help on 16 levels and misleading advice on 11. Of these, the run without a reviewer had not cleared six of the helped levels and had cleared five of the misled levels that the reviewed run did not: six levels gained and five lost. The reviewed run scored 81.23 versus 82.85

and used 9.6% of its output tokens on reviews. This single comparison, which also added SciPy, did not establish a score benefit. A variant that launched reviews after 20 minutes of stall instead of 45 was not selected: 39% of its reports arrived after the player had already cleared the level.

5.4 The selected configurations

The two selected submissions share everything described above, including the model package at the start of the run. The second differs in five ways.

First, it preinstalls a toolkit, `lib/arc.py`, of helpers that read `record/`: changed cells between two steps, animations frame by frame, objects grouped by kind, what the level's record has and has not tried, tracks of moving objects, a world-coordinate map and camera log for moving-camera levels, and a replay of the recorded actions through the agent's own simulator. In the second-generation harness, on three copies of each public game, the helpers for moving-camera levels and moving objects improved score over the earlier toolkit version at a 300k-token budget; the estimate at 1M tokens remained uncertain (Table 7).

Second, its prompt asks the agent to watch an animation on the first use of an action or of an action on a new kind of object, on a death or loss, on a level clear, and when the board changes much more than usual or unexpectedly. The first submission asks for a check whenever an action returns more than one frame.

Third, 15 minutes before the end, every game still playing receives a final-call notice: stop long thinking, note-keeping and tidying, and send the actions most likely to clear the current level in long batches. In a test run that sent it ten minutes before the end, the notice raised the action rate without a measurable level gain (Table 7).

Fourth, its reviewer reads game cards, short descriptions of other games: 25 static cards for the public games, written offline from recorded play, and cards that a card writer adds during the run after a player clears a level. A reviewer never receives a card of the game under review. It opens two or three cards of similar games and checks their recorded blind spots and breakthroughs against the current record.

Fifth, six hours into the run the server swaps the RL weights into the running model in place (Section 11). The intent was to clear easier levels efficiently during the first six hours and to spend the last three on the unfinished, harder levels with the RL weights; Section 11 reports the measured effect of the checkpoint. Figure 4 shows the schedule of this submission.

We retained these additions based on the mechanism-level observations above and the toolkit's low-budget result, while treating their effect at the full deployment budget as unresolved: run-to-run variation, including a per-game score-difference SD of 19.2 in repeated runs (Section 3), limited what these comparisons could show. The selected public scores are 75.56 and 75.44 (Table 2; settings in Appendix A). Because the entries differ in several settings, this 0.12-point difference describes the submissions rather than an ablation of any one component.

5.5 Choosing the agent

Three published-style ARC-AGI-3 harnesses (Duck, Tycho and NOOA), run with the same model on the 25 public games for six hours, scored at most 17.73 mean RHAE at medium effort and less at

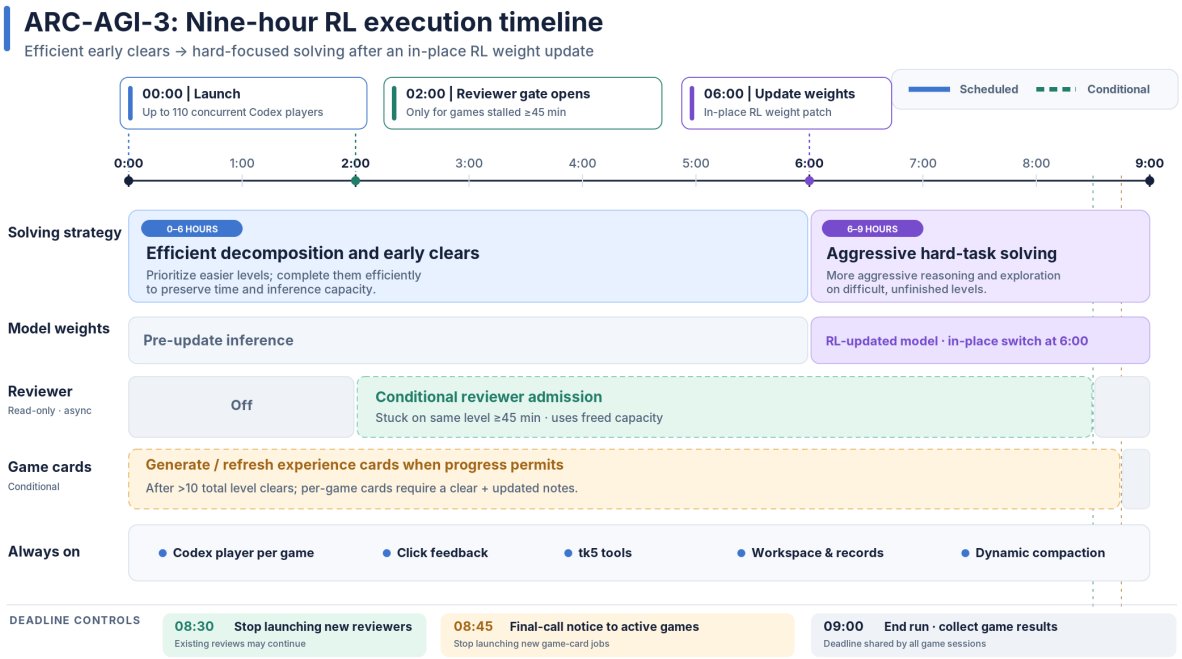


Figure 4: Schedule of the second ARC-AGI-3 submission (57846476) over its nine-hour run. Up to 110 games start together. From 2 h, a read-only reviewer is admitted for games stuck on one level for at least 45 min. At 6 h the RL weights are installed in place. The intent was to clear easier levels efficiently in the first six hours and to spend the last three on the unfinished, harder levels. New reviewers stop at 8 h 30 min, the final-call notice goes out at 8 h 45 min, and the run ends at 9 h. Solid boxes are scheduled; dashed boxes are conditional.

xhigh. A plain Codex agent with a workspace later scored 69.73 under a changed protocol (Section 9 and Table 7).

In budget sweeps, a tuned Duck harness led at 150k tokens per game, where Codex needed 1.6–2.3× the tokens per level and spent 13–17% of its tokens on compaction and re-orientation; the Codex arms led at 800k. A two-hour Kaggle run generated 25.0M tokens for 110 games, suggesting an average budget on the order of 1M tokens per game over nine hours. The higher-budget comparisons therefore supported using Codex for the competition deployment.

5.6 Relating tokens, actions and score

Neither output-token count nor turn count alone is a reliable proxy for RHAE. In an early six-hour comparison, reducing the served context from 262k to 120k tokens changed the score from 79.70 to 78.60 while using 65% of the actions and 72% of the tokens. Reasoning per response was similar (1,853 vs. 1,926 tokens); what changed was reasoning per *action*, 982 vs. 2,079 tokens. With less prefill work, the run completed 33% more turns. A model pruned to 256 experts with REAP (Section 7) scored 63.70 (−16.0; the separate comparison in Table 9 measured a larger loss) and batched about 30 actions per command, compared with 9 for the full model. In a separate effort comparison, medium beat low (Table 7). These runs do not isolate reasoning per action as the cause of the score differences.

Table 7: ARC-AGI-3 development comparisons (public 25 games unless noted; single runs; CIs are paired game-level bootstrap 95%). G4 runs are 9 h simulations of the Kaggle budget.

Comparison	Result
Harness bake-off, 6 h	Duck 5.46, Tycho 10.24, NOOA 17.73; xhigh 30–76% worse
Duck vs. Codex	36.1 vs. 21–23 at 150k tokens/game; 57.80 vs. 64.69–69.99 at 800k
Context 262k→120k	79.70→78.60 with 65% of actions; reasoning/action 982→2,079
Pruned to 256 experts (REAP)	63.70 (−16.0)
Effort medium vs. low	+20.97 [+11.23, +32.43]
Add-ons judged by tokens	1.04–1.18× actions/level; efficiency 0.971→0.886
Gen-2 vs. gen-1 harness, 2 h	21.71 vs. 38.68; both 16.9 levels per 1M tokens
Reviewer and SciPy, G4	81.23 vs. 82.85; 26% of verifiable findings labeled wrong
Moving-camera toolkit	+4.5 [+0.6, +8.2] at 300k; +2.7 [−0.5, +5.9] at 1M
Animation format	per-frame PNG 18/24 correct vs. mp4 11–15
Final-call notice, G4	actions/min 2.1→8.7; level gain not measurable

Harness additions first judged by tokens per level (tail reminders, nudges, recaps, trimmed windows) cleared as many levels at 1.00–1.06× the tokens per level but spent 1.04–1.18× the actions; action efficiency fell from 0.971 to 0.886 in one arm and from 0.962 to 0.851 in another, lowering RHAE. Conversely, on levels where Qwen spent over 300k tokens, 70% of its output fell in stretches of at least 50k tokens without an action; some such stretches contained legitimate offline search, so the count alone does not measure wasted computation.

A separate seed-collection study over 183 levels ran on development hardware with non-QAD W3A16 weights and the second-generation harness. Successful hard-level attempts used 0.58× the baseline actions but often required 200k–500k tokens, and 84.5% of cleared attempts already earned the maximum credit, so further score improvement there depended mainly on clearing more levels.

6 Developing the Harnesses with a Human in the Loop

Neither harness was designed in advance. Both evolved over two months through a human-in-the-loop form of recursive self-improvement (Figure 5). LLM agents read the system's own recorded trajectories (traces), located failure mechanisms, proposed changes and implemented them. The author chose directions and hypotheses, designed the comparisons and decided what to keep. This section states who did what, how the agents' findings were checked, which changes were kept or reverted (Table 8), how the loop appears in the submission history (Figure 6), and what another team could reuse.

6.1 Division of labor

LLM agents did most of the reading, counting and implementation. They read complete traces rather than samples, counted a suspected mechanism across all episodes before anything was changed, and implemented proposals behind flags so that a paired comparison could switch them on and off. The author chose what to try, stated the hypothesis a comparison should test, fixed the protocol and the measure, and decided when to stop.

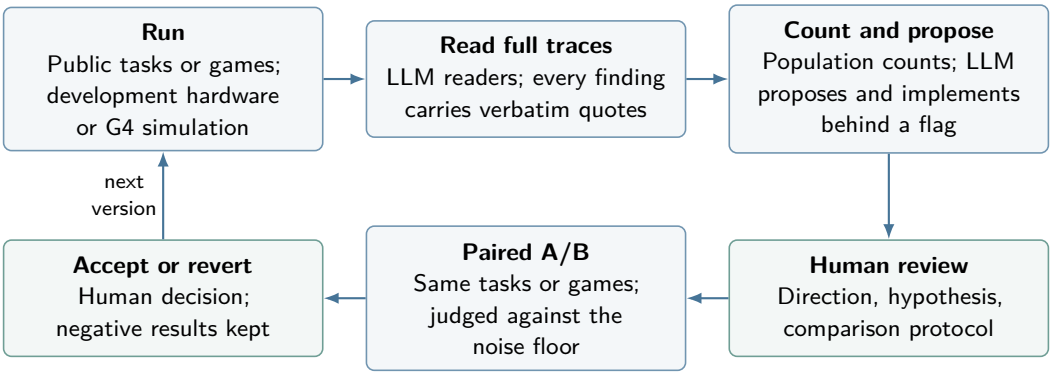


Figure 5: The development loop behind both harnesses. Blue steps were carried out by LLM agents or the running system; green steps by the author. A finding from a few traces was counted across all episodes before a change was proposed. A change was kept when a paired comparison supported it, when it removed a directly observed fault, or for a stated secondary reason such as lower token cost. G4 is the RTX PRO 6000 development machine, which can simulate the Kaggle time budget (Section 2).

Some directions recorded as the author's were themselves reactions to an LLM finding. For example, an LLM reading ARC-AGI-3 traces reported that the game-playing agents often clicked beside an object, saw no change and concluded the object was inert. Clicks that changed nothing were 8.6% and 5.3% of clicks in two 9 h runs, against 2.1% for other actions. The author proposed reporting what each click hit, which became part of the selected ARC-AGI-3 configuration (Section 5).

Other directions recorded as the author's include moving ARC-AGI-2 to the Codex runtime, the two-attempt pipeline, the verifier comparison, the requirements for the second-generation ARC-AGI-3 harness, the rule to change only the solver side of the ARC-AGI-2 harness during its late iteration, and the robustness requirement behind the final ARC-AGI-2 scheduler. Systems ideas followed the same pattern. The author asked whether KV entries that a session did not currently need could live in host memory; a model-assisted literature search found that HiSparse already did this, and it became the basis of our sparse-attention serving (Section 8).

6.2 Checking what the readers report

A finding passed three checks before it led to a change: of its quotations, of its interpretation, and of its frequency across all episodes.

Traces are cut by level into segments of at most 200k characters with numbered lines. A reader is a small model that receives one whole segment in a single tool-free call and attaches a line number and a verbatim quotation to every finding. A script checks each quotation against its source: in the first ARC-AGI-3 reading pilot, 99.8% of 5,771 quotations matched, and the remainder were paraphrases or splices. This check validates quotation fidelity only. It does not validate the reader's classification or causal explanation, so interpretations went back to a stronger model against the original text, and key numbers were checked by hand.

Reading was cheap enough to be exhaustive. A full segment of about 100k tokens took one call and 118 s; letting an agent page through the same text took 16 calls, 9.4 minutes and 2.34M input tokens.

For failure analysis, a reader first examined a failing run without seeing its successful counterpart, then compared it with the successful run or the gold answer. The analysis used 223 calls ($163.62) on 75 selected contrasts: 33 initial ARC-AGI-3 pairs, 20 Kaggle deployment pairs and 22 ARC-AGI-2 pairs. The ARC-AGI-3 pairs contrasted fast successes with long stalled runs; the ARC-AGI-2 pairs emphasized programs that passed the demonstrations but failed the test. The reader classified only one of the 75 failing runs as missing the deciding information. In 14 of the 22 examined ARC-AGI-2 solver failures, it identified a correct interpretation that the solver had considered and rejected for reasons not supported by the examples. Selection was purposive, and the second reading had access to the outcome. These are qualitative annotations of the inspected traces: they suggest failure mechanisms to test and do not estimate population-wide error rates.

For the frequency check, the first ARC-AGI-3 pilot read 41 segments in full, then counted its four candidate blind spots over 1,197 levels from seven rounds.

6.3 Changes kept and reverted

Table 8 lists representative changes from both tracks, chosen to show the range of outcomes rather than only successes. The changes that moved the score changed what the agent could keep or how long it could work: state kept in the sandbox, independent attempts, and removing an infrastructure fault. Most prompt, tool and reviewer proposals derived from trace reading changed tokens or actions but not the score beyond the noise floor (Section 3); Table 14 lists further cases. The component sections describe most of them in full (Sections 4, 5 and 9). One outcome needs a note: the second-answer prompt did not resolve a gain in a single session, and both selected ARC-AGI-2 submissions use it on the second solver only (Table 16).

6.4 The loop in the submission history

Figure 6 shows the public score of every submission over the two months, which is the output of the loop over time. The labels identify deployment milestones, including serving changes and harness revisions. On ARC-AGI-2, the first labeled point (48.89) ran an agentic harness on Qwen3.6. The first Qwen3.8-Flash-Next submissions (55.28) were throughput-limited: a 9 h rehearsal of that configuration completed only 155 of 240 tasks. The next version kept the same agentic harness and added host-memory collectives on the L4s, reaching 77.22. A later Codex configuration with two independent solvers and a refiner scored 82.64, followed by 84.72 with the distilled model and agent-trace drafter. On ARC-AGI-3, the Codex arm, HiSparse serving, 110 concurrent games and the server-side thinking budget each set a new best.

Three kept rows of Table 8 are visible as submissions: the two solvers and a refiner (82.64), the idle-timeout change (78.47 to 84.31) and the click report (75.56). Changes between versions can involve several components, and identical versions also vary across runs: five scored runs of the selected ARC-AGI-2 base notebook ranged from 84.31 to 87.22 (Table 2). This history records progress; it does not estimate the isolated contribution of each change, which is the job of the paired comparisons in Table 8 and the component sections.

Table 8: Representative changes produced by the loop. Parentheses give the origin recorded in the project log, for the finding or for the change. Paired results are on the same tasks or games, and intervals are 95%. "a vs. b" gives the score with the change first; "$a \to b$" gives the value before and after. ARC2 and ARC3 abbreviate ARC-AGI-2 and ARC-AGI-3.

Track	Finding (found by)	Change	Paired result	Outcome
ARC2	Hand-off reports lost the state of unfinished reasoning (analysis of the first agentic harness)	Keep programs, checks and notes in the sandbox	pass@2 72.78 → 81.11 at equal tokens	kept
ARC2	114 of 120 first solvers passed all demonstrations, yet pass@1 was 82.5 (analysis)	Independent second solver, then a refiner (author)	+6.11 [+2.5, +10.4]; refiner +2.92 [+0.4, +6.3]	kept
ARC2	Same finding	Verifier session in place of the second solver (author)	−5.97 [−10.6, −2.1]	reverted
ARC2	Half of tasks re-implemented BFS, components and flips (8 readers, 72 transcripts)	Toolkit mined from helpers the model wrote in 240 sessions	+0.57 [−2.04, +3.11]; tokens −3.1%	kept for cost
ARC2	14 of 22 failures had considered and rejected the correct reading (contrast)	Second answer from an alternative reading	one session with the second answer +1.81 [−0.12, +3.78]; first answer −0.04; equal tokens	kept in part
ARC2	2,597 resends after 300 s idle timeouts in a Kaggle log (log analysis)	Idle timeout raised to the episode wall-clock limit	Kaggle 78.47 → 84.31 (not paired)	kept
ARC3	Origin not recorded	Compact the context at every cleared level	32.08 vs. 36.20 RHAE (25 games, 90 min); three paired readers traced wrong rules carried through summaries	reverted
ARC3	Stalled players spent 70% of tokens in ≥50k-token stretches without acting (first reading pilot)	Action expectations, two-zone notes, helper library	actions per level 0.86–0.91×; RHAE 66.1–67.7 vs. 68.2	not adopted
ARC3	The Duck harness cleared levels with fewer tokens at small budgets (harness comparison)	Six rounds of its add-ons on the Codex harness, chosen by tokens per level	actions 1.04–1.18×; action efficiency 0.971 → 0.886	reverted
ARC3	8.6% / 5.3% of clicks changed nothing vs. 2.1% of other actions (LLM reading)	Report what each click hit (author)	in selected submission 57678190, public 75.56 (not paired)	kept
ARC3	Players stuck on one level for a long time	Read-only reviewer	81.23 vs. 82.85 (G4, 9 h; the run also added SciPy); 26% of verifiable findings labeled wrong	kept, net ≈0

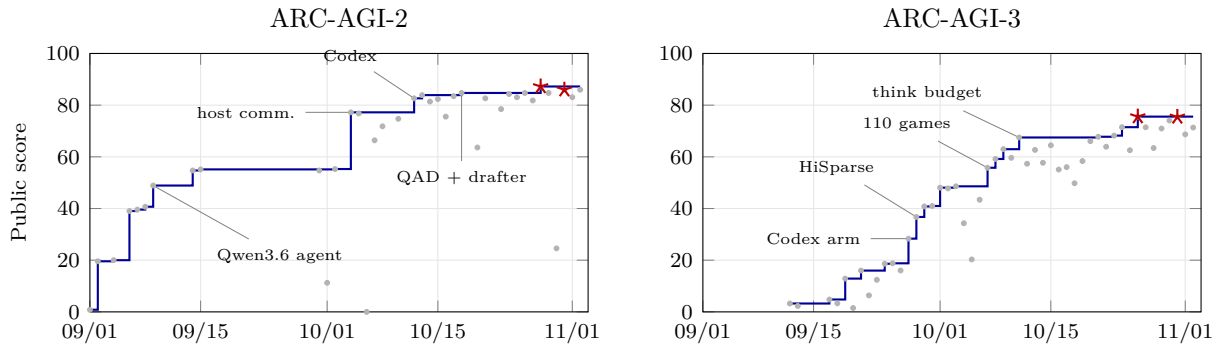


Figure 6: Public scores of every scored submission before the deadline (grey), the best score so far (line) and the selected finals (stars). Labels identify milestones in the development history (host comm.: host-memory collectives; QAD: the distilled model). Each point is a single run; run-to-run variation and simultaneous changes between versions prevent causal attribution from the timeline alone (Table 2).

6.5 What another team can reuse

Another team can reuse the reading setup above (one tool-free call per segment, line-numbered verbatim quotations, a script that checks them) and four practices.

- **Finding a mechanism is not predicting a gain.** The readers were reliable at locating where agents lost time, and their quotations could be checked. Their proposals were not reliable predictors of score: most removed the observed behavior and still tied the baseline. In the first ARC-AGI-3 batch, the proposals cut actions per level to 0.86–0.91× while RHAE stayed at 66.1–67.7 against 68.2. The baseline's action efficiency was already 0.94–0.97, so fewer actions could add at most 2–4 points, and the number of cleared levels did not rise. We therefore treated every proposal as a hypothesis to test, not as an improvement.
- **The measure chooses the change.** Selecting ARC-AGI-3 add-ons by tokens per level favored changes that spent more actions, which RHAE penalizes quadratically (Section 5). Comparisons were moved to the competition metric and to equal-cost settings.
- **Count before changing.** A mechanism seen in a few traces was counted over all episodes first; several plausible stories did not survive the count, and the retractions in Section 12 came from the same check.
- **The human sets the stopping rule.** Paired comparisons against the noise floors of Section 3 decided adoption. Changes that did not clear the floor were reverted or kept only for a stated secondary reason, such as lower token cost.

7 Model Compression and Adaptation

All four selected submissions serve one model package (Tables 16 and 17). It is Qwen3.8-Flash-Next after three steps (Figure 7). First, expert pruning keeps 320 of the 512 routed experts in every layer, selected by a gate-weighted statistic we call gEAN. Second, quantization stores the kept experts as 3-bit integers (INT3) fitted with GPTQ [10] and runs them with FP8 activations (W3A8); the dense layers stay in FP8 or BF16. Third, quantization-aware distillation (QAD) freezes the 3-bit

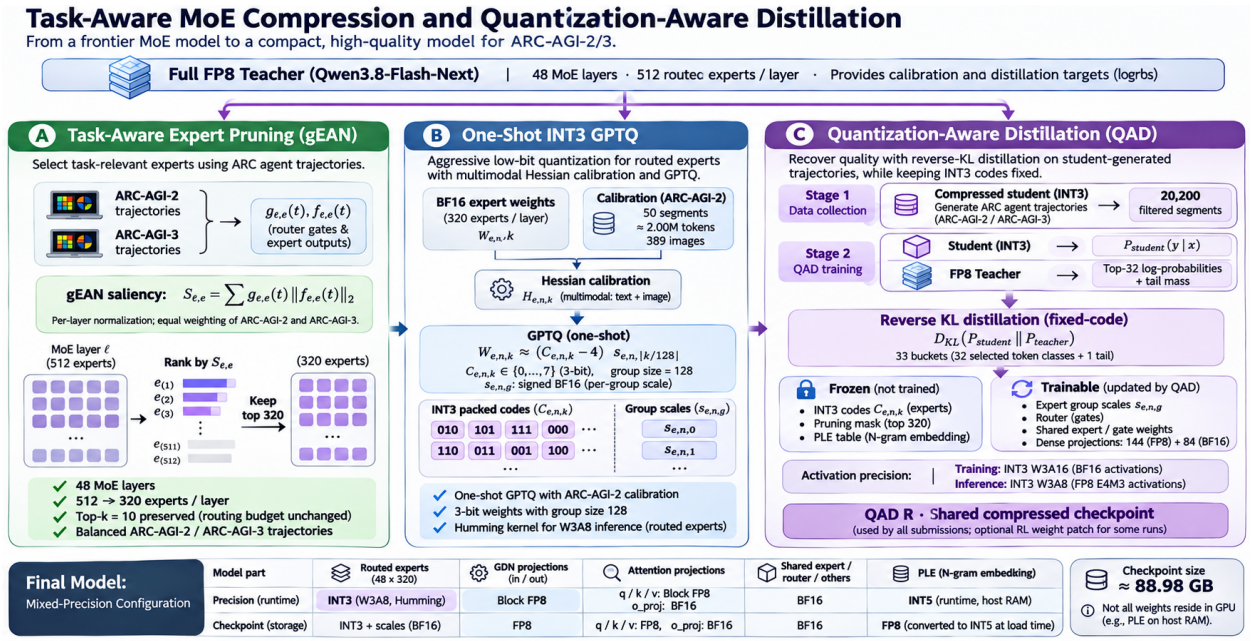


Figure 7: The compression pipeline of the shipped model package. (A) gEAN ranks the routed experts on ARC-AGI-2 and ARC-AGI-3 agent trajectories and keeps 320 of 512 in each layer. (B) GPTQ quantizes the kept experts to 3-bit codes with one BF16 scale per group of 128 weights. (C) Quantization-aware distillation keeps the codes fixed and trains the continuous parameters against the FP8 teacher with reverse KL on trajectories generated by the compressed student. Bottom: the precision of each part of the package. The row for the n-gram embedding (PLE) table shows the ARC-AGI-3 setting; the ARC-AGI-2 submissions hold that table in NVFP4.

codes and trains the scales, router, dense projections and shared expert to match the unpruned FP8 model on the student's own trajectories. The package is 88.98 GB against 185.5 GB for the FP8 checkpoint. Of that, 51.2 GB is the n-gram embedding table, which stays in host memory, and 28.3 GB is the routed experts.

The backbone must share limited accelerator and host memory with many concurrent agents. In the configurations tested (Table 9), aggressive expert removal caused larger task-score losses than weight quantization. We therefore retained more experts where memory allowed, and protected the dense layers and activations.

7.1 Backbone

Qwen3.8-Flash-Next [7] has 176B parameters, of which 51B are the n-gram embedding component (PLE), and about 6B are active per token. Its 48 layers interleave 36 Gated DeltaNet (GDN) recurrent layers [11] with 12 Qwen Sparse Attention (QSA) layers; each mixture-of-experts (MoE) layer routes a token to 10 of 512 experts plus one shared expert. We keep the PLE lookup table in host memory, so its weights do not occupy accelerator memory; the remaining 125B parameters must be compressed.

The FP8 model scores 88.61 on the public ARC-AGI-2 evaluation at its highest effort and 77.78 at low effort, and zeroing the n-gram tables costs 17.5 and 14.6 points respectively (Section 3); we

Table 9: Compression measurements from separate experiments. A: ARC-AGI-2 public evaluation (pass@2, official partial credit, highest effort), and ARC-AGI-3 six-hour runs on 25 games; the two FP8 ARC-AGI-3 values are repeated runs. B: a separate ARC-AGI-2 precision comparison, with changes relative to its own FP8 baseline. C: the steps of the shipped package, with approximate teacher top-128 KL on generated tokens of 32 held-out traces, and repeated ARC-AGI-2 pass@1 measurements. Both distillation rows start from the INT3 row; the last row is the package in the selected submissions.

A. Initial task-score comparisons

Configuration	ARC-AGI-2 score	ARC-AGI-3 score
512 experts, FP8	88.61	40.86–47.96
n-gram tables zeroed	71.11	–
weights 4-bit, activations 16-bit (W4A16)	–	37.86
weights and activations 4-bit (W4A4)	–	35.52
256 experts (REAP), FP8	82.36	14.69
256 experts (REAP), W4A4	–	11.76
128 experts (REAP), FP8	13.19	–

B. Separate ARC-AGI-2 precision comparison

Configuration	Score	Δ vs. matched FP8
512 experts, FP8	89.9	0
512 experts, W4A16 (two-run mean)	90.5	+0.6
512 experts, W4A4	–	−3.6 ($p = 0.26$)

C. Distillation measurements

Configuration	Approx. KL_{128}	pass@1
Teacher rerun (floor)	0.032	–
320 experts (gEAN), FP8	0.071	–
+ INT3 experts, FP8 activations (W3A8)	0.108	80.21 (6 rounds)
+ QAD at fixed codes	0.079	82.20 (6 rounds)
+ QAD on own trajectories, reverse KL (shipped)	–	85.00 (5 rounds)

Panel B is an eight-arm run on the same public set as A; its FP8 baseline differs. In C, the six-round comparison and the final five-round row were measured on different days (same-day base for the last row: 81.63). The KL approximation and its rerun floor are defined in the distillation subsection.

therefore evaluate compression at the effort settings we intend to deploy, and the package keeps the tables.

7.2 Expert pruning: gEAN keeps 320 of 512 experts

The shipped package keeps 320 of the 512 experts in each layer. Our main reason to prune was to fit the model and session state in memory: with 4-bit experts on one 96 GB accelerator, 512 experts leave a key–value (KV) cache of 1.18M tokens, 320 leave 3.51M and 256 leave 4.30M. Pruning did not improve decoding speed at batch 1 (216.0→214.0 tok/s); small gains at larger batches were single observations.

Uniform pruning with REAP [12] to 256 experts costs 6.25 points on ARC-AGI-2; pruning to 128 collapses the output protocol (13.19). ARC-AGI-3 is more sensitive: in six-hour runs on the same 25 games, 256-expert models cleared 53–54 levels against 97–113 for 512 experts at similar

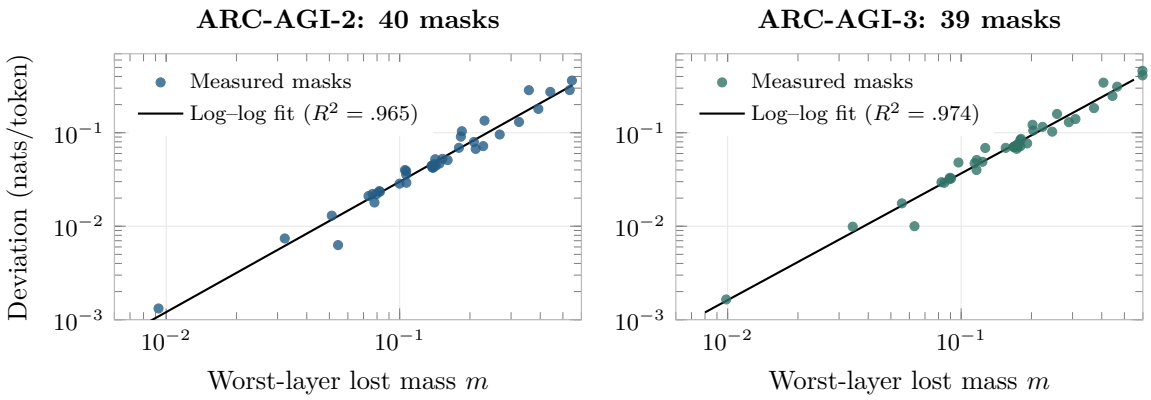


Figure 8: Pruning deviation (mean log-probability gap to the unpruned model on its own generated tokens) against m, the largest per-layer loss of gate-weighted expert-output mass. Points are measured masks retaining 128–448 experts per layer, including gEAN, REAP, other selection/calibration variants, and random selection. Fits use all complete observations except a repeated measurement of the incumbent mask. Deviation is measured on 2.20M generated tokens from 50 ARC-AGI-2 tasks and 2.60M from 11 ARC-AGI-3 games. Retained mass is evaluated on those same traces, so these fits describe an association within this sweep, not out-of-sample prediction.

action counts, a loss of 24–26 points against a 7.1-point difference between identical runs (Table 9, panel A).

For each expert j in layer ℓ, we accumulate $S_{\ell j} = \sum_x g_{\ell j}(x)\|f_{\ell j}(x)\|_2$ over calibration tokens, where g is the normalized top-10 routing weight and f is the expert output. Unselected experts contribute zero. We call this gate-weighted expert-output statistic *gEAN*. REAP divides the same total by the number of calls to the expert; gEAN retains the frequency contribution. We normalize scores within each layer and calibration domain, mix ARC-AGI-2 and ARC-AGI-3 scores equally, and keep the top-ranked experts in each layer. The ARC-AGI-3 calibration uses 12 games; 11 other games are held out for evaluation.

We compare masks by the mean difference $\log p_{\text{full}}(y_t) - \log p_{\text{mask}}(y_t)$ on tokens generated by the full model, in nats per token. We call this the *deviation*; it is a token-fidelity measure, not a task score. In a sweep retaining 128–448 experts per layer, gEAN had lower deviation than the matched REAP selection at every tested size. At 320 experts, deviation was about 58% below the earlier deployed REAP-320 mask: 0.022 vs 0.053 on ARC-AGI-2 and 0.030 vs 0.072 on ARC-AGI-3. The new, equally mixed REAP-320 mask had deviations of 0.044 and 0.070.

Across the tested masks, deviation was strongly associated with the largest fraction of S removed in any layer, with retained fractions measured on the evaluation traces themselves, separately from the calibration scores used to select experts. With $m = 1 - \min_\ell(\sum_{j\ \text{kept}} S_{\ell j} / \sum_j S_{\ell j})$, a log–log fit on 40 ARC-AGI-2 masks gives deviation $\approx 0.743\, m^{1.39}$ ($R^2 = 0.965$); 39 ARC-AGI-3 masks give $R^2 = 0.974$ (Figure 8). This within-sweep association motivated preserving total expert contribution; it is not a validated conversion from pruning statistics to task score. Two older task-score measurements provide only rough context: under a different harness, deviations of 0.072 and 0.286 accompanied a 6.25-point loss and protocol collapse. Expert-merging variants reduced deviation by at most 1% in these comparisons.

7.3 Quantization: 3-bit experts, protected activations and dense layers

The shipped package stores the routed experts in INT3, quantized with GPTQ [10], with one BF16 scale per group of 128 weights. The activation format of the experts depends on the hardware. The Kaggle L4 and RTX PRO 6000 deployments run them with FP8 activations (W3A8). Inference and RL on the B200 development servers use the same INT3 weights with 16-bit activations (W3A16). The GDN projections and the attention query, key and value projections are block-scaled FP8. The attention output projection, shared expert, router, embeddings and output head stay in BF16.

Quantizing the dense layers cost more than quantizing the experts. Measured as NLL on generated tokens against FP8 (rerun floor 0.0001), our first package, NVFP4 rounded to INT3 plus 4-bit dense layers, lost 0.242 nats per token, about 0.18 of it from the dense layers alone. GPTQ experts with BF16 dense layers lost 0.016 (round-to-nearest 0.058–0.065). Dense layers are a small share of the bytes but sit on every token's path, whereas a routed expert sees only a fraction of tokens. In a separate comparison, the shipped gEAN-320 INT3 package is within 0.026 nats of same-mask FP8 (KL 0.068 against a rerun floor of 0.024; top-1 agreement 84.8%).

The n-gram tables ship in FP8 (47.7 GiB of host memory) and are served in a smaller format: 5-bit integers (INT5) in the ARC-AGI-3 submissions (32.8 GiB, 0.0005 extra KL) and NVFP4 in the ARC-AGI-2 submissions (26.8 GiB, 0.0011 extra KL).

Before INT3 we evaluated 4-bit weights and activations with NVIDIA's NVFP4 checkpoint [13]; the shipped package uses neither for the experts. On ARC-AGI-2, W4A4 gave a 3.6-point lower estimate than FP8, with insufficient evidence to establish a difference ($p = 0.26$). On ARC-AGI-3 the differences among the 512-expert precision variants were no larger than the 7.1-point spread between FP8 repeats (Table 9, panels A and B). W4A8 has no native FP4×FP8 path; a CUTLASS prototype ran 4–6× slower than FP8, so we dropped it (Table 14).

7.4 Quantization-aware distillation at fixed codes

Quantization-aware distillation (QAD) trains the compressed student to reproduce the 512-expert FP8 teacher. The shipped package keeps the INT3 codes frozen and distills on the student's own agent trajectories with a reverse-KL objective.

Straight-through training of the 3-bit weights flips no code at a small learning rate (a flip needs half a quantization step); at a larger one it flips codes, recovers 10.7% of the gap after 100 steps and then loses ground (−4.4% at 300), while training only the scales keeps improving. We therefore freeze the INT3 codes and train what moves continuously—per-group scales, router, dense projections and the shared expert—by distillation [14, 15].

The training objective uses the teacher's top-32 log-probabilities plus one tail bucket: top-32 holds 99.83% of the mass at generated positions, and ranks 33–128 carry 0.4% of the KL. A separate serving probe, approximate KL_{128}, sums forward KL over the teacher's top 128 tokens. If a teacher token is absent from the student's top 128, the probe substitutes the student's 128th log-probability, underestimating that contribution. After pruning and INT3, the probe value is 0.108 on generated positions against a teacher rerun floor of 0.032; pruning and quantization account for roughly equal increments above that floor (Table 9, panel C).

In the first round, after 70M tokens, held-out top-32-plus-tail KL fell 24.7% (0.0793→0.0597).

In the serving probe, approximate KL_{128} fell from 0.108 to 0.079, recovering about 78% of the increment attributed to quantization. These percentages use different metrics and evaluation data. Training only the scales recovered 97.2% of the full training-objective gain, and the router alone 52.2%, which favored scale adaptation over changing integer codes. ARC-AGI-2 pass@1 rose from 80.21 to 82.20 (+2.0, 95% CI $[-0.5, +4.5]$; higher in 5 of 6 rounds), and DFlash acceptance rose 1.6%.

The shipped package is the reverse-KL arm of a second round. That round restarts from the undistilled INT3 package with the same trainable set and trains on the student's own agent trajectories [16, 17]: 1.15B tokens sampled from the INT3 student at temperature 0.7, including 723M supervised training tokens; the set is released (Appendix C.2). A reverse-KL arm and a forward-KL arm each trained for 1,000 steps, about 40% of the data, on the same top-32-plus-tail categories. Reverse KL scored 85.00 pass@1 against 82.31 for forward and 81.63 for the same-day base (reverse minus forward +2.69, CI $[+0.22, +5.36]$, 5 rounds).

On ARC-AGI-3, two full runs per arm put both directions within noise of the undistilled package (reverse -2.3 points, CI $[-7.6, +2.7]$; forward -0.8, CI $[-6.9, +5.3]$), and an ARC-AGI-2 comparison with another prompt variant did not establish a difference between directions. Data provenance also limits interpretation: 20.6% of the 723M supervised training tokens came from four task pools constructed from evaluation-task parents or families (`quality300`, `independent1000`, `nvarc_full`, and `synth108`). Task identifiers do not overlap. This percentage describes pool provenance, not a measured rate of near-duplicate examples.

7.5 Speculative drafter

A DFlash block drafter [9] initialized from the target's native MTP layer proposes tokens for rejection-sampled verification; the deployed version has eight layers and uses K=4 draft tokens at temperature 0.7. Appendix E covers its design, training and serving behavior.

8 Inference under Hardware Constraints

Serving throughput limits how much planned agent work can finish before the deadline (Section 3). The workload consists of up to 128 agent sessions that alternate long reasoning, tool calls and resumption over 50–100k-token histories. All serving work is in our fork of SGLang [18, 19], published at https://github.com/ycchen-tw/sglang-arc-2026; the selected submissions run its tag `kaggle-final`.

Two serving configurations shipped (Tables 16 and 17). Both run the same SGLang tree and the DFlash drafter of Appendix E with four draft tokens per cycle (K=4).

- *ARC-AGI-3: one RTX PRO 6000, 110 concurrent games.* One process holds the whole model (tensor parallelism of one, TP1). Target KV is stored in NVFP4. Session history lives in a host-memory pool of 11,714,560 KV tokens (the manifest's "logical KV tokens"); the GPU keeps only the sparse-attention index and a buffer of hot blocks.
- *ARC-AGI-2: four L4s without NVLink, 128 concurrent sessions.* One server spans the four GPUs (TP4). Attention runs as two data-parallel pairs, each tensor-parallel over two GPUs

Table 10: Measured serving effects. Single runs unless stated; each row compares configurations on the same hardware and workload. B200 rows come from development servers (Section 2). The HiSparse row changes both cache management and speculation. The weight-loading row excludes pre-reading and other startup stages. Cn denotes n concurrent requests.

Platform	Change: measured effect	Ratio
B200	GDN state slots 5→20 per game: uncached tokens/request 17,331→4,038	4.3× fewer
B200	NVFP4 target KV: token slots 11.07M→18.10M	1.64×
RTX PRO 6000	NVFP4 target KV: token slots 2.80M→4.50M	1.60×
RTX PRO 6000	HiSparse + session cache, 110 games; speculation on→off: 478→1,461 tok/s	3.1×
RTX PRO 6000	Direct NVFP4-pool decode: verify gather+attention 6.53→2.05 ms	3.2×
RTX PRO 6000	Further decode +6.5% (1,980→2,109 tok/s): public score 40.80→40.97	–
RTX PRO 6000	Extra checkpoint pre-read: weight-load stage 939–2,062 s→30.7 s	≥30×
L4×4	GPU-side consistency check: idle 29.5→1.9 ms/round	+13% tok/s
L4×4	DFlash K=4 vs none, C16: 407→828 tok/s	2.04×
L4×4	C32: 659→1,279	1.94×
L4×4	C64: 971→1,852	1.91×
L4×4	C96: 1,241→2,107	1.70×
L4×4	C128: 1,330→2,343 (context 8–13k)	1.76×

(DP2×TP2) with its own host-memory pool of 5,127,616 KV tokens. Each layer's 320 experts are divided 80 per GPU (EP4), and collectives pass through host memory (Figure 9).

Session state and the cost of each decoding step shaped both configurations; Table 10 lists what each change bought.

8.1 Session state: recurrent slots, KV capacity, NVFP4

In a hybrid model a cached prefix is reusable only if the recurrent GDN state at that point is also kept, so a session returning from a tool call needs both its KV entries and a GDN state slot.

With the default of five state slots per concurrent game on B200, 92% of prefill was recomputation (50.21M recomputed tokens against 4.23M new) while the KV pool sat at 26% occupancy. Raising slots to 20 per game cut uncached tokens per request from 17,331 to 4,038 and raised the hit rate from 80.3% to 96.0%.

A full KV pool is the other failure: at 0.999 occupancy the prefix cache was evicted entirely and the same workload took 1,096 s instead of 319 s. Sizing the pool from measured per-game growth predicted peak demand within 0.3%.

Storing target KV in NVFP4 [13] (E2M1 values with 16-element FP8 scales) raised capacity by 63.6% on B200 (11.07M→18.10M token slots) and 60% on the RTX PRO 6000 (sm_120), without a resolved difference in decode speed or perplexity in these measurements. Implementing this path exposed a scale-layout bug that our initial smoke tests missed (Section 12.3); SGLang's backend also supplied the wrong layout. Lower memory traffic alone did not ensure faster verification on sm_120: NVFP4 verify read 1.52 instead of 2.64 GB but executed 3.10× the warp instructions because of a long conversion chain.

8.2 Sparse attention with host-memory KV

The model's QSA sparse attention reads only a bounded set of KV blocks per step [20, 21], so the remaining history can live in host memory; this is how one RTX PRO 6000 serves 110 games. Our HiSparse-based path [8] keeps indices and hot blocks on the GPU, adds a session-aware prefix cache, and decodes directly from the NVFP4 pool.

The session cache finds a returning request by the session identifier that Codex sends and checks token by token that the new prompt extends the cached prefix; on a mismatch it recomputes the session from the start (Section 10). A parked session keeps one GDN state snapshot in host memory, so GPU state slots are needed only for running requests.

We keep QSA instead of converting the model to dense attention. With 64 concurrent requests, dense decoding is 14–16% faster at 8k context but ties at 64k, and on sm_120 dense attention took 1.48× longer per round at 80k context. Accuracy at equal 64k context was 55.9 (QSA) vs 52.0 (dense), a difference within noise.

On the RTX PRO 6000 with 110 concurrent games, the HiSparse/session-cache configuration generated 1,461 tok/s without speculative decoding, compared with 478 tok/s for the earlier configuration, which admitted 28 running requests at a time and speculated with the native MTP head. This 3.1× configuration comparison also changed speculation, so it does not isolate HiSparse's effect. The cache hit rate rose from 9.8% to 95.3%, and the median request queue fell from 82 to zero. In these one-hour runs the 11.7M-token pool peaked at 0.77 occupancy with no evictions; a 7.2M-token pool filled at minute 45, evicted 201 times, and then had a hit rate of zero.

In a separate kernel comparison, decoding directly from the pool cut target-verify gather plus attention from 6.53 to 2.05 ms per step. A later configuration raised decode throughput another 6.5% and changed the public score by +0.17 (Table 4).

8.3 Four L4s without NVLink

The ARC-AGI-2 machine adds two problems: the model and 128 sessions must be divided over four 24 GB GPUs, and the GPUs have no fast link between them. Figure 9 shows the resulting layout.

Each of the two attention pairs holds its own KV pool and serves 64 of the 128 sessions; the runtime pins every episode to one pair because its cached prefix lives there (Section 9). The split nearly doubles KV capacity: on B200 with FP8 KV, one pool of 10.53M token slots became two of 10.23M each. To free GPU memory at 128 sessions, each QSA layer's index is stored on one GPU of its pair instead of both (396→198 bytes per token per GPU, block selection bit-identical); the hot-block setting then rose from 2,304 to 3,392 at the same pool size, and decode throughput rose 2.0%.

On Kaggle, NCCL over peer-to-peer was 6–15× slower than over shared memory, and on a rented L4×4 pod the driver reported peer access while a GPU-to-GPU read returned all 262,144 test values wrong. We therefore run collectives through pinned host memory with FP8 payloads, verified bit-exactly on every toolchain (one compiler version miscompiled the FP8 round trip).

With this layout, steady state spends 71–77% of each speculative cycle in target verification; a cycle of about 139 ms splits into MoE 32.6, dense layers 27.9, communication 21.5, recurrent-state replay 12.7, drafter 15.1, and other work about 29 ms (one profile; the communication contribution

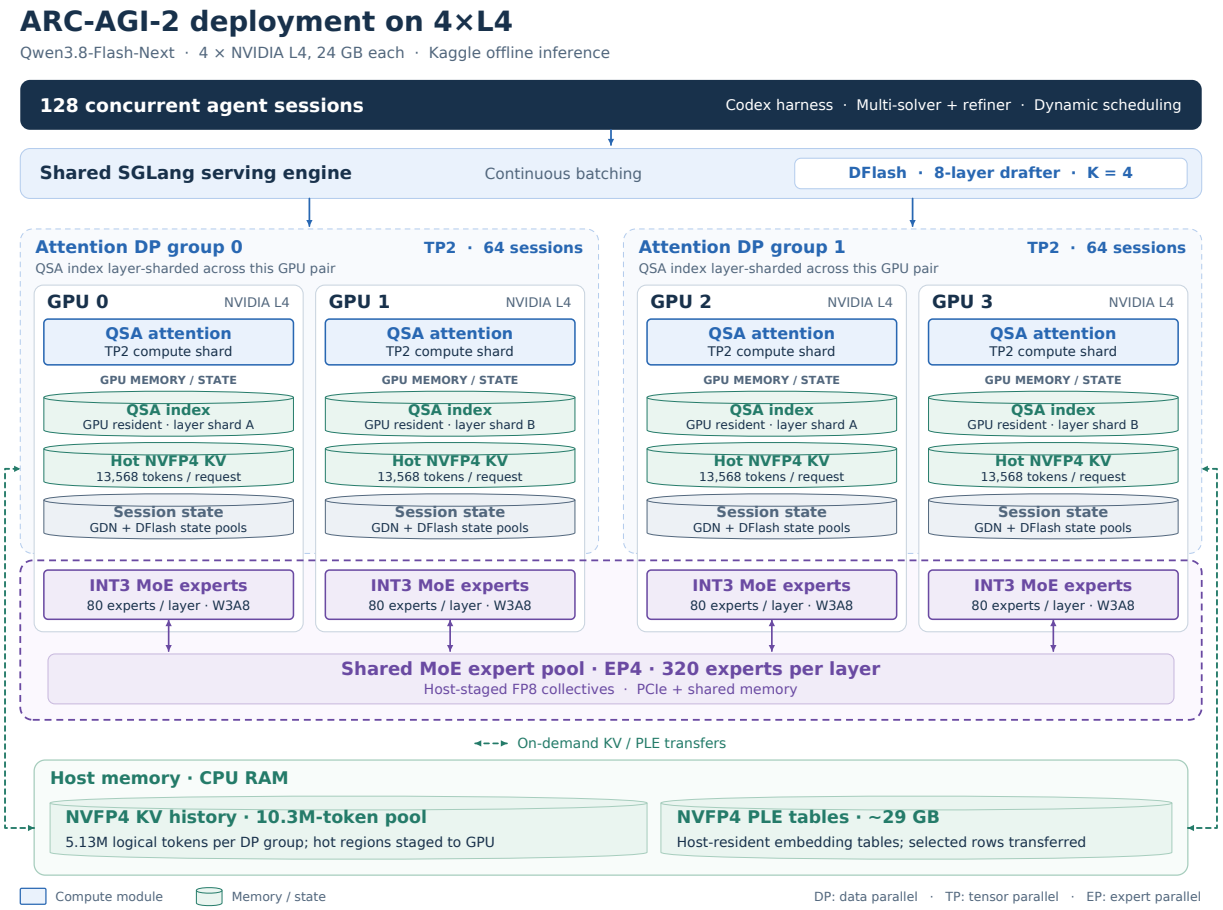


Figure 9: Serving layout of the ARC-AGI-2 submissions on four L4 GPUs. One SGLang engine serves 128 agent sessions. Attention runs in two data-parallel groups, each a tensor-parallel pair of GPUs that holds 64 sessions; each sparse-attention (QSA) layer's index is stored on one GPU of its pair. Every layer's 320 experts are split 80 per GPU, and the collectives between GPUs pass through host memory. Session history stays in a host-memory KV pool of 5.13M tokens per group, and only the blocks a request attends to are staged on the GPU. The n-gram embedding (PLE) tables also stay in host memory. Table 16 lists the settings.

varied across launches). The largest single fix was a `.cpu()` cross-device consistency check that left the GPUs idle 29.5 ms per round; comparing on the GPU cut this to 1.9 ms and raised throughput 13%. Smaller steps followed: a draft vocabulary subset covering 99.91% of emitted tokens, a pruned target output head (3.16→1.34 ms per step), and higher concurrency as memory was freed (1,425 tok/s at 64 concurrent sessions, 1,590 at 96, and 2,141 at 128).

8.4 Speculative decoding: acceptance is not throughput

Speculative decoding [22, 23] spreads the fixed cost of a decoding step over several accepted tokens, and that fixed cost is large here. An early FP8 profile on four B200s (tensor and expert parallelism of four, batch 32, 33k context, three MTP draft steps) recorded about 2,300 kernels per step, averaging 9 µs. The routed-expert matrix multiplications reached 91% of HBM bandwidth, while many smaller components took 13–40× their bandwidth-only lower bounds, so subsequent work

reduced host synchronizations and launches.

With the DFlash drafter at K=4, decode on the Kaggle L4×4 build is 1.70–2.04× faster than without speculation from 16 to 128 concurrent requests (Table 10). This sweep used simulated single turns at 8–13k context; it does not measure the speedup at the deployed 50–100k contexts, where the sparse-attention indexer costs more per verified row.

Acceptance length alone does not determine throughput. An MTP3-initialized DFlash drafter accepted 1.25× more tokens per cycle than the native MTP head yet ran at 0.99× its throughput, and only removing host synchronizations and recompilations from the cycle (+23.4%) turned acceptance into speed. Appendix E.11 reports how offline acceptance carried over to serving and why K=5 lost to K=4.

8.5 Measurement and startup

We report three quantities separately: steady-state decoder throughput, emitted tokens per wall-clock second over a full run, and task score. SGLang's generation-throughput metric counts interleaved prefill in its denominator, which biased it by 12–20%, growing with concurrency. Microbenchmarks misled twice: back-to-back CUDA-graph replay held the L4 at its 72 W limit and about 1.05 GHz instead of the deployed 2,040 MHz, so a kernel fix predicted to save 14.8 ms per step saved 2.35 ms in deployment; and a single end-to-end sample once showed a 21% speedup that repeats revealed as ±23–30% benchmark jitter. These failures motivated repeated or deployment-matched measurements when evaluating serving changes.

On Kaggle, pages read once were evicted mid-load, so loading weights took 939–2,062 s. A second read before launch (about 4 s) promoted cached pages and cut the subsequent *weight-loading stage* to 30.7 s; the full server stage took 184 s.

9 The Codex–SGLang Runtime Contract

Qwen3.8-Flash-Next supplies the model's reasoning capabilities. Codex is the agent runtime: it calls the model, executes tools, and edits workspace files. The task harness supplies the problem or game interface and schedules agent runs. SGLang serves the model locally and manages accelerator state. A proxy between Codex and SGLang enforces the agent's budget and closes its serving session when work ends. Figure 10 shows these components and the training data collected during execution.

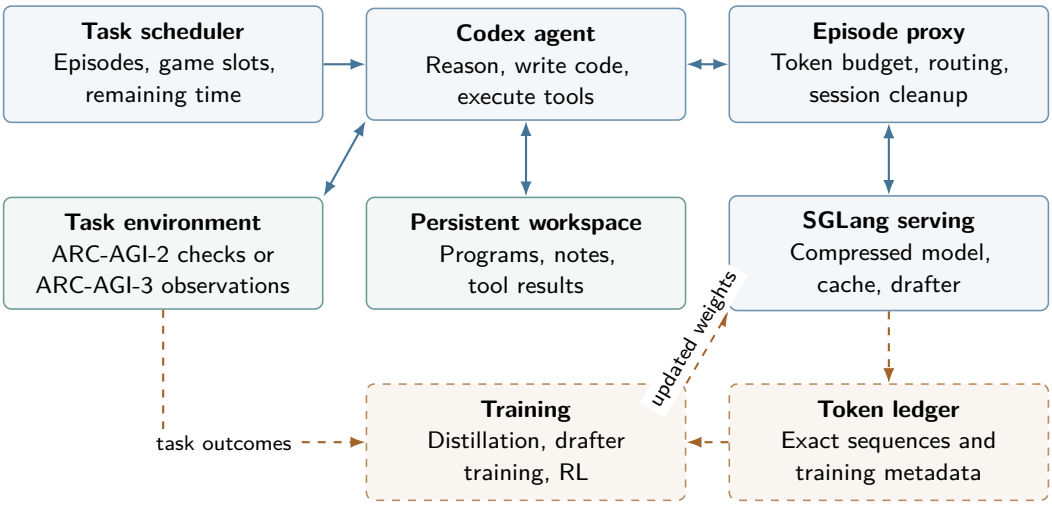


Figure 10: Components of one agent episode and the training loop around it. The task harness (scheduler and task environment) starts a Codex agent, which acts on the task through tools and retains files in its workspace; model requests go through an episode proxy to the serving engine. Dashed paths are enabled for training on development hardware (Sections 10 and 11). Competition execution does not require a live learner. Training fields in the ledger are optional; session replay can also be used during deployment.

All four selected submissions run Codex 0.154.0 with our patches (Table 19) against the same SGLang tree, in two builds. Build p7 applies patches 0001–0007; the 87.22 ARC-AGI-2 submission uses it. Build p8 adds patch 0008 and revises the video-token estimate of patch 0001; the other three submissions use it (Appendix A). All four enable session release, tool-preserving compaction and the pre-compaction reminder (Section 9.2). The two ARC-AGI-3 submissions also enable video input and harness-requested compaction. On Kaggle, every task or game runs under its own unprivileged user id (Section 9.6).

Execution units. An *episode* is one run of an agent in a task workspace: for example, an ARC-AGI-2 solver or refiner, or an ARC-AGI-3 player. A *turn* comprises model generation and its tool interaction. A serving *session* identifies the cached model state reused across requests. A *context window* is the active conversation retained by the agent. When the window reaches the compaction threshold, Codex asks the model for a summary and continues from it in a new window (*compaction*), so an episode may span several windows. Session identifiers and ledger segments (Section 10) track serving state and token history; they are not interchangeable with task or episode identifiers. In ARC-AGI-2 scoring, an answer *attempt* is a candidate output grid, not a separate agent episode. One episode can produce more than one candidate.

9.1 The agent client

Both tracks run the same agent: Codex 0.154.0 [24], talking to our SGLang fork [18, 19] through the Responses API. We use a general coding agent rather than a task-specific loop because its shell and file tools already provide the executable state that Section 3 found decisive. We build Codex as one static musl binary that is bit-identical on the B200 development machines and in the Kaggle image, whose glibc 2.35 rejects a glibc build.

Codex was written for a different server, and several mismatches appeared only in long sessions. We corrected them with patches to Codex (Section 9.2), changes to the SGLang fork (Section 9.4) and the proxy (Section 9.5).

9.2 Preserving context across compaction

Patches 0002–0006 keep the serving session, the tool definitions and the agent's notes consistent when Codex compacts; Appendix D gives the full inventory.

The largest measured effect came from patch 0003. Codex sent its compaction request without the tool schema, which the Qwen chat template puts at the head of the system block, so every compaction diverged from the cached prefix at about token 41 and re-prefilled the whole conversation. In one ARC-AGI-3 run, 286 compactions re-prefilled about 24.1M tokens, 63% of all new prefill. With the patch, which sends the tools of the interrupted turn, the prefix hit rate was 97.6%.

Patch 0002 sends a session id with every request and closes the session after compaction and at exit, because SGLang's session cache has no expiry. Patch 0004 lets the harness, rather than Codex's fixed threshold, decide when to compact; the ARC-AGI-3 harness uses it to scale the compaction point with the number of live games (Section 5). Patch 0005 injects a reminder once per context window so that the agent writes its notes before compaction, and patch 0006 drops the reminder at compaction so that copies do not accumulate. Patch 0001 adds video input, which the ARC-AGI-3 agents use to view animations, and patch 0007 truncates tool output by the tokenizer's count instead of bytes divided by four.

Build p8 corrects two errors in the context-length estimate that triggers compaction. It estimates the tokens of a returned video from the mp4 header instead of its byte size, and patch 0008 (`server_reasoning_included`) stops Codex from adding reasoning that the server's reported usage already counts, which had started compaction near 92k tokens (the Kaggle median) instead of 100k.

9.3 Comparison with other agent clients

Holding weights, serving flags, prompts and reasoning effort fixed, Codex scored 69.73 on the 25 public ARC-AGI-3 games and Qwen Code 3.10. Codex, on the Responses API, had 0 provider retries and 0.00% non-Latin output; Qwen Code, on the chat-completions API, had 154 retries and 0.48%. The two clients also differ in context management, so the gap cannot be assigned to wire format alone; we did not separate the two.

A second client, pi, scored 9.15 against Codex's 67.76 under identical conditions. Its gap traced to two client bugs: compaction inside a turn never fired, and the token estimate ignored thinking blocks, so contexts ran to the 262k window. Fixing them raised pi from 55.63 to 69.14, against 78.63 for Codex under the same prompts.

9.4 Server-side mismatches that failed silently

Four mismatches between what Codex sends and what SGLang assumes produced wrong behavior without an error; Section 12.3 collects further silent failures.

1. SGLang 0.5.20 split each assistant turn into three blocks by Responses *phase*. The model, seeing

"empty think, blank line, end", ended 4–10 of 60 probe turns without calling a tool (0/64 after the fix).

2. The Responses output budget defaults to context length minus prompt length, and Codex sends no `max_output_tokens`. Reasoning was cut mid-sentence, yet the stream still ended with `response.completed`, which Codex treats as success.
3. Codex sends no sampling parameters, so requests used the Responses class default T=0.7, not the chat default 1.0. A proxy now writes explicit values and the RL servers check them.
4. SGLang dropped images returned inside tool outputs, so image viewing never worked until patched. A revision then fixed long-context viewing (60–90k-token prefixes: 0/8→8/8).

9.5 The episode proxy

In ARC-AGI-2 and in RL rollouts, each episode talks to the server through its own proxy. The proxy rewrites `max_output_tokens` on every request to the remaining episode budget (reasoning included), so budgets are exact rather than enforced by killing a process after an overrun; a single response at the highest reasoning effort (xhigh) can exceed 57k tokens. Below 64 remaining tokens the budget counts as exhausted, because the server reserves two tokens and otherwise returns empty completions that Codex resends forever.

The proxy forwards only the inference routes the client needs. With data-parallel serving, each replica (rank) keeps its own session cache, so the proxy pins every episode to one rank: round-robin placement gave a 24.3% prefix hit rate (the 1/4 expected by chance), pinning 90–99%. A related bug had a concurrency semaphore wrap one HTTP call instead of a rollout; fixing it raised the hit rate from 27.7% to 94.1% and cut computed prompt tokens from 15.21M to 2.85M.

When an episode ends, the proxy aborts in-flight requests and closes the session; if Codex was killed, the harness closes the sessions that the proxy recorded as open. The ARC-AGI-3 competition notebooks instead use one shared proxy, which writes the sampling parameters (T=0.7, top-p 1) into every request.

9.6 Isolation and failure semantics

Every agent runs behind a filesystem fence that hides the task corpus. The Kaggle image permits neither bubblewrap (no user namespaces, no `CAP_SYS_ADMIN`) nor Landlock (`ENOSYS`); only seccomp works. The fence is therefore measured on the machine that will run, not configured: bubblewrap, else Landlock, else a per-game (per-task on ARC-AGI-2) unprivileged uid with the corpus hidden by `chmod 700` on its mount point's parent, and the run refuses to start if none holds. This rule came from a run that started behind a weakened fence, where two agents read their own game's source. On development machines, the RL rollout jail now also removes the network and reaches the model through a bridged Unix socket; the early RL runs audited in Section 11 predate this.

An audit of agent behavior went from 5 of 11 failing passes to 0 after two silent environment faults were fixed. One was a start-up warning from the Kaggle image's Python (`sitecustomize`) that led agents to try installing the missing package from the network; Appendix C lists the scorer and environment checks.

Timeouts must match the workload. With Codex's default 300 s stream-idle timeout, a compaction summary that the server held until completion (171–290 s at 128 sessions) triggered 2,597 resends of orphaned long requests; the cache hit rate fell to 0.1 and the submission scored 78.47. Setting the timeout to the 9 h episode wall gave 0 resends and 84.31 for the same configuration. The selected ARC-AGI-2 submissions keep this value (Appendix A).

A thinking cap (server setting `ARC_THINK_BUDGET`) bounds the reasoning in one response without invalidating the session cache: when a response reaches the cap with reasoning still open, the server closes it with exactly the template's own closing bytes and generation continues. At a stress budget of 1,024 tokens on 110 concurrent games, the server forced 1,236 closes; the 1,208 that had a following request all resumed the cached session exactly, with 0 mismatches. Tables 16 and 17 give the cap of each selected submission. In a two-hour Kaggle run without a cap, only 0.035% of outputs exceeded 24k tokens, so a cap at that level would have affected few responses in that workload.

10 TITO: Exact Trajectory Capture

Distillation, drafter training and RL all need the token sequence the model actually consumed and produced, with its sampling probabilities. The Codex agent does not supply it: it speaks the Responses interface, which carries text and no token ids, re-renders earlier turns as text, and re-sends them on every request. Re-tokenizing that text does not reproduce the sampled ids, and every divergence also forces the server to recompute state it already had. Our token-in, token-out (TITO) layer therefore keeps the authoritative record on the server, and serving and training read the same record.

TITO has two parts. *Replay* replaces the client's re-rendered text with the recorded token ids whenever a request continues a known session. The *ledger* writes those sequences to disk, with optional training fields, when the server is given a ledger directory; without one the server replays and writes nothing. The selected submissions carry models trained from ledger records: the distilled target model (Section 7) and the DFlash drafter (Appendix E) in all four, and RL weights (Section 11) in one submission per track.

Figure 11 separates token replay from reuse of computed session state.

10.1 Text transcripts are not trajectories

The client changes text in ways that are invisible in a transcript. Codex re-renders truncated reasoning and malformed tool calls lossily. It re-sends a compaction request verbatim after discarding the reply (five times in a row in one case). The chat template appends a closing `</think>` to a turn cut by the output budget.

In the deployed ARC-AGI-2 server, before the ledger existed, a re-rendered turn diverged from the cached sequence once per 6×10^4 generated tokens (1.63×10^{-5}; 1.71×10^{-5} with host-memory sparse attention), which hit 2.5% of requests and accounted for 9.6% of all new prefill; with the HiSparse cache (Section 8), which recomputed from the start after a divergence, the same events cost 50% of new prefill. For training the error is worse than the cost: a re-tokenized sequence assigns the policy loss to tokens the model never sampled.

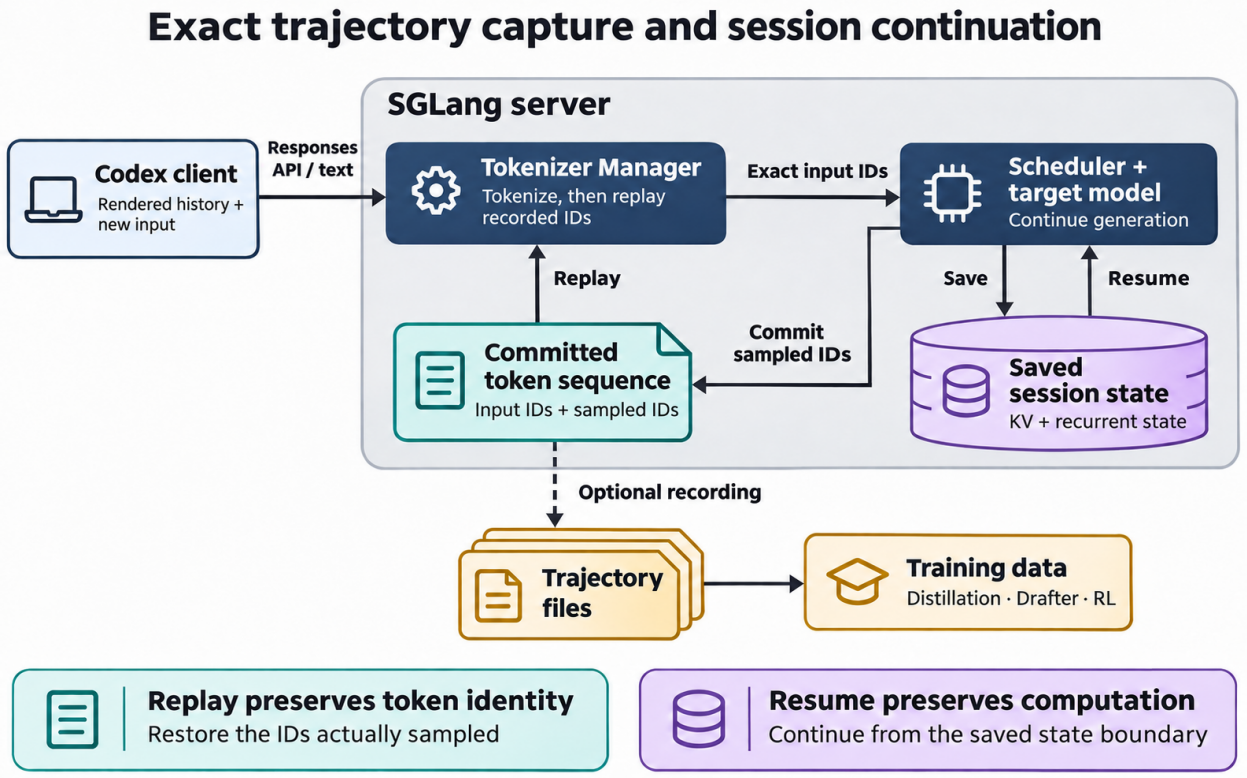


Figure 11: Exact trajectory capture and session continuation. After tokenization and before scheduling, replay restores the recorded token IDs rather than relying on the client-rendered history. Exact session resume separately reuses saved KV and recurrent state from the saved computation boundary, which need not include the last sampled token. Optional recording exports the trajectory for distillation, drafter training, and RL; training metadata can include sampling probabilities, media, expert routing, and weight versions. Deployment can use replay without writing trajectory files.

10.2 Session ledger and replay contract

The server keeps one committed token sequence per session, and that sequence, in place of the client's text, is the history the model sees. Replay and the ledger sit in the server's tokenizer process, after tokenization and before scheduling. Codex sends one session id per context window (Section 9), so the ledger holds one exact sequence per context window. A session's record is a series of *segments*; the requests in a segment extend the same committed sequence. Each request falls under one of five rules:

- A request whose rendered prefix matches the ledger is an *extension* and reuses the recorded ids.
- A turn the model itself produced is replayed from its sampled ids even when the client renders it differently (*own-last-turn replay*).
- A client retry or a template-inserted closure is a *rewind*: the committed sequence rolls back to the divergence point inside the last generated turn, and the segment continues.
- A genuine client edit opens a new segment.
- An aborted request is logged but never committed.

Replay makes the token ids exact. Exact session resume extends this to the computed state: it parks the recurrent and KV state at the end of a turn and continues from it instead of re-prefilling, so the matched-prefix path reuses previously computed positions.

Table 11 gives the evidence; its first two rows are acceptance tests. With exact session resume on real Codex traffic, recomputed positions per new token on turn extensions fell from 2.64% to 0.011% (1.1×10^{-4} as a ratio). Reuse also keeps the recorded expert routing valid, because the INT3 MoE kernel is not deterministic: probes that re-read 256 positions behind each continuation found the stored expert routing bit-identical in 51,000 (non-speculative) and 67,702 (speculative) positions, whereas recomputed positions choose different experts (716 of 765 and 255 of 255).

Own-last-turn replay removed the remaining rewinds. RL data preparation drops an episode when any of its positions was computed twice under different weight versions, and it stops the run when more than 2% of the episodes are dropped; we call this check the RL data gate. Without own-last-turn replay 13 extensions rewound and the gate dropped 8 of 96 episodes; with it the next turn's cache hit rose from 62.5% to 99.7% and its time to first token (TTFT) p90 fell from 3.6 to 0.59 s. The contract also holds across in-place weight updates (Section 11): 77 synchronizations under stress and real Codex load succeeded with 0.58–1.05 s pauses, and an RL replica with 44 updates recorded all 2,734 extension turns exactly, so all 192 episodes passed preparation with no cross-version recomputation.

The guarantee has limits. Genuine client edits and other fallback cases can still require recomputation; with exact session resume enabled, the ledger labels each turn as an exact resume or a fallback. Exact recording preserves the sampled trajectory and its serving probabilities; it does not by itself make a learner's recomputed probabilities numerically identical. We examine that separate train–serve issue in Section 11.

Table 11: TITO contract checks (upper block) and recording costs (lower block), measured on B200 with real Codex sessions (acceptance on FP8, resume and replay on the INT3 deployment model). An arrow gives the value before and after the change named in the row; in row 3 the change is enabling exact session resume. A rewind rolls the committed sequence back inside the last generated turn. The 2% gate drops RL episodes with cross-version recomputation and stops a run that loses more than 2%. TTFT is time to first token; non-spec and spec are speculative decoding off and on.

Check or cost	Result
Unit tests / injected mutants caught	115 pass / 29 of 29
Codex response ids joined to the ledger (3 ARC-AGI-3 games, 8 ARC-AGI-2 tasks)	676 of 676
Recomputed positions per new token on a turn extension	2.64% → 0.011%
Routing differences in re-read cached positions (non-spec / spec)	0 of 51,000 / 0 of 67,702
Rewinds; episodes dropped by the 2% gate (without → with own-turn replay)	13; 8/96 → 0; 0
Next-turn cache hit; TTFT p90 (without → with own-turn replay)	62.5%→99.7%; 3.6→0.59 s
In-place weight syncs; pause	77 of 77; 0.58–1.05 s
Sampled log-probabilities (full-batch decode)	−3.8% tok/s
MoE routing record	TTFT p90 +35%, +11 GiB host, 960 B/token
Top-32 distribution at concurrency 128 (first version → final)	−30% → −9% tok/s

10.3 What is recorded and what it costs

Every field is opt-in so that deployment pays only for replay. For training the ledger adds the sampled log-probability under the effective temperature, the top-32 distribution and its residual mass, raw media with their positions, per-layer expert routing (the top-10 experts in each of 48 layers), and the weight version of every token. The exporter rejects data whose recorded sampling settings differ from the learning recipe.

Costs are measured with runs that alternate the settings in ABBA order (Table 11). Recording ids alone has no measurable cost. Routing is the expensive field: decode is unaffected, but extension TTFT rises because routing is copied to host memory. The first top-32 implementation cost 30% of throughput through per-row device-to-host copies and Python lists; batching the copies and reusing the sparse-attention top-k kernel brought it to 9% at concurrency 128 and 0% at 64.

10.4 Collection at scale

One collection of about 19 hours on B200s (8 GPUs, then 4) produced 7.68×10^8 generated tokens from the ARC-AGI-2 and ARC-AGI-3 Codex harnesses, served by the unpruned FP8 model with the DFlash drafter: 3.94×10^8 with ids and media only and 3.74×10^8 with log-probabilities and top-32 distributions. The run had 0 lossy records and 0 writer errors, and every response id in the harness manifests was found in the ledger (0 of 56,028 and 0 of 76,588 missing). These data trained the DFlash drafter (Appendix E) and the distillation runs (Section 7).

The audit caught one more silent mismatch: a CPU emulation of the deployment image prepro-

cessor differed from the deployment path on 5,707 of 31,914 media items (by at most one intensity level), so samples are now built with the deployment processor itself.

11 Streaming RL for Long-Horizon Agents

Two of the four selected submissions carry RL weights. On ARC-AGI-2, one submission loads the checkpoint r4@30 at start-up. On ARC-AGI-3, one submission installs the checkpoint v8@20 after six hours of its nine-hour run. A name such as r4 or v8 identifies one training run, and @N denotes its weights after N optimizer steps. Each of these submissions also changes the harness relative to its corresponding base submission, so the submission-score differences in Table 2 are not weight-only comparisons.

Against its starting model, r4@30 gained 4.2 ± 1.7 points of strict accuracy on 60 held-out tasks at its 40k training budget. With the deployed solver on 513 held-out tasks, it used fewer output tokens and lost single-attempt strict accuracy. v8@20 changed the full-game score by $+1.5 \pm 2.7$ points at a 1M-token budget and by -1.5 ± 2.5 at 1.5M (Tables 12 and 13).

The RL system runs the Codex agent in the same sandbox, tools and serving engine used for deployment, then joins task outcomes with trajectories from the TITO ledger (Section 10). Section 11.1 defines the episodes, groups and objective; the later subsections cover the streaming schedule, train–serve fidelity and stability, and then the RL runs on each benchmark with their settings and results (Sections 11.5 and 11.6).

11.1 Training setup

An *episode* is one agent run on an ARC-AGI-2 task or from an ARC-AGI-3 level snapshot. We call such a level-start snapshot a *level seed*. A *group* contains independently sampled episodes with the same task or seed; their rewards provide a within-group baseline. Both selected checkpoints start from the distilled INT3 model in Section 7. The ARC-AGI-2 run, r4, trains LoRA adapters. The ARC-AGI-3 run, v8, directly updates 372 attention, GDN and shared-expert projections, using FP32 master weights and FSDP2. The source configuration calls this mode "full" training; it covers these projections and not every parameter of the backbone. The complete recipes are in Appendix B.

For ARC-AGI-2, r4 uses 90 tasks: 60 public evaluation tasks and 30 audited hard tasks. The reward is 1 only when every test output is correct, and 0 otherwise. An episode has a 40k output-token budget and a 64k context window; wrap-up begins at 36k output tokens or 58k context tokens and allows another 4k output tokens. Sixty other public tasks form the initial evaluation split. The later 513-task comparison excludes all r4 training tasks, but does not establish exclusion from earlier model adaptation or from related task families.

For ARC-AGI-3, v8 trains on 235 level seeds from 18 public games. Each episode stops at a level clear or the first context-window boundary. A clear receives an action-efficiency reward. Reaching the normal boundary without a clear receives a reward based on an LLM assessment of the agent's notes, capped by the estimated reward for eventually clearing the level; early self-triggered window changes and other unsuccessful stops receive zero. The window capacity is 235k tokens, with handoff triggered at 130k. Training ends before generation in the next window: this recipe trains the first-window behavior and handoff notes, not credit assignment over a completed multi-window

game. Five public games are held out from v8 training; two additional games have no training seeds and are reported separately in the run record. All of these games were available during system development. The r4 task pool and the v8 level seeds are released (Appendix C.2).

Both selected runs use masked token-level importance sampling, called `MIS` in the implementation. For episode i in group g, the advantage is $A_i = R_i - \bar{R}_g$, without division by the reward standard deviation. For a generated token t, let ℓ_{it} be the learner log-probability and b_{it} the recorded behavior log-probability. Their ratio and masked weight are

$$\rho_{it} = \exp(\ell_{it} - b_{it}), \qquad w_{it} = \rho_{it}\,\mathbf{1}[0.2 \le \rho_{it} \le 5]. \tag{1}$$

Both selected runs sample with temperature 1.0, top-p 0.95 and top-k 20. For these filtered rollouts, the ledger records the retained token set, and the learner replays that set when normalizing its probabilities. The ratio therefore compares learner and behavior probabilities on the same recorded sampling support. Let N_g count all eligible generated-token positions in group g before applying the ratio mask, and G the number of retained groups in an optimizer step. The loss is

$$\mathcal{L} = -\frac{1}{G}\sum_{g}\frac{1}{N_g}\sum_{i\in g}\sum_{t\in i} A_i\ \mathrm{sg}(w_{it})\,\ell_{it}, \tag{2}$$

where sg stops the gradient through the weight. Tokens outside the ratio band contribute zero; they are not clipped to a boundary value, and they remain in the denominator N_g. This reduction averages tokens within each group and weights groups equally. Prompt and tool tokens are excluded, groups with zero advantages are discarded, and neither selected run adds a KL penalty. Some earlier experiments discussed below use the BPO objective [25], which has a different weighting and masking rule.

11.2 From round barriers to streaming groups

Agent episodes vary from minutes to hours, so a schedule that waits for a whole round of rollouts waits for its slowest episode. In the streaming design (Figure 12), the learner takes an optimizer step as soon as enough completed groups are ready and publishes the resulting *weight version* to the live rollout servers, while the other episodes continue.

In a synchronous round, learning waits until the slowest rollout finishes. A round pipeline overlaps learning with the next round's rollouts but retains these group-wide gates. Streaming admits completed groups continuously [26–28]. The schedule is independent of whether an individual training episode spans one or several context windows.

We compared the three modes on one toy ARC-AGI-2 task set. Time per weight version fell from 15.6 minutes (synchronous) to 11.3 (round pipeline) and 6.6 (streaming), while generation throughput rose from 3.4k to 4.7k and 7.4k tokens/s. Streaming kept a sliding window of 64 groups in flight and triggered learning when 272 eligible episodes were ready. Staleness was capped at eight weight versions, and the observed lag was one or two versions. Weight updates preserved live sessions and caches, with pauses of 0.4–1.1 s; episode-specific cache salts prevented prefix reuse across separate episodes. These measurements concern the toy workload, not the final training recipes.

Completed groups contribute to a common optimizer step. In a numerical check, accumulating groups separately changed the gradient by 0.96% in relative L_2 norm, compared with 0.89% for the

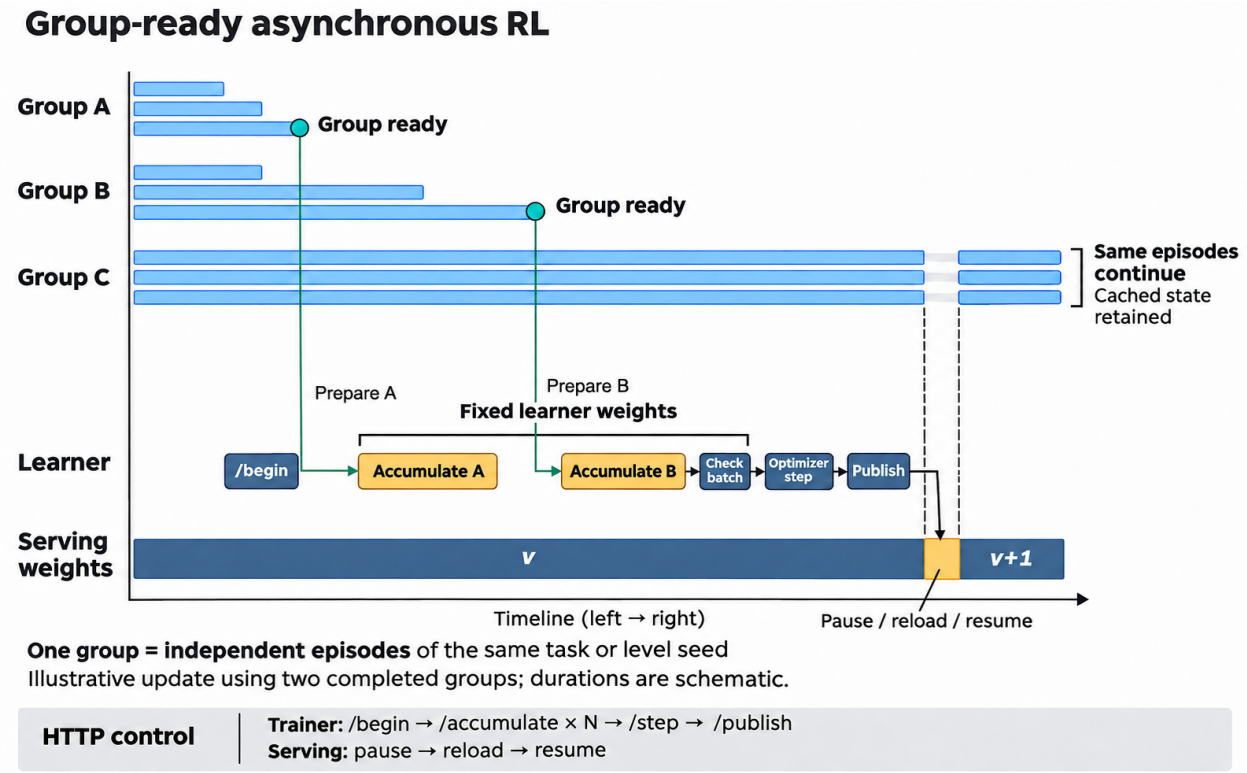


Figure 12: Group-ready asynchronous RL on development hardware. Each group contains independent episodes of the same task or level seed. Once a group finishes, its data are prepared and its gradients can be accumulated while other groups continue. In this illustrative update, groups A and B contribute to one optimizer step after batch checks; learner weights remain fixed during accumulation. Publication precedes a brief serving pause for weight reload. Group C continues with its session and cached state retained. HTTP calls control the trainer; prepared trajectories are read from shared files. Durations are schematic, not measured timings.

control perturbation. On the toy set at learning rate 10^{-4}, held-out reward rose by 0.064 at weight version 6 of a synchronous run ($z = 3.6$) and by 0.102 at version 18 of a streaming run ($z = 7.2$). These checkpoints have different update counts and do not isolate a policy-quality advantage from streaming. An earlier synchronous run at 2×10^{-5} changed reward by -0.004 (SE 0.022); only 9% of its bf16 weight elements changed.

Profiling the toy loop found the rollout servers idle 40–46% of the time, with about 2.1k small prefills per round and 75–130 ms of fixed overhead per prefill. Raising concurrency from 256 to 512 did not increase throughput, and DFlash added 19% at 256 sessions but 4% at 512, where verification became compute-bound.

11.3 Train–serve fidelity

Importance weights compare learner probabilities with the behavior probabilities recorded during rollout. Numerical differences between the trainer and the serving engine can change these ratios even when their weights match. Both selected runs therefore replay the recorded expert routing in the trainer (Table 18).

In one INT3 comparison between the trainer and a server at deployment precision, the mean $|\Delta \log p|$ is 0.0412, and 3.96% of tokens have a ratio outside $[0.8, 1.28]$, a diagnostic band narrower than the training mask $[0.2, 5]$. Replacing the server's FP8 KV cache with a bf16 cache lowers the mean to 0.0367, switching its MoE kernel to a Triton implementation lowers it to 0.0368, and both changes together give 0.0318. SSM-state dtype, DFlash, the decode path and TP2 showed no material effect in these checks. Re-rounding the trainer to imitate the server makes things worse: an A16 trainer is closer to the W3A8 rollout than an A8 simulation (0.0656 vs 0.0701), consistent with small input differences crossing quantization-bin boundaries.

Expert routing was measured in a separate comparison. Only 44.9% of positions select the same top-10 experts when the trainer routes on its own. Replaying the recorded routing [29] halves the mean $|\Delta \log p|$ (0.092→0.045), cuts the k_3 KL estimate from 0.065 to 0.0075, and cuts the share outside $[0.8, 1.28]$ from 12.0% to 4.5%.

Weight publication updates the live server in place. On FP8 the update loads in about 1 s and the changed parameters match the files bit for bit while the other 1,396 tensors are untouched. On NVFP4 the same call silently re-packed untouched experts and mean log-probability fell from -0.39 to -3.40, so NVFP4 servers are not used for rollout.

Because sessions survive updates, 52% of loss tokens in one audited run were generated on KV and SSM state built by an older version. The measured policy shift per version (about 2×10^{-4} nats/token) is far below the numerical floor (0.011), so the audit could not resolve the additional effect of stale cached state. Weight-version labels alone do not describe that state history.

11.4 Stability and audit

Several unstable runs developed repetitive output. An INT3 LoRA run with the BPO objective at temperature 1.0 and learning rate 5×10^{-5} improved for 24 weight versions, then fell into "the the the" loops from version 25; by version 30 half the sessions looped and the run was stopped. At 10^{-5} the same setup ran 40 versions stably. A projection-training experiment showed similar

behavior at lower learning rates: on a 16-task toy, 3×10^{-6}, the rate that v8 uses, raised reward from 0.45 to 0.77 without loops, while 10^{-5} looped from version 7 (85% of sessions at version 12). The train–rollout KL left its floor 2–3 steps before loops appeared, making it a possible early-warning signal in that run. The gain at 3×10^{-6} was efficiency: the share of episodes that submitted within budget rose from 0.15 to 0.65.

A read-only implementation audit identified three data and optimization issues. First, the rollout sandbox was not network-isolated at the time (isolation was added later, Section 9.6); a scan of 16,688 episodes found no use of the network. Second, the efficiency term in the audited runs' objective, which favors shorter episodes, counted proxy tokens, including 2.47M tokens that Codex discarded after truncation. Third, Adam's $\epsilon = 10^{-8}$ was comparable to the LoRA-A gradients, scaling their effective learning rate to 0.58–0.74×. The two selected recipes use $\epsilon = 10^{-12}$, and their rewards have no token-count term (Table 18).

11.5 ARC-AGI-2 experiments

Table 12 lists the ARC-AGI-2 RL runs in order, each with its setting and its result against the model it started from. The run behind the selected submission's checkpoint, r4, is marked †.

Table 12: ARC-AGI-2 RL runs in chronological order (2026). Scores are mean episode scores in points (0–100): partial credit up to the tool-use run, and strict (every test output correct) from r4 on unless stated. A difference is checkpoint minus starting model on the same tasks; ± is one standard error, brackets a 95% interval. @N is the Nth published weight version, which is one optimizer step except in the toy runs before October 29 (eight updates per version). "Held-out" means excluded from the named run. † marks the run whose checkpoint is in a selected submission.

Run	Setting	Result
Unpruned FP8 model; LoRA (rank 8); BPO objective; $T = 0.7$		
toy, rounds (Oct 7–8)	467 public training tasks that the model solves within 4k–16k output tokens, 64 of them held out; 8k output budget. Rounds of 512 episodes (64 tasks × 8) with 4 or 8 updates per round.	Held-out base 77.9. lr 2×10^{-5}, 5 rounds: -0.4 ± 2.2; the weights moved by 0.2%. lr 10^{-4}: +6.4 at @6 ($z = 3.6$).
toy, streaming (Oct 8)	Same tasks and budget without rounds: up to 64 groups of 8 in flight, one version per 272 episodes; lr 10^{-4}; 20 versions. Repeated with speculative decoding in the rollout.	Held-out base 77.1. Seven evaluated versions: +2.8 to +10.2, mean +5.9 (@18: +10.2, $z = 7.2$). With speculative decoding: mean +6.3. One version took 6.6 minutes; one round had taken 15.5.
hard269 (Oct 10)	936 public training tasks (64 held out), 80% of groups drawn from a 269-task subset; 32k budget; one update per 32 episodes. lr 10^{-4} to @103; a second arm at 2×10^{-5} to @57.	Held-out 89.5 → 87.9: −1.56 [−4.30, +1.17]. Trained subset 73.1 → 69.2. Second arm, 101 trained tasks: −3.5 [−6.6, −0.6].
eval120 (Oct 10)	The 120 public evaluation tasks as the training set; 32k budget; 128 episodes per step (16 groups × 8); lr 10^{-4}; 25 steps analysed.	Training reward 44.2 → 43.3. Same task, first against latest visit: −0.2 [−3.1, +2.7]. Budget exhaustion 63%→65%.
Undistilled INT3 package; LoRA (rank 8) with QAT; BPO objective		

Table 12, continued

Run	Setting	Result
INT3 toy (Oct 11)	The streaming toy on the INT3 package. Four runs: lr 10^{-4} and 5×10^{-5} at $T = 0.7$ (20 versions each); lr 5×10^{-5} and 10^{-5} at $T = 1.0$ (40 versions planned).	Held-out base 79.2–79.4. lr 10^{-4}: mean +3.1, unstable at @14–17. lr 5×10^{-5}, $T = 0.7$: mean +1.2. lr 5×10^{-5}, $T = 1.0$: repetition loops from @25, in 52% of sessions at @30. lr 10^{-5}, $T = 1.0$: +2.2 to +4.9 from @10 to @40.
eval28 (Oct 15)	28 public evaluation tasks that the base solves 20–80% of the time; 64k budget plus 8k wrap-up; 64 episodes per step; lr 2×10^{-5}; $T = 1.0$; 38 steps.	Training reward $+0.7 \pm 3.0$. The weights had moved by 0.04% at step 30.
overfit10 (Oct 15–16)	10 of those tasks; 256 episodes per step (16 groups $\times$ 16); lr 2×10^{-4}; advantage $R - \bar{R}_g$ without normalization, token mean within a group; $T = 1.0$; 60 steps. A second run at $T = 0.7$, 50 steps.	Trained tasks at $T = 1.0$, @34: 49.7 $\rightarrow$ 71.9. 90 unseen public evaluation tasks at $T = 0.7$: 78.2 $\rightarrow$ 68.0 at 64k (-10.2 [-15.7, -5.0]) and 82.9 $\rightarrow$ 76.9 at 128k (-6.0 [-11.1, -1.1]); mean output +20% and +32%. $T = 0.7$ run, trained tasks after 10–14 steps: +1.9 [-2.0, +5.9], against +9.8 at $T = 1.0$.
train60 (Oct 17–18)	60 public evaluation tasks; 384 episodes per step (24 $\times$ 16); lr 2×10^{-4}; an efficiency term of weight β that prefers the shorter of the all-correct episodes. Five runs of 18–38 steps: $\beta = 0.5$ and 2 at 64k and $T = 0.7$; $\beta = 3$ at 100k and $T = 0.8$, under two group schedules and at lr 10^{-5}.	Training reward flat at $\beta = 0.5$, at $\beta = 2$ and at lr 10^{-5}; $\beta = 3$: -5.0 [-8.1, -1.9] after 24–28 steps. All-correct episodes 6.9% shorter at $\beta = 2$ than at 0.5. $\beta = 0.5$, @38, on 51 held-out tasks: -7.2 [-11.6, -3.0].
Distilled INT3 package; LoRA (rank 8); MIS objective; $T = 1.0$, top-p 0.95, top-k 20		
clean15 (Oct 24–26)	15 hard public evaluation tasks, 16 episodes each per step; 150k output budget. lr 10^{-4} for 28 steps, then 5×10^{-4} for 11 more. Training score only.	lr 10^{-4}: $+0.4 \pm 2.0$ against the start. lr 5×10^{-4}: reward 49–54 for seven steps, then 29 at step 11; budget exhaustion from 20–32% to 58%; mean output from 96–106k to 125k tokens.
tool-use (Oct 28)	488 audited synthetic hard tasks (50 more held out). Reward: partial score, plus an efficiency term among all-correct episodes, plus a term for less hand computation and more toolkit calls. Runs 1–2: 44k budget, 32 episodes per step, 22 steps each. Run 3: 110k, 256 per step, lr 10^{-4}, 14 steps.	At 44k the base scores 38.5 with 54% budget exhaustion (72.5 at 110k); run 1: $+7.4 \pm 4.5$ at @10, then 20.6 at @20. Run 3, held-out base 72.8: -2.9 ± 2.5 at @5, -3.8 ± 3.2 at @10; hand computation -12% and -18%; output tokens and toolkit share unchanged.
r4 (Oct 29)[†]	90 tasks (60 public evaluation, 30 audited hard); reward 1 only if every test output is correct; 40k output and 64k context; 576 episodes per step (24 $\times$ 24); lr 10^{-4}; 36 steps (Table 18).	60 held-out tasks. Training conditions, base 56.5: @30 $+4.2 \pm 1.7$, budget exhaustion -10.1. At 200k, base 76.7: -0.8 ± 1.9 / -1.3 ± 2.1 / -2.5 ± 2.6 at @20/25/30 with 20–25% fewer output tokens; @35 -6.7 ± 3.0.
r4, deployed solver (Nov 1)	r4@20/25/30 in the single-solver episode of the submission ($T = 0.7$, 200k plus extension); 513 tasks outside the r4 pool, four runs each.	Base 56.4: -2.2 / -2.2 / -3.8 (± 1.1). Pass@2: $+0.1$ / $+0.4$ / -0.9 (± 1.3). Finished-but-wrong +7.8 to +11.1, budget exhaustion -5.7 to -7.6; output -9 to -13%.
Distilled INT3 package; direct updates to the 372 projections; all-correct reward; same sampling		

Table 12, continued

Run	Setting	Result
toy sweep (Oct 29)	16 public training tasks; 6k budget plus 2k wrap-up; 128 episodes per step; lr 10^{-5}, 3×10^{-6} and 10^{-6}; up to 40 steps.	Training reward from 45. lr 3×10^{-6}: 77 at steps 18–21, no loops. lr 10^{-5}: 73 at steps 3–4, then repetition loops (85% of episodes at step 12). lr 10^{-6}: 60 at steps 26–29.
hq741 (Oct 30)	428 hard tasks (a 741-task pool without its 240 easiest and 73 held-out tasks); 80k output, 120k context; 256 episodes per step (16 × 16); lr 5×10^{-6}; 19 steps.	@14 on the 120 public evaluation tasks at 200k: -13.3 ± 2.6 (checkpoint sampled at $T = 1.0$, base at 0.7); finished-but-wrong +15.1.
hq741b (Oct 30–31)	The same 428 tasks in the deployed episode: 200k output, compaction at 100k. lr 3×10^{-6}, raised to 10^{-5} at step 14; 16 steps.	Seven checkpoints on the 120 public evaluation tasks: −1.9 to +1.1 (se 1.5–1.9). In training, reward minus each task's base solve rate stayed within two standard errors at every step. One step changed 0.2% of the served FP8 values.
half-budget I (Oct 31)	Start: a variant of the student trained further on grid descriptions (80.8 on the 120 public evaluation tasks at 200k, partial credit; the distilled package scores 84.0). First 1,240 unscreened tasks at an 8k budget; then 994 tasks, each with half the output tokens a baseline run used on it (2k–16k), to @47; then 199 hard tasks at 100k from @47 for 18 steps. lr 3×10^{-6}; 256 episodes per step.	8k stage, held-out tasks at 8k: 16.3 → 24.0 ($+7.7 \pm 1.8$) at @29, flat after step 18. Half-budget stage @47 at 200k: 68.1 and 65.1 in two runs. 100k stage @15: $+6.2 \pm 2.9$ against @47 with 22% more output tokens.
half-budget II (Oct 31–Nov 1)	Start: a second such variant (79.4 on the same evaluation). The same 994 tasks and budgets; lr 3×10^{-6}; 50 steps. Then the 199 hard tasks at 100k from @20; lr 5×10^{-6}; 8 steps.	Training reward 49 → 65; held-out tasks at an 8k budget $+5.4 \pm 1.5$ at @29. At 200k: -0.8 ± 1.6, -2.1 ± 1.4, -3.5 ± 2.0, -9.1 ± 2.6 and -18.6 ± 3.0 at @10/20/30/40/50, with output per task falling from 68k to 37k. Continuation @5: 81.0, $+1.6 \pm 2.0$ against the start model.
r5half (Nov 1)	1,483 tasks from the distilled package, each with half the output tokens of a 200k baseline run (2k–112k) plus 4k wrap-up; 576 episodes per step; lr 3×10^{-6}; 37 steps.	60 held-out tasks at 200k, base 78.3: $+0.4 \pm 1.8$ at @10 with 15% fewer tokens; -5.0 ± 2.7 at @20; -12.1 ± 2.9 at @30. r4@30 under the same protocol: -5.8 ± 2.5.
schedule simulation (Nov 1)	Base, r4@30 and r5half@10 in a time-scaled simulation of the whole Kaggle schedule (240-task roster with solvers, refiners and voting).	94.7 and 93.3 (base); 95.0, 93.5 and 93.4 (r4@30); 94.3, 93.4 and 94.7 (r5half@10). r5half@10 against r4@30: −0.4 [−2.0, +1.4]. Episode length: r4@30 < r5half@10 < base.

The toy learned; the 32k runs did not. The first runs used the BPO objective [25] on the unpruned model. On the 8k toy, a learning rate of 2×10^{-5} changed nothing and 10^{-4} gained up to 10 points on held-out tasks. The gain was efficiency: the share of episodes that ran out of budget fell from 0.35 to 0.16. Replacing rounds with streaming groups cut the time per weight version from 15.5 to 6.6 minutes with a similar gain, and rollout with speculative decoding did not change it. At a 32k budget the same recipe improved nothing. On hard269 the trained tasks lost score at both learning rates. On the public evaluation tasks the training reward stayed flat for 25 steps while 63% of episodes exhausted the budget, and 23% of the episodes that scored had been cut at the budget and graded from the answer left in the workspace.

Learning the training tasks made unseen tasks worse. On the INT3 package the toy gain halved, and the runs exposed the learning-rate and temperature limits described under stability above. With 28 public evaluation tasks, 64 episodes per step and a learning rate of 2×10^{-5}, the weights barely moved. Ten tasks, 256 episodes per step and 2×10^{-4} did learn: the trained tasks rose from 49.7 to 71.9. On 90 unseen tasks at the deployment temperature the same checkpoint lost 10.2 points at 64k and 6.0 at 128k. Its outputs were 20–32% longer, and the loss sat in the 27 tasks whose budget-exhaustion rate had risen (-33 points; the other 63 tasks changed by -0.4). The next runs therefore added an efficiency term. It shortened all-correct episodes, but the training reward rose in none of the five train60 runs and 51 held-out tasks lost 7.2 points. A review of these runs put the missing rise down to dose: a task that could still be learned received 0.41 mixed-outcome groups per step, against 1.64 in overfit10, and 32 of the 60 tasks already passed at least 90% of the time.

Task quality, step size and reward terms. A run on synthetic hard tasks was stopped after 13 steps and discarded: for 60 of its 132 tasks from one synthetic source, several independent samples agreed on an answer that passed every training pair and differed from the reference. Later task pools were audited task by task. On 15 hard evaluation tasks, a learning rate of 10^{-4} moved the policy less than the numerical gap between trainer and server, and 5×10^{-4} made it think longer after seven steps until 58% of episodes exhausted the budget. The tool-use run showed two reward problems. Its 44k budget was too small to measure anything but the budget (the base scored 38.5 there and 72.5 at 110k). At 110k its added tool term was as large as the score term, so 14% of full-score episodes received a negative advantage; the policy reduced the hand computation that the term measured, and the score did not rise. r4 therefore dropped both added terms and rewarded only a fully correct answer.

r4 and the runs after it. Against its starting model, r4@30 gained 4.2 points on the 60 held-out tasks at its 40k training budget and changed the 200k score by -2.5 ± 2.6 points with a quarter fewer output tokens; at @35 the 200k score fell by 6.7 points, so checkpoints after @30 were not used. On the 513-task set in the deployed solver, the three tested checkpoints used 9–13% fewer output tokens and lost 2.2–3.8 points of single-attempt accuracy, with pass@2 unchanged: finished-but-wrong episodes rose by more than budget exhaustion fell. The later runs updated the projections directly. Trained at 80k, hq741 lost 13.3 points at 200k; its traces showed the model submitting the first rule that fit the training pairs. Trained on the deployed 200k episode, hq741b did not move in 16 steps, where one step changed 0.2% of the served FP8 values. In the half-budget runs, reward rose in training and on small budgets while the 200k score fell step by step as output per task fell from 68k to 37k tokens; r5half matched the base at step 10 with 15% fewer tokens and had lost 12.1 points by step 30. Five steps at a 100k budget brought a half-budget checkpoint back to the score of its starting model (81.0 against 79.4). Under the simulated Kaggle schedule the base, r4@30 and r5half@10 scored between 93.3 and 95.0. r4@30 produced the shortest episodes, and it was the only one of the three with which the late refinement stages had time to start; the submission kept r4@30.

11.6 ARC-AGI-3 experiments

Table 13 lists the ARC-AGI-3 runs. All of them train on public games, and the 25 public games are also the evaluation set; held-out seeds and games are named where a run had them. The run

behind the selected submission's checkpoint, v8, is marked †.

Table 13: ARC-AGI-3 RL runs in chronological order (2026). "Full game" is the mean RHAE over the 25 public games at $T = 0.7$ with 1M output tokens per game unless a budget is given; a difference is checkpoint minus starting model, ± is one standard error over games and brackets are 95% intervals. Clear rates are fractions of single-level episodes. @N is the checkpoint after N optimizer steps, a the number of actions and b the human baseline for the level. † marks the run whose checkpoint is in a selected submission. A new seed from a trained game tests transfer within that game; a game excluded from the run tests a different split.

Run	Setting	Result
Undistilled INT3 package; LoRA (rank 8); single-level episodes; full-game base 76.7, 78.6 and 79.4 in three runs		
overfit10 (Oct 18–19)	Level 1 of 10 games, each with its own output quota (16k–64k); reward 1 for clearing the level within the quota; 256 episodes per step (16 groups × 16); lr 2×10^{-4}; $T = 1.0$; 60 steps.	Trained levels: clear rate 0.56 → 0.85 by step 15. Full game, seven checkpoints: 58.9–68.2 against 79.4, that is −11 to −20 per game. Actions on the same levels ×1.25–1.78. Single-level evaluation: tokens per action from 1,542 to 612–747.
overfit10, $T = 0.7$ (Oct 19)	The same run at the deployment temperature; 18 steps.	Trained levels at steps 13–15: $+0.11 \pm 0.01$, against $+0.28 \pm 0.02$ at $T = 1.0$. Not evaluated further.
round 2 (Oct 20)	198 level tasks from 20 games, level 1 to the second-to-last level; 5 games held out. Each episode starts from a workspace replayed from a recorded full game. 100k output; reward clear × $(1 + wE)$ with E a within-group efficiency in log actions and log tokens; 24 steps.	Same task, first against latest visit: clear rate +0.047, tokens ×0.93, actions ×1.09. Full game @15: 72.4, −7.0 per game ($t = -1.5$).
v3 (Oct 20–22)	221 tasks (round 2 without its task filter); MIS; a smoothed efficiency term weighted toward tokens; 90k output. lr 2×10^{-4} twice, 5×10^{-5}, then 10^{-4} with a per-sequence mean for 44 steps.	lr 2×10^{-4}: after six steps, tokens per action 1.4k→ 9k and clear rate 0.2. Final run, same tasks on their third pass: clear rate $+0.147 \pm 0.012$, tokens ×0.82, actions ×1.06. Full game: 68.7 at @15; 75.3 at @27 with 160 levels cleared (base 155–158) at efficiency 0.90 (base 0.96–0.97).
v3.2 (Oct 22)	v3 with the action weight raised to 2.5 against 0.5 for tokens; $T = 0.8$; 100k output; 43 steps.	Same tasks: clear rate $+0.019 \pm 0.008$, actions ×1.00. Full game @40: 76.6 (154 levels, efficiency 0.96).
v3.3 (Oct 22–23)	Reward: clear × a function of the official level score. Episodes end at $1.2b$ actions; a task is retired once all 16 of its episodes score 1; $T = 1.0$; 60 steps.	Same tasks: clear rate $+10.6 \pm 1.3$ points by the fifth pass, then −3.9 [−7.0, −1.0]; tokens per action ×0.81. Full game @20: 77.4 (154 levels, efficiency 0.99).
v3.4 (Oct 23–24)	Continues v3.3 from step 30 at lr 2×10^{-5} with the token mean taken per group; 45 steps.	Trained tasks: $+5.0 \pm 1.2$ points over the start. Full game @37, two runs: 68.0 and 75.4; against the three base runs -6.5 ± 3.3.
Distilled INT3 package; LoRA (rank 8); MIS; $T = 1.0$, top-p 0.95, top-k 20		
v4b (Oct 24–26)	Whole games as episodes: 5 games, 800k tokens with compaction; reward = game score; 120 episodes per step (5 groups × 24); lr 10^{-4}; 16 steps.	Reward, late against early groups: $+0.001 \pm 0.008$. Policy entropy 0.49–0.50 throughout.

Table 13, continued

Run	Setting	Result
v5 (Oct 26)	Level seeds taken from recorded games: levels 2 onward of the same 5 games, 306 training and 44 held-out seeds. 48k output; reward $\mathbf{1}[\text{clear}] \cdot \min(1, (b/a)^2)$; 576 episodes per step (24 × 24); lr 2×10^{-4}; 22 steps.	Held-out seeds: clear rate 0.19, 0.39 and 0.61 at @0/10/20; tokens per action from 3,731 to 950. Full game, base 78.4: -2.3 ± 3.0 at @10, -22.3 ± 5.0 at @20 (efficiency 0.98 → 0.68).
v5k (Oct 26–27)	v5 from @20 with full action reward only below $0.7b$; 21 steps.	Full game: -14.1 ± 3.8 at @10, -27.8 ± 6.1 at @20. Held-out clear rate -7.4 ± 2.5 points against its start.
v6b (Oct 27–28)	80k output; hard levels restored (385 training, 61 held-out seeds). Reward $s\,(1 - \rho\,(0.3\,r_a + 0.2\,r_t))$: s is the action score, r_a and r_t are within-group ranks of actions and tokens among cleared episodes, ρ is the group's clear rate; ranks are off for the first 5 steps. 20 steps.	Held-out seeds @10: 0.451 → 0.558 ($+0.108 \pm 0.025$); hard-level seeds 0.045 → 0.101. Full game, base 76.7: $+3.2 \pm 3.6$ at @10, -2.3 ± 5.4 at @15, -25.6 ± 5.1 at @20.
v7 (Oct 28–29)	300k output with context-window changes as deployed; 34 levels of the same 5 games, half of the episodes on hard levels. A cost applies only above per-level token and action thresholds, which are frozen after 5 warm-up steps. 576 episodes per step; lr 2×10^{-4}; 11 steps.	Full game: $+7.0 \pm 3.0$ at @5; $+3.6 \pm 2.9$ at @10, which is -3.5 ± 1.8 against @5 with the same number of levels cleared (efficiency 0.94 against 0.98).
Distilled INT3 package; direct updates to the 372 projections; first context window only; same sampling		
v8 (Oct 30)†	235 seeds from 18 games (5 games held out). An episode ends at a clear or at the first window boundary; without a clear, an LLM judges the handoff notes (Table 18). 256 episodes per step (16 × 16); lr 3×10^{-6}; 20 steps.	Full game, base 75.0 at 1M and 81.9 at 1.5M: @20 $+1.5 \pm 2.7$ and -1.5 ± 2.5. Ten hard games: $+7.0 \pm 4.6$ at 1M, $+0.7 \pm 4.3$ at 1.5M. Efficiency 0.94 against 0.98.
v8b (Nov 1–2)	v8 with a different notes judge that also ranks the notes within a group. lr 5×10^{-6} for 9 steps; restarted at 3×10^{-6} for 21 steps.	lr 5×10^{-6}: repetition loops in 35% of episodes at @9. lr 3×10^{-6}, full game at 1.5M: -1.1 ± 1.8, $+2.4 \pm 2.1$, -0.2 ± 2.5 and -4.4 ± 1.7 at @5/10/15/20; ten hard games at @20 -11.4 ± 2.9.
v38 (Oct 30)	Two complete configurations on the G4 development machine, nine hours each: the first ARC-AGI-3 final, and the same harness with the toolkit and the v7@5 weights.	20 games v7 did not train on: +1.2 [−1.3, +3.8]. Its 5 training games: +11.4 [+2.3, +19.0].

Single levels taught the agent to think less and act more. The first run rewarded clearing one level within a token quota. The clear rate on its ten levels rose from 0.56 to 0.85, and full-game RHAE fell by 11 to 20 points per game. The agent now spent less than half as many tokens on each action and took 1.25 to 1.78 times as many actions on the same levels, and RHAE falls with the square of the action count. The next four runs changed the reward to stop this. An efficiency term weighted toward tokens (v3) bought tokens with actions. Weighting actions more (v3.2) held actions level and left the clear rate where it started. The official level score with a tight action cap (v3.3) raised the clear rate by 10.6 points and then lost 3.9 as failures moved from exhausted budgets to the action cap. In full games these checkpoints scored 68.7 to 77.4 against 76.7 to 79.4 for the base, and v3.4 was 6.5 points below it.

The score is decided on hard levels. The base is 20–23 points short of a full score, and 18–20 of those points are levels it does not clear. In full games it spends 200k–500k tokens on such a level, while these runs allowed 100k per episode. In v3.4, 15% of the groups failed all 16 episodes; they consumed 21% of the rollout tokens and produced no gradient. Training on whole games removed the budget limit and with it the signal: v4b assigned one reward to about 790k tokens, and neither reward nor entropy moved in 16 steps. v5 therefore returned to single levels, started from positions recorded inside real games.

The same drift at every budget. At 48k tokens v5 learned quickly: the clear rate on held-out seeds went from 0.19 to 0.61. Full-game RHAE fell by 22 points. Median tokens per action fell from 3,731 to 950, and on the 20 games outside training the agent restarted a level 658 times against 27 for the base. Training was limited by tokens and deployment by actions. Tightening the action term (v5k) recovered part of the loss by step 10 and lost it again by step 20. v6b raised the budget to 80k, restored the hard levels that the 48k seed selection had filtered out, and scaled its efficiency ranks by the group's clear rate. Its step-10 checkpoint gained 3.2 ± 3.6 points in full games at 1.08 times the base's actions on the same levels. From step 11 the drift returned: @15 newly cleared 11 hard levels, nine of them in the trained games, and lost 11.7 ± 2.9 points against @10 on the other 20 games; @20 lost 25.6. v7 gave episodes 300k tokens with the deployed window changes. While its cost term was off, @5 gained 7.0 ± 3.0 points with unchanged action counts; with the term on, @10 took 1.15 times the base's actions and fell 3.5 points behind @5. Three attempts ended before any evaluation: v3.1 during its second step, v6 at step 4 because it still used the 48k seed selection, and v7b, a 20-game follow-up to v7, after one step.

First-window training. One reward for an episode of several windows gave each window little signal, so v8 trains only the first window and scores the handoff notes when the level is not yet cleared. v8@20 cleared hard levels sooner without raising the ceiling: on the ten hard games it was 7.0 points ahead at 1M tokens and 0.7 ahead at 1.5M. Over all 25 games the extra levels were offset by lower action efficiency (0.94 against 0.98). v8b changed the judge and drifted the other way: tokens per level grew by factors of 1.09, 1.30 and 1.64 at steps 10, 15 and 20, and the step-20 checkpoint lost 4.4 points. The notes judges of v8 and v8b were evaluated on 60 test groups; their within-group outcome-ranking AUCs were 0.641 and 0.675.

Which games a gain comes from. The v7@5 checkpoint trained on five games; in the G4 nine-hour comparison, it gained 11.4 points on those games and 1.2 points on the other twenty (95% interval $[-1.3, +3.8]$). The selected v8@20 checkpoint used the different 18/5/2 split described in Section 11.1. Its five held-out games changed by $+1.3 \pm 1.2$ points at 1M tokens and $+0.3 \pm 0.3$ at 1.5M. Some 1.5M-budget runs were stopped before exhausting that budget, although all had exceeded 1M.

Snapshot quality. Level seeds require a separate audit from token fidelity. Trace replay reproduced the recorded ledger rows in two checks (109/109 and 206/206 responses). A collection of 198 replayed games yielded 1,109 seeds, and a later live collection produced 6,338 snapshots. However, snapshots taken at the first action of a level could already contain work on that level: 64% in one audit did

so, and 8 of 36 last-level seeds that always passed carried a precomputed answer. The collection procedure was changed to capture the transition after the previous level's notes were complete and before the next level became visible.

Deploying the checkpoints. Uploading only the 516 changed tensors (3.35 GB for the ARC-AGI-3 patches, versus a 21 GB overlay) fit the private model quota; the INT3 runtime swaps them in by name with per-tensor hashes. On Kaggle the in-run swap took 24 s of preload and verification and paused games for 0.33 s, with decode speed and DFlash acceptance unchanged.

12 Lessons, Negative Results, and Silent Failures

This section records what did not work, grouped by what failed: a proposed change (Table 14), an interpretation of a measurement (Table 15), or an implementation check (Section 12.3). Each entry gives what we tried or believed and what we observed; the component sections hold the detail. The uncertainty of each comparison depends on its evaluation protocol (Section 3). Most of these records came out of the development loop described in Section 6.

12.1 Negative results

Table 14 lists tested changes that did not show a reliable improvement in the reported settings. Some prompt and toolkit changes reduced token use without a resolved score difference. Several output-format changes introduced copying or formatting errors. Lower-precision representations sometimes saved memory but increased generated tokens or kernel cost. Two of these changes still shipped, the reviewer and the later toolkit (Table 8), and the re-refiner result became a skip rule. These observations guided which changes we retained; they do not rule out benefits under other budgets or models.

12.2 Retracted conclusions

Table 15 lists conclusions we reported internally and later withdrew. In every case the first number was real and its attribution was wrong. Four were instrument faults, one a confounded flag, and two measured the wrong quantity. The common fix was a control that should have shown nothing: an identity rewrite, a run with the suspect flag alone changed, or a microbenchmark at the production shape.

12.3 Silent failures

The following faults produced plausible output without raising an error. Reference comparisons, positive controls and trace inspection exposed failures that basic execution checks had missed.

- **NVFP4 KV scale layout** on sm_100: a wrong layout gives finite results correlated with the right ones, wrong in proportion to the spread of block scales (Section 8). A CPU test now pins the layout formula.

- **Missing MTP dispatch entry**: the embedded drafter ran unquantized. Adding the entry raised acceptance from 1.17 to 2.17 and single-stream decode from 64.3 to 117.8 tok/s.
- **Draft probabilities overwritten** by the target verify replay, which wrote into a shared tensor view: 8 of 32 outputs garbled; a clone gave 0 of 32.
- **Sparse-attention pending ring**: the stock ring used the wrong compressed key for 84% of generated tokens.
- **Liger MoE input gradient** wrong by 55–65% without our patch, in the fused MoE kernel of the Liger-Kernel library; the trainer ran normally. Root cause: two chained `tl.dot` into one accumulator on sm_100 (relative error 0.642). The trainer now asserts this gradient at start-up.
- **FP8 PLE lookup with int32 addresses** wrote 12,516,352 elements out of bounds on large batches; it surfaced only as routing mismatches on recomputation.
- **DFlash trainer gradients** 52–74% too small in the backbone from a cut graph; a single cosine-similarity gate had passed it. Correcting it raised held-out acceptance from 3.578 to 3.776, with a learning-rate and optimizer reset confounded (Appendix E.10).
- **Taint scanner** printed `tainted=0 CLEAN` on streams it could not parse, byte-identical to a clean result. It now raises on unknown stream formats and is verified with positive controls (Appendix C).
- **Short-id scorer**: a roster of short game ids against a ledger keyed by full ids returned 0 for every game.
- **Kaggle `machine_shape`**: three misspellings silently ran on no GPU or a P100. We now read the stored value back after each push.

Table 14: Negative results. Single comparisons unless stated; ARC-AGI-2 numbers are public-evaluation points unless the row names another task set. TIR is the tool-integrated reasoning protocol of Section 3. Rows end with the status in the selected submissions (Appendix A) where one applies.

Change tried	Observed
Read-only reviewer (ARC-AGI-3)	LLM readers classified 26% of verifiable findings as wrong, 16 levels as helped and 11 as misled. RHAE 81.23 vs. 82.85 without it; SciPy also changed. Kept in both finals (Section 5).
Workspace toolkit and tool cards (ARC-AGI-2)	Tokens −5.1 to −5.2%; score −0.04 [−2.19, +2.16] over 8 rounds. Toolkit kept in 57848658 (Section 4).
Prompt variants, 11 arms (TIR)	No arm above the replicate range on pass@2. Structured RULE/CODE output: −4.7 to −6.2 pass@1. Space-separated grid: −9 pass@1 (14% malformed output).
Task augmentation (rotation, flip, recoloring; answer mapped back)	One session: 82.64 vs. plain 83.98±1.91 over eight samples. With the refiner: 93.75 vs. plain 94.03/92.78. Not used (Section 4).
Chunked grid rendering	Without tools, on real training tasks with output side 11–20: −10.7 [−16.5, −4.9], tokens +45%. Not used (Section 4).
ThinkingCap, a third-party fine-tune for shorter reasoning (NVFP4)	TIR: 69.3 vs. 89.6 (submits early). Codex: 79.2 vs. 80.3, output −14%. Not used.
Reasoning in Chinese (all-Chinese prompt, TIR)	Paired tasks: 25 better, 66 worse, $p = 2.0 \times 10^{-5}$. Not used.
4-bit KV with TurboQuant (RTX 4090)	KV bytes −48.4%, but 17–22% more generated tokens and batches 18–22% slower (six tasks). FP8 KV kept there; the ARC-AGI-3 finals use NVFP4 KV (Section 8).
NVFP4 weights × FP8 activations	Real MoE GEMM 4–25% slower than W4A16. Dropped (Section 7).
Second-generation ARC-AGI-3 harness	Same levels per token as the first generation; lost on re-prefill and decode speed (Table 4). Not used (Section 5).
Grid-perception fine-tuning	Held-out perception 0.43→0.83; ARC-AGI-2 −3.2 / −4.6 (two versions). Not used.
Re-refining tasks all solvers agree on	52–81% of re-refiner tokens for +0 to +0.25 tasks. Skipped in 57848658.
RL on real tasks	Reduced token use in some settings; no consistent held-out gains at deployment budgets. RL weights still ship in one final per track (Section 11).

Table 15: Retracted conclusions and what they actually measured. sm_100 is the GPU architecture of the B200.

Original claim	Actual cause
4-bit weights cost 10–15 points (RTX 4090)	A speculative-decoding state flag (ReplaySSM) stopped long chains from finishing: finish rate 12.6%→41.9% without it ($p = 3.3 \times 10^{-21}$), same FP8 weights. Not reproduced on B200; mechanism unresolved.
PLE row pruning costs 12 points	The null control, an identity rewrite, cut MTP acceptance from 2.71 to 1.10; the patch had unknown side effects, and all its scores were withdrawn.
Replacing sparse attention with dense attention costs 27 points (75.0→47.9)	History filter used `len(BatchEncoding)`, which is 2, as the token count, so sparse runs with a 262k budget were compared with 64k dense runs. At equal 64k context: 55.9 vs. 52.0, within noise (Section 8).
MoE is 4% of decode time on sm_100	Trace analyzer bug: it filed the grouped-expert kernels under "other". The true share is about 50%.
Indexer patches give 1.54–1.61× and 1.20×	Microbenchmark shrank the page table to fit the batch; at production width 1.25–1.57×, and the second patch was a regression (97.2→120.0 μs) and was reverted.
Checkpoint prefetch removes Kaggle cold-start load time	Only the prefetch was timed; model load still took 985 s.
Re-tokenization divergences grow with RL training (Section 10)	The counter was cumulative per server instance.

13 Prior and Related Work

This section places each component of the system next to its closest prior work: first what exists, then how our system differs. Section 1 introduces the benchmarks. The ARC Prize technical reports summarize the 2024 and 2025 competitions [4, 30], and a living survey compares more than eighty approaches across the three benchmark versions [31].

Solving static ARC tasks. Approaches to ARC-AGI-1 and ARC-AGI-2 either induce an explicit program or transduce the output grid directly [32]. Transductive systems fine-tune language models on augmented task data and adapt them to each task with test-time training [33, 34]; small recursive networks trained from scratch [35, 36] and vision models over rendered grids [37] are further weight-space alternatives. Inductive systems sample candidate programs and filter them against the demonstrations, in the spirit of AlphaCode [38]. The ARC Prize 2025 report identifies the per-task refinement loop as that year's defining theme [4]. Examples include evolutionary program synthesis with a self-improving model [39], refinement by algorithmic debugging [40], multi-agent synthesis with reflective feedback [41], harnesses that separate pattern discovery from program synthesis [42], and candidates generated across text, image, and code and selected by a holistic judge [43]. Closest to our operating setting, a recent study compares training and calibration data for open models that solve ARC-AGI-2 without internet access on four L4 GPUs within 12 hours [44]. Our system belongs to the inductive refinement-loop family, but its search operator is a general-purpose coding agent working in a persistent workspace, and its refiners inspect competing programs, check results, and workspace artifacts rather than only final grids. Our emphasis on the serving and scheduling system that lets such agents run within the competition hardware budget is complementary to work on training data (Sections 4 and 8).

Agents for ARC-AGI-3. Interactive agents build on designs that interleave reasoning with actions [45], accumulate executable skills [46], and manage memory beyond a single context window [47]. Early ARC-AGI-3 entries used training-free exploration over a graph of observed states [48], and an analysis of the public games argues that they cannot separate intelligent exploration from simple heuristics [49]. Several recent systems use coding agents that write and check executable world models: Codex-based agents with world-model, simplification, and replay-verification variants [50, 51]; Tycho [5] and Kepler [6], which validate executable hypotheses against recorded transitions and report high RHAE on the 25 public games with frontier API models; and a study tracing how a coding agent's file-based beliefs form and are corrected across levels [52]. Our ARC-AGI-3 harness shares the coding-agent and workspace design, but runs an open-weight model without internet access on one competition GPU, which shifts the problem toward concurrency, context management, and sharing one serving budget across many games (Section 5).

Coding agents and test-time compute. Program-aided reasoning delegates computation to an interpreter [53], and agent–computer interfaces let models edit files, run commands, and test code over many turns [24, 54, 55]. Self-feedback and verbal reflection improve outputs without weight updates [56, 57], while repeated sampling and compute-optimal allocation show how test-time compute can substitute for model scale [58, 59]. We run one such agent, Codex, in both tracks

rather than a task-specific loop (Section 9). Under a fixed accelerator budget these ideas are coupled to inference efficiency: the number and length of agent attempts per task are bounded by serving throughput (Section 3).

Expert pruning, quantization, and distillation. The backbone, Qwen3.8-Flash-Next, combines a sparse mixture of experts with Gated DeltaNet layers [11], Qwen Sparse Attention, and an n-gram embedding table held off the accelerator [7]. Expert pruning removes experts selected from router and activation statistics [12, 60]; post-training quantization [10] and block-scaled formats including NVFP4 [13, 61] reduce weight and cache storage; and distillation [14], quantization-aware training [15], and on-policy distillation on student-generated sequences [16, 17] recover quality after compression. We apply all three to this backbone (Section 7): our pruning statistic, gEAN, keeps the call-frequency contribution that REAP divides out, and our distillation leaves the three-bit GPTQ expert codes fixed and trains only continuous parameters against the unpruned FP8 teacher, with a second round on the student's own agent trajectories.

Speculative decoding and block drafters. Speculative decoding verifies drafted tokens with the target model [22, 23]; drafters range from extra decoding heads and multi-token prediction [62–64] to feature-conditioned autoregressive drafters [65, 66] and block-diffusion drafters [9], with open training frameworks such as SpecForge [67]. Our drafter follows the DFlash block design but is initialized from the target's native multi-token-prediction layer, replaces that layer's mixture-of-experts feed-forward network with a dense one, and is trained on agent trajectories with an objective that optimizes the acceptance probability at the serving temperature (Appendix E). We evaluate it by serving throughput, because accepted length alone did not determine throughput (Section 8).

Serving long agent sessions. Our serving stack extends SGLang [18], whose prefix-sharing runtime builds on paged KV-cache management [68]. Sparse-attention methods read a small, query-dependent subset of the KV cache at each step [20, 21], and hierarchical caches keep the full history in host memory with a bounded GPU working set [8, 69]. We build on HiSparse, adding a session-aware prefix cache, for up to 128 agent sessions that alternate reasoning, tool calls and resumption over 50–100k-token histories (Section 8). Because a cached prefix of this hybrid backbone is reusable only if the recurrent Gated DeltaNet state at that point is also kept, our fork parks the recurrent and KV state at the end of a turn, and a server-side token ledger replays the model's own turns from their sampled token ids, so that the client's re-rendering of a turn does not force recomputation (Section 10).

Reinforcement-learning infrastructure. Policy-gradient fine-tuning of language models commonly derives from PPO [70], with critic-free group-relative variants [71, 72] and refinements of clipping and sequence-level ratios [73, 74]; our learner supports Bellman Policy Optimization [25] and a masked token-level importance-sampling objective. The selected checkpoints use the latter, defined in Section 11. Decoupling rollout generation from learning follows asynchronous actor–learner designs with importance-weight corrections [26] and their language-model counterparts [27, 28, 75], including multi-turn agent training frameworks [76]. Numerical differences between training and inference are a known source of instability; remedies include higher-precision rollouts [77] and

replaying inference-time MoE routing during training [29]. FlashNext-RL optimizes long-horizon agentic RL for the same backbone on software-engineering rollouts [78]. Our system collects rollouts through the deployed coding-agent harness itself and records tokens, behavior probabilities, and routing decisions at the serving boundary rather than reconstructing them from client transcripts.

14 Discussion and Limitations

14.1 Transfer across tasks, models, and hardware

Both tracks run the same compressed base model, drafter and inference-engine source tree under the Codex runtime (Tables 16 and 17), on a static program-induction task and an interactive exploration task. The harness and the deployment settings (GPU count, concurrency, served context and thinking cap) are specific to each track (Sections 4 and 5).

The ARC-AGI-3 harness also ran with a second model. With the same Codex arm and a proxy that converts video observations to image frames, DeepSeek-V4.1-Flash reached RHAE 85.77 at high and 89.90 at max reasoning effort (one run each), against 82.20±3.75 for five rounds of FP8 Qwen3.8-Flash-Next (paired difference for max +7.7 [+2.0, +14.8]). This comparison uses an API model on development hardware and the public tasks.

One fork of the inference engine runs on three GPU generations (sm_89 L4, sm_100 B200, sm_120 RTX PRO 6000). The four-L4 deployment required its own interconnect implementation (Section 8).

Two measured gains did not carry over to deployment conditions. The ARC-AGI-2 RL checkpoint used in the second final (r4@30) gained 4.2 ± 1.7 points of strict accuracy on 60 held-out tasks at a 40k output budget and changed it by -2.5 ± 2.6 points at 200k; an earlier ARC-AGI-3 RL run (v5) raised the held-out-seed pass rate from 0.19 to 0.61 while full-game RHAE fell by 22 points (Section 11). The eight-layer drafter's +2.3% gain in offline expected accepted drafts became +0.5–1% in serving throughput (Appendix E.11). The negative results in Table 14 do not rule out benefits under other budgets or models. We have not evaluated the stack on ARC-AGI-1 or on non-ARC tasks.

14.2 Limitations

Scores and comparisons. The two scores in the abstract, 87.22 on ARC-AGI-2 and 75.56 on ARC-AGI-3, are single runs on the public leaderboard, which covers part of the hidden tasks; private results were not available at the time of writing. The 87.22 run is the highest of five runs of the selected ARC-AGI-2 base notebook (script version 362595216; 84.31–87.22, mean 85.64), and the 75.56 run is one of three (mean 74.55). The means of the two finals of a track differ by less than one point; on ARC-AGI-2 that difference lies inside the range of either final, so the repeated runs do not rank one above the other (Section 2 and Table 2). A separate, unselected scheduler variant, `x36-base` (script version 364209164), scored 83.06, 86.81, 69.58 and 87.22; the cause of the 69.58 run remains unresolved, and that run is not part of the five-run mean.

The two finals of a track differ in several components at once (Tables 16 and 17): on ARC-AGI-2, the RL weight patch, a later harness, a thinking cap and a longer served context; on ARC-AGI-3,

the toolkit, game cards, a final-call notice and RL weights swapped in at six hours. No leaderboard score therefore isolates the RL weights or any other single component.

Data provenance. Public ARC-AGI-2 tasks may appear in the backbone's pretraining data. In the student-trajectory distillation collection, 20.6% of 7.23×10^8 supervised training tokens came from four task pools constructed from evaluation-task parents or families. This is a pool-provenance statistic, not a measured near-duplicate rate for individual tasks. A game or snapshot withheld from a particular ARC-AGI-3 RL run was not necessarily withheld from all development (Appendix C details both). Hidden Kaggle tasks provide a separate evaluation, without establishing that the pretrained model has never encountered related material.

Evaluation integrity. Two early local ARC-AGI-3 runs were rejected as evaluations and do not enter the selected-submission summary. In the first, agents in 3 of 25 games read the game source on disk. The second, the first jailed run, was incompletely isolated and still exposed upstream replay files. Subsequent runs restricted the mounted interface and added positive-control checks for the detector (Section 9.6 and Appendix C).

A command-pattern audit of 16,688 episodes from two early ARC-AGI-2 RL runs (full commands and digests for one run, digests for the other) found no evidence of network answer retrieval (Section 11). This audit does not establish the provenance of every later checkpoint; tracing corrected snapshot collection and exclusions into each final training manifest remains a separate verification step.

Simulation. Many ARC-AGI-2 scheduling decisions used a time-scaled simulation on an RTX PRO 6000. For one configuration run both ways, the final scores agree to within 0.7 points (88.12 in simulation, 88.82 in a public-set rehearsal on Kaggle hardware, both on the same duplicated public-task roster), but the extra solvers and refiners raised the score in simulation and lowered it on Kaggle (Table 6). Each score comes from a single run.

15 Conclusion

The final system runs the Codex coding agent on an open reasoning model, Qwen3.8-Flash-Next, served locally by our fork of SGLang. On ARC-AGI-2, independent solvers write and test transformation programs in separate workspaces, and a refiner inspects their solutions and revises the candidate outputs. On ARC-AGI-3, each agent infers the game rules through interaction and keeps scripts, observations and hypotheses across context windows. Compression (expert pruning, three-bit expert quantization and quantization-aware distillation) and serving changes (sparse-attention history in host memory, session state preserved across tool calls and a speculative drafter) make these workloads feasible without internet access and within the competition's hardware and time limits: 128 concurrent sessions on four L4 GPUs within twelve hours, and 110 concurrent games on one RTX PRO 6000 within nine hours.

The selected submissions scored 87.22 and 85.97 on the ARC-AGI-2 public leaderboard and 75.56 and 75.44 on the ARC-AGI-3 public leaderboard. Their means over repeated runs are 85.64, 85.03,

74.55 and 75.32 (Table 2).

Four lessons follow from the measurements. Executing the model's own program was more accurate than its reported answer, and keeping task state in files rather than in a written summary raised accuracy at the same generated-token budget (Section 3). Serving throughput limited how much planned work an early four-L4 build finished, while a later +6.5% in decode throughput moved the public score by +0.17 (Table 4). RL on trajectories recorded through the deployed runtime improved results under training conditions but gave no consistent held-out gain at deployment budgets (Tables 12 and 13). For the conclusions we later withdrew, the common fix was a control that should have shown nothing (Section 12). Section 14 states what these scores and comparisons do not establish.

References

[1] François Chollet. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

[2] Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. *arXiv preprint arXiv:2505.11831*, 2025. URL https://arxiv.org/abs/2505.11831.

[3] ARC Prize Foundation. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. *arXiv preprint arXiv:2603.24621*, 2026. URL https://arxiv.org/abs/2603.24621.

[4] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical Report. *arXiv preprint arXiv:2601.10904*, 2026. URL https://arxiv.org/abs/2601.10904.

[5] Jens Lehmann, Andrei Aioanei, and Sahar Vahdati. Tycho: Active Abstraction with Programmatic World Models for ARC-AGI-3. *arXiv preprint arXiv:2607.28287*, 2026. URL https://arxiv.org/abs/2607.28287.

[6] Wensen Wu. Kepler: Auditable World Models for ARC-AGI-3. *arXiv preprint arXiv:2610.00834*, 2026. URL https://arxiv.org/abs/2610.00834.

[7] Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bochao Mao, Chengruidong Zhang, Fan Zhou, Hao Luo, et al. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability. *arXiv preprint arXiv:2608.30320*, 2026. URL https://arxiv.org/abs/2608.30320.

[8] Zhiqiang Xie, Zhangheng Huang, Tingwei Huang, Ziyi Xu, Ruiyang Ma, and Christos Kozyrakis. HiSparse: Scaling Sparse-Attention Decoding with Hierarchical KV Cache Management. *arXiv preprint arXiv:2608.07009*, 2026. URL https://arxiv.org/abs/2608.07009.

[9] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block Diffusion for Flash Speculative Decoding. *arXiv preprint arXiv:2602.06036*, 2026. URL https://arxiv.org/abs/2602.06036.

[10] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL https://arxiv.org/abs/2210.17323.

[11] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*, 2024. URL https://arxiv.org/abs/2412.06464.

[12] Mike Lasby, Ivan Lazarevich, Nish Sinnadurai, Sean Lie, Yani Ioannou, and Vithursan Thangarasa. REAP the Experts: Why Pruning Prevails for One-Shot MoE compression. *arXiv preprint arXiv:2510.13999*, 2025. URL https://arxiv.org/abs/2510.13999.

[13] NVIDIA. Pretraining Large Language Models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025. URL https://arxiv.org/abs/2509.25149.

[14] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*, 2015. URL https://arxiv.org/abs/1503.02531.

[15] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. *arXiv preprint arXiv:2305.17888*, 2023. URL https://arxiv.org/abs/2305.17888.

[16] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. *arXiv preprint arXiv:2306.13649*, 2023. URL https://arxiv.org/abs/2306.13649.

[17] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-Policy Distillation of Large Language Models. *arXiv preprint arXiv:2306.08543*, 2023. URL https://arxiv.org/abs/2306.08543.

[18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*, 2023. URL https://arxiv.org/abs/2312.07104.

[19] SGLang Team. Sglang. https://github.com/sgl-project/sglang, 2026. Software repository, consulted November 2026; deployment revisions are tracked separately.

[20] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *arXiv preprint arXiv:2406.10774*, 2024. URL https://arxiv.org/abs/2406.10774.

[21] DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv preprint arXiv:2512.02556*, 2025. URL https://arxiv.org/abs/2512.02556.

[22] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *arXiv preprint arXiv:2211.17192*, 2022. URL https://arxiv.org/abs/2211.17192.

[23] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL https://arxiv.org/abs/2302.01318.

[24] OpenAI. Codex: Open-source coding agent. https://github.com/openai/codex, 2026. Software repository, consulted November 2026; deployment revisions are tracked separately.

[25] Zhuoqing Song, Haotian Xu, Xikun Zhang, and Lidong Bing. Bellman Policy Optimization. *arXiv preprint arXiv:2609.15987*, 2026. URL https://arxiv.org/abs/2609.15987.

[26] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymir Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures. *arXiv preprint arXiv:1802.01561*, 2018. URL https://arxiv.org/abs/1802.01561.

[27] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. *arXiv preprint arXiv:2410.18252*, 2024. URL https://arxiv.org/abs/2410.18252.

[28] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, et al. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *arXiv preprint arXiv:2505.24298*, 2025. URL https://arxiv.org/abs/2505.24298.

[29] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers. *arXiv preprint arXiv:2510.11370*, 2025. URL https://arxiv.org/abs/2510.11370.

[30] Francois Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical Report. *arXiv preprint arXiv:2412.04604*, 2024. URL https://arxiv.org/abs/2412.04604.

[31] Sahar Vahdati, Andrei Aioanei, Haridhra Suresh, and Jens Lehmann. The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning. *arXiv preprint arXiv:2603.13372*, 2026. URL https://arxiv.org/abs/2603.13372.

[32] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, Wei-Long Zheng, Zenna Tavares, et al. Combining Induction and Transduction for Abstract Reasoning. *arXiv preprint arXiv:2411.02272*, 2024. URL https://arxiv.org/abs/2411.02272.

[33] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. *arXiv preprint arXiv:2411.07279*, 2024. URL https://arxiv.org/abs/2411.07279.

[34] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective. *arXiv preprint arXiv:2505.07859*, 2025. URL https://arxiv.org/abs/2505.07859.

[35] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical Reasoning Model. *arXiv preprint arXiv:2506.21734*, 2025. URL https://arxiv.org/abs/2506.21734.

[36] Alexia Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv preprint arXiv:2510.04871*, 2025. URL https://arxiv.org/abs/2510.04871.

[37] Keya Hu, Ali Cy, Linlu Qiu, Xiaoman Delores Ding, Runqian Wang, Yeyin Eva Zhu, Jacob Andreas, and Kaiming He. ARC Is a Vision Problem! *arXiv preprint arXiv:2511.14761*, 2025. URL https://arxiv.org/abs/2511.14761.

[38] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, et al. Competition-Level Code Generation with AlphaCode. *arXiv preprint arXiv:2203.07814*, 2022. URL https://arxiv.org/abs/2203.07814.

[39] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. *arXiv preprint arXiv:2507.14172*, 2025. URL https://arxiv.org/abs/2507.14172.

[40] Yu-Ning Qiu, Lin-Feng Zou, Jiong-Da Wang, Xue-Rong Yuan, and Wang-Zhou Dai. Procedural Refinement by LLM-driven Algorithmic Debugging for ARC-AGI-2. *arXiv preprint arXiv:2603.20334*, 2026. URL https://arxiv.org/abs/2603.20334.

[41] Kunbo Zhang, Lei Fu, Zeyu Wang, Zijing Liu, and Kejian Tong. ARCANA: A Reflective Multi-Agent Program Synthesis Framework for ARC-AGI-2 Reasoning. *arXiv preprint arXiv:2607.09059*, 2026. URL https://arxiv.org/abs/2607.09059.

[42] Kabir Moghe and Peter Chin. Cost-Effective Agent Harnesses for Abstract Reasoning and Generalization on ARC-AGI-1. *arXiv preprint arXiv:2607.06764*, 2026. URL https://arxiv.org/abs/2607.06764.

[43] Johan Land. Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2. *arXiv preprint arXiv:2606.31543*, 2026. URL https://arxiv.org/abs/2606.31543.

[44] Nguyen Minh Thang and Trinh Thi Phuong. Rules, Traces and Routing Statistics: Training and Calibration Data for Open Models on ARC-AGI-2. *arXiv preprint arXiv:2611.06827*, 2026. URL https://arxiv.org/abs/2611.06827.

[45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022. URL https://arxiv.org/abs/2210.03629.

[46] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

[47] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[48] Evgenii Rudakov, Jonathan Shock, and Benjamin Ultan Cowley. Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks. *arXiv preprint arXiv:2512.24156*, 2025. URL https://arxiv.org/abs/2512.24156.

[49] Liew Keong Han. Explore Before You Solve: The Speed–Depth Trade-off in Epistemic Agents for ARC-AGI-3. *arXiv preprint arXiv:2605.25931*, 2026. URL https://arxiv.org/abs/2605.25931.

[50] Sergey Rodionov. Executable World Models for ARC-AGI-3 in the Era of Coding Agents. *arXiv preprint arXiv:2605.05138*, 2026. URL https://arxiv.org/abs/2605.05138.

[51] Sergey Rodionov. Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026. URL https://arxiv.org/abs/2607.15439.

[52] Chen Wu, Josh Passenger, and Yin Song. Tracing the Thoughts of a Coding Agent Playing ARC-AGI-3: Lessons for Continual Learning. *arXiv preprint arXiv:2610.11450*, 2026. URL https://arxiv.org/abs/2610.11450.

[53] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. *arXiv preprint arXiv:2211.10435*, 2022. URL https://arxiv.org/abs/2211.10435.

[54] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

[55] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741*, 2024. URL https://arxiv.org/abs/2407.16741.

[56] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, et al. Self-Refine: Iterative Refinement with Self-Feedback. *arXiv preprint arXiv:2303.17651*, 2023. URL https://arxiv.org/abs/2303.17651.

[57] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

[58] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. *arXiv preprint arXiv:2407.21787*, 2024. URL https://arxiv.org/abs/2407.21787.

[59] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL https://arxiv.org/abs/2408.03314.

[60] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models. *arXiv preprint arXiv:2402.14800*, 2024. URL https://arxiv.org/abs/2402.14800.

[61] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, et al. Microscaling Data Formats for Deep Learning. *arXiv preprint arXiv:2310.10537*, 2023. URL https://arxiv.org/abs/2310.10537.

[62] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint arXiv:2401.10774*, 2024. URL https://arxiv.org/abs/2401.10774.

[63] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & Faster Large Language Models via Multi-token Prediction. *arXiv preprint arXiv:2404.19737*, 2024. URL https://arxiv.org/abs/2404.19737.

[64] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

[65] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. URL https://arxiv.org/abs/2401.15077.

[66] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. *arXiv preprint arXiv:2503.01840*, 2025. URL https://arxiv.org/abs/2503.01840.

[67] Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang, Fan Yin, Shuai Shi, Yefei Chen, Xiaomin Dong, Qiaoling Chen, Jin Pan, Ji Li, Laixin Xie, et al. SpecForge: A Flexible and Efficient Open-Source Training Framework for Speculative Decoding. *arXiv preprint arXiv:2603.18567*, 2026. URL https://arxiv.org/abs/2603.18567.

[68] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. URL https://arxiv.org/abs/2309.06180.

[69] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference. *arXiv preprint arXiv:2410.21465*, 2024. URL https://arxiv.org/abs/2410.21465.

[70] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[71] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[72] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

[73] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025. URL https://arxiv.org/abs/2503.14476.

[74] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group Sequence Policy Optimization. *arXiv preprint arXiv:2507.18071*, 2025. URL https://arxiv.org/abs/2507.18071.

[75] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv:2409.19256*, 2024. URL https://arxiv.org/abs/2409.19256.

[76] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, et al. AgentRL: Scaling Agentic Reinforcement Learning with a Multi-Turn, Multi-Task Framework. *arXiv preprint arXiv:2510.04206*, 2025. URL https://arxiv.org/abs/2510.04206.

[77] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Defeating the Training-Inference Mismatch via FP16. *arXiv preprint arXiv:2510.26788*, 2025. URL https://arxiv.org/abs/2510.26788.

[78] Zhewei Yao and Yuxiong He. FlashNext-RL: A System Optimization Study for Long-Horizon Agentic RL of Qwen3.8-Flash-Next. *arXiv preprint arXiv:2610.28998*, 2026. URL https://arxiv.org/abs/2610.28998.

A Deployment Manifest of the Selected Submissions

This appendix is the exact reference for the four selected submissions (the finals); Table 3 summarizes them and Sections 4 and 5 describe them in prose. Each final is identified by its Kaggle submission id and by the script version of the notebook that produced it. The deployment manifest inside the notebook pins model, drafter, SGLang tree and Codex binary by SHA-256. Tables 16 and 17 list what each final ran, and Appendix C gives the steps that regenerate each notebook from the public repository.

Competition hardware. The ARC-AGI-2 notebooks run on four 24 GB L4 GPUs (sm_89), connected through PCIe Gen3 without NVLink, with 46 CPU cores and 173 GiB of host RAM. The ARC-AGI-3 notebooks run on one RTX PRO 6000 (sm_120), reporting 97,887 MiB of device memory, with 46 CPU cores and 175 GiB of host RAM. These specifications come from in-notebook probes. Both ARC-AGI-2 finals run 128 concurrent agent sessions; the model, speculative drafter, and cached session state share the available device memory. The manifests below give the model and serving configurations.

A.1 Selection notes

Sections 4 and 5 explain the settings in both tables and the measured alternatives that neither final uses, and Section 11 reports the in-place weight switch. Submission 57747104 is an unchanged resend of 57677693, which had scored 84.72, and its Codex idle timeout is raised to match the 9 h episode limit.

Table 16: ARC-AGI-2 finals (4×L4, 12 h). Column heads give the Kaggle submission id and its public score. Both run TP4 with attention DP2×TP2 and EP4, host-memory collectives and HiSparse.

	57747104 (87.22)	**57848658** (85.97)
Kaggle notebook @ script version	`qwen38fn-l4x4-c128-ext300-arcv6` @362595216	`qwen38fn-l4x4-c128-v10-rl30p8-x39` @364318725
Code in the public repository	`submissions/agi2-57747104-ext300-arcv6`	`submissions/agi2-57848658-x39-rl` and `agi2/`
Target model	gEAN320 INT3 GPTQ, W3A8, QAD v3 (student trajectories, reverse KL)	same
RL weights	none	r4@30 patch: 516 tensors, 3.35 GB, swapped in at load
Drafter	DFlash plan B, 8 layers, step 25,562, 16k draft vocabulary, K=4	same
SGLang fork	`sglang-arc-2026`, tag `kaggle-final`	same
Codex build	p7 (patches 0001–0007)	p8 (adds 0008)
Concurrency; KV pool	128 sessions; 5,127,616 logical KV tokens per DP	same
Context tokens: served / Codex window / compaction point	153,600 / 150,000 / 100,000	179,968 / 150,000 / 100,000
Thinking cap	none	49,152 tokens
Solvers	first + independent second (may write `solution2.py`)	same, plus up to two extra solvers for tasks still failing the demonstrations at 80k tokens
Refiners	refiner + re-refiner in spare slots; vote	same; re-refiner skipped when all candidates agree
Output-token budget	200k per solver; first solver +68k if the time gate allows; refiner 100k	same, plus one idle extension
Helper library; prompt sets	`arc_v6`; v3k, v3dualk, v3refine-dual	`arc_v10`; v4y, v4ydual, v4refine
Time limits	9 h per episode; deadline 700 min after start	same, plus a wall-clock end-by guard

The body defines the terms used here: model and drafter in Section 7 and Appendix E; serving terms in Section 8; Codex builds p7 and p8 in Section 9 and Appendix D; the RL checkpoint r4@30 in Section 11; solvers, refiners, the condition for the first-solver extension (the time gate) and the idle extension in Section 4. `arc_v6` and `arc_v10` are two versions of the helper library; the names after them are the prompt sets of the first solver, the second solver and the refiner, in that order. The end-by guard caps the harness deadline so that the notebook ends by 23:30 UTC on 2 November 2026.

Table 17: ARC-AGI-3 finals (1×RTX PRO 6000, 9 h). Column heads give the Kaggle submission id and its public score. Both serve with TP1, W3A8 experts, NVFP4 KV and the QSA/HiSparse session cache, and play 110 games concurrently.

	57678190 (75.56)	**57846476** (75.44)
Kaggle notebook @ script version	`arc-code-hs110-click` @362591255	`arc-code-hs110-v14tk5an-cards-swv8` @364285204
Code in the public repository	`submissions/agi3-57678190-v35-click`	`submissions/agi3-57846476-v40-cards` and `agi3/`
Target model; RL weights	QAD v3 package as in Table 16; no RL	same; RL v8@20 swapped in place at 6 h (516 tensors)
Drafter, SGLang tree, Codex	as ARC-AGI-2; Codex p8	same
KV pool	11,714,560 logical tokens, NVFP4	same
Context tokens: served / Codex window / compaction point	240,000 / 235,000 / 100k–150k (dynamic)	same
Thinking cap; sampling	24,576 tokens; T=0.7, top-p 1	same
Harness; player prompts	first generation; click report; prompts v14	same, plus toolkit tk5; prompts v14tk5an
Reviewer	r7: from 2 h, games stuck ≥45 min; ≤24 reviewers, ≤6 reports per game	r9, reading game cards from the public games and from other games in the run
Final-call notice	none	15 min before the end
Agent environment	scipy 1.18.1	same

Section 5 describes the dynamic compaction point, the click report, the reviewer, the toolkit (tk5), the game cards and the final-call notice; Section 11 describes the RL checkpoint v8@20. r7 and r9 are the reviewer's prompt versions. v14 and v14tk5an are the player prompt sets; v14tk5an is v14 with the toolkit's instructions added and its paragraph on animations rewritten as a rule.

B Training Recipes of the RL Checkpoints in the Finals

Two finals carry RL weights (Table 2). Both checkpoints were trained with the streaming framework of Section 11 from the same distilled INT3 package that the base finals serve, with routing replay and in-place merged weight publication. Table 18 records the recipes used for these checkpoints. The objective, group definition and first-window restriction are introduced in Section 11.1; the results of these and all other RL runs appear in Tables 12 and 13.

Table 18: Recipes of the two RL checkpoints used in finals, as recorded for the runs. A run name with @N is the checkpoint after N optimizer steps. "Same" repeats the ARC-AGI-2 entry.

	ARC-AGI-2 r4@30 (in 57848658)	**ARC-AGI-3 v8@20** (in 57846476)
Starting weights	QAD v3 INT3 package (Section 7)	Same
Trainable	LoRA adapters	direct updates to 372 projections (228 attention/GDN and 144 shared-expert); FP32 master, FSDP2; remaining backbone parameters frozen
Objective	$A = R - \bar{R}_{\text{group}}$; masked token-level importance sampling (MIS), Equation (2), with detached weight $\rho\,\mathbf{1}[0.2 \le \rho \le 5]$; token mean within group, equal weight across groups; no KL term	Same
Optimizer	AdamW (0.9, 0.95), $\epsilon = 10^{-12}$, no weight decay, clip 1.0; lr 10^{-4}, 2 warm-up steps, then constant	Same, except lr 3×10^{-6}
Target batch per step	24 groups × 24 episodes	16 groups × 16 episodes; groups with equal rewards discarded
Checkpoint	step 30 of 36 trained	step 20 (run paused there)
Reward	1 if every test output is correct, else 0	level cleared: action-efficiency reward $R_{\text{clear}}(r)$, Equation (3); normal first-window boundary without a clear: $\min(T, \widehat{R}_{\text{clear}})$ with T the tier that an LLM judge gives the player's notes; early self-triggered handoff and other unsuccessful stops: 0
Episode	40k output tokens (wrap-up at 36k output or 58k context, then 4k more); 64k context; effort xhigh	first context window only (235k capacity, handoff at 130k); no generation in the next window; 300k output cap and 3 h wall limit
Training data	90 tasks: 60 public evaluation tasks (the other 60 held out) and 30 audited hard tasks	235 level seeds from 18 public games; 5 public games held out from this run; 2 further games had no seeds
Rollout sampling	$T = 1.0$, top-p 0.95, top-k 20	Same
Routing replay	on	on
Delivered as	weight patch, 516 tensors, 3.35 GB, loaded at start-up	weight delta, 516 tensors, swapped in place at 6 h

LoRA recipe provenance. The resolved trainer configuration of the r4 run is archived in the public repository at `training/run_records/C2/config/trainer-config.json`. It records LoRA rank 8, scaling parameter $\alpha = 16$, adapters on the shared expert, a learning rate of 10^{-4} with two warm-up steps, and Adam $\epsilon = 10^{-12}$. The trainer code applies no LoRA dropout and attaches adapters to 228 QSA/GDN projections and 144 shared-expert gate/up/down projections (372

modules).

The ARC-AGI-3 reward has three cases. Let r be the number of actions divided by the human baseline, and let $L = \min(115,\ 100/r^2)$. A cleared level receives

$$R_{\text{clear}}(r) = \begin{cases} 0.9 + 0.1\,(L - 100)/15, & r \le 1, \\ 0.9\,e^{-5(r-1)}, & r > 1. \end{cases} \tag{3}$$

An episode that reaches the normal first-window boundary without a clear receives $\min(T, \widehat{R}_{\text{clear}})$. Here T is the tier that an LLM judge (gpt-6-astra via Codex) gives the player's notes, one of 0.9, 0.8, 0.55, 0.25, 0.1 and 0. $\widehat{R}_{\text{clear}}$ is the clear reward at the estimated total actions: the actions already taken plus the estimated remaining actions. An early self-triggered handoff and any other unsuccessful stop receive 0.

The notes reward is assigned to the frozen first-window handoff. The judge assesses each group in two shuffled orders; adjacent tiers use the lower score, while a larger disagreement triggers a third assessment and the median tier. This reward is a proxy for later progress, not a measured continuation through the next window.

Evaluation splits are relative to the named RL run. The ARC-AGI-2 60-task split and the later 513-task comparison exclude r4 training tasks; neither establishes family-disjoint evaluation across all preceding development. The ARC-AGI-3 full-game evaluations use public games and report trained, held-out and seedless-game subsets where available. A held-out snapshot from a trained game is a different evaluation unit from a held-out game.

C Reproducibility and Data Separation

C.1 Rebuilding a selected submission

The public repository (https://github.com/ycchen-tw/ARC-Forge-2026) is a single release of the code behind the four selected submissions. It has five top-level directories.

- `submissions/` holds the four notebooks exactly as Kaggle ran them, their Kaggle input lists and, for ARC-AGI-2, the asset manifest. Its script `extract_code.py` unpacks the code a notebook carries and checks it against the notebook's own manifest.
- `agi2/` and `agi3/` are snapshots of the ARC-AGI-2 stack behind 57848658 and of the ARC-AGI-3 stack behind 57846476, each with its notebook builder.
- `runtime/` builds the offline Kaggle inputs: the SGLang runtime archive, the SGLang fork tree and the patched Codex binaries (p7 and p8, Appendix D).
- `training/` shows how the weights were produced: pruning, GPTQ, distillation, the two RL runs, the drafter and the data collection behind them. Its `run_records/` directory holds the launch scripts, resolved configurations and logs of the actual runs.

The exact code of any selected submission comes out of its notebook:

```
python submissions/extract_code.py \
    submissions/agi2-57747104-ext300-arcv6/notebook.ipynb <dir>
```

The two later submissions can also be rebuilt from source. For ARC-AGI-3 (57846476), run in `agi3/`

```
python work/arc_code/kaggle/build_notebook.py --config <notebook config> --out <dir>
```

For ARC-AGI-2 (57848658), run in `agi2/`

```
python work/qwen38fn_l4/kaggle/build_notebook.py --preset <preset> \
    --asset-manifest ../submissions/agi2-57848658-x39-rl/asset-manifest.json \
    --harness-root . --kernel-id <id> --title <title> --out-dir <dir>
```

The repository README gives the full command lines. Both builds reproduce the submitted notebook's configuration, asset manifest, generated cells and list of carried files (264 files for ARC-AGI-2, 660 for ARC-AGI-3). The carried files in the repository have English comments and messages, so they differ from the submitted files in that text only. The two base submissions, 57747104 and 57678190, were built from earlier revisions of the same code; their exact code comes from `extract_code.py`.

The list gives the configuration file of each submission. The ARC-AGI-2 presets are in `agi2/work/qwen38fn_l4/kaggle/l4opt/`, and each names its harness configuration. The ARC-AGI-3 notebook configurations are in `agi3/work/arc_code/kaggle/`.

- 57747104: `kaggle-1027-c128-planb-l8s25562-qadR-dual-idlefix-plenvfp4-ext300-arcv6.json`
- 57848658: `kaggle-1101-c128-planb-l8s25562-qadR-dual-idlefix-plenvfp4-ext300-v10-skipagree-rl30p8-x39.json`
- 57678190: `notebook-reap320-0520-v12-hs110-dense-k8-v35-int3-w3a8-tb24k-planb-l8s25562-allmerge-qadv3-dyn-v512r2-scipy-review-click.json`
- 57846476: `notebook-reap320-0520-v12-hs110-dense-k8-v40-int3-w3a8-tb24k-planb-l8s25562-allmerge-qadv3-swv8at20-dyn-v512r2-scipy-v14tk5an-review9-cards-click.json`

The notebooks run offline and mount Kaggle models and datasets: the model package, the two RL weight patches, the drafter, the SGLang runtime and fork tree, the Codex binaries and a few Python wheels (Appendix A). The repository README lists each input with the script that builds it. A rebuild fixes the configuration; the score still varies between runs of one notebook version (Section 2).

C.2 Released serving code and training data

The SGLang fork is at https://github.com/ycchen-tw/sglang-arc-2026, with one tag per use. The four selected submissions serve the tree at `kaggle-final`. Trajectory capture and calibration used the tags `train-tito-collect`, `train-gean-reap`, `train-int3-gptq` and `train-rl-int3` (Sections 7 and 10). The rollout servers of the two selected RL runs are `rl-rollout-node1` for r4 and `rl-rollout-node2` for v8 (Section 11).

The training data is at https://huggingface.co/datasets/ycchen/arc-2026-training-data. It has five parts.

- `arc2_pools/`: the ARC-AGI-2 task pools, including the corrected public evaluation set *eval120fix* and the 90-task pool of the RL run r4 (Section 11).
- `arc3_seeds_v8/`: the level seeds of the RL run v8 with their workspaces: 235 training seeds from 18 public games and 2 held-out seeds (Section 11).
- `qadmix-1019-1022/`: the distillation set of Section 7: 20,200 student-trajectory segments with 1.15B tokens, of which 723M are supervised.
- `corpora/`: three corpora used to train the drafter (Appendix E): generations of the FP8 model on Nemotron prompts (300,010 rows), teacher rollouts on NVARC puzzles (47,335 rows), and agentic segments on NVARC puzzles (125,711 rows).
- `dflash_plans/`: the drafter's capture plans.

Four things are not included: the teacher targets of the distillation set (139 GB), the TITO ledgers that the drafter plans index, the NVARC puzzle files, and the community-made ARC-AGI-3 games. The dataset card gives formats, readers, licenses and the overlap with public evaluation tasks; the pools and sets named in the next subsection are the ones in this release.

C.3 Data roles

Training data was not fully separate from the public evaluation sets, and the list below states where they overlap. Kaggle scores come from the competition's hidden tasks and games (Section 2).

- **ARC-AGI-2 development.** Local comparisons use the 120 public evaluation tasks; the corrected set *eval120fix* repairs six tasks with defective demonstrations or references and splits the set into 60 development and 60 holdout tasks. Scheduling rehearsals use a 240-task roster (two copies of the public set) to match the size of the hidden run.
- **ARC-AGI-2 training data.** The student-trajectory distillation collection (Section 7) combines generated trajectories from several task pools. In that collection, 20.6% of 7.23×10^8 supervised training tokens came from evaluation-derived pools: `quality300` (10.2%), `independent1000` (6.1%), `nvarc_full` (3.4%), and `synth108` (0.9%). These pool-level shares identify evaluation-task parents or families; they are not per-task measurements of semantic overlap. Disjoint task identifiers do not establish family isolation. The selected ARC-AGI-2 RL run (Section 11) used 60 public-evaluation tasks and 30 audited hard tasks; its held-out sets exclude that run's training tasks, not necessarily all prior adaptation data.
- **ARC-AGI-3.** Development used the 25 public games, and RL runs trained on subsets of their game snapshots. "Held-out seeds" means snapshots withheld from the named run; "untrained games" means games not used to train that checkpoint. Neither designation implies separation from all earlier development. Hidden Kaggle games were evaluated separately under the competition protocol.

C.4 Verification of the environment and of deployments

- **Local ARC-AGI-3 environment.** Recorded solutions replay on the official engine for all 25 public games and 183/183 levels (6,358 actions), and local scores match the official calculator on

4,000 of 4,000 checks.

- **Trajectory joins.** Every Codex response id in the acceptance runs joins to a serving-ledger session (676 of 676; Section 10).
- **Weight delivery.** Model and drafter files are pinned by SHA-256 in the deployment manifest, and RL weight patches log the number of tensors loaded at start-up.
- **Detectors.** Leakage and taint scanners are validated with positive controls for the parser and patterns exercised by those controls (Section 12). The historical audit scope and remaining checkpoint-lineage check are described in Section 14.

D Codex Runtime Patch Inventory

Table 19 lists every patch we apply to Codex 0.154.0. The selected submissions use two builds. Build p7 applies patches 0001–0007 and runs the 87.22 ARC-AGI-2 submission. Build p8 revises video-token estimation in patch 0001 and adds patch 0008, `server_reasoning_included`; its patches 0002–0007 are identical to p7. It runs the other three selected submissions. The deployment manifests in Appendix A record the build of each submission; the mechanisms and measured effects are discussed in Section 9.

Table 19: Patches on Codex 0.154.0. Each row gives the problem, then the change. Build p7 contains 0001–0007; build p8 contains the revised 0001 and adds 0008. All are off by default and enabled per configuration, except the p8 video-token estimate, which has no switch.

Patch	Problem and change
0001	No video input: adds a `view_video` tool and video modality (frames for ARC-AGI-3 animations).
0001 (p8)	A returned video was counted as text by byte size (a 450 kB recording as about 150k tokens, compacting at once): estimates video tokens from the frame count and frame size in the mp4 header.
0002	Session-pinned radix cache has no TTL: sends `session_id` and calls `/close_session` after compaction and at exit.
0003	Compaction dropped the tool schema, so its prefix diverged at token ≈41: compaction now reuses the turn's tools (prefix hit 97.6%).
0004	Harness-requested compaction at the next turn boundary (file trigger with token threshold).
0005	Pre-compaction reminder injected once per window so the agent writes notes first.
0006	Compaction kept stacked reminders (84 of 87 compactions kept 1–4 copies): drops them.
0007	Tool output truncated by bytes/4 (about 10k tokens of hex grid instead of 2.5k): truncates by true tokenizer count.
0008 (p8)	After the 0005 reminder, the context check re-estimated reasoning already in the server's usage, about 11k tokens too many (compaction near 92k, the Kaggle median, instead of 100k): trusts the server's count.

E Designing and Training the Speculative Drafter

All four selected submissions serve the same speculative drafter (Tables 16 and 17). It is a DFlash-style block drafter [9] with eight decoder layers in the "plan B" architecture of Appendix E.9, taken at training step 25,562. In each verify cycle it proposes K=4 tokens at draft temperature 0.7 from a 16k draft vocabulary (Section 8). Decoding on both deployment machines is dominated by target verification (Section 8), so a drafter is useful only if it is accepted often and costs little per cycle. Six ideas produced this drafter, and the section presents them in order:

1. initialize the drafter from the target's native multi-token-prediction (MTP) layer (Appendix E.3);
2. deepen it with layers of the target, because stacked copies of the MTP layer fail (Appendix E.4);
3. convert the MTP layer's mixture-of-experts (MoE) feed-forward network (FFN) into a fixed dense one (Appendix E.5);
4. make the drafter's first step the same computation as the MTP step (Appendix E.6);
5. train on the probability that the verifier accepts the draft (Appendix E.7);
6. correct for training on recorded, off-policy data, including an on-policy variant (Appendix E.8).

Appendices E.9 to E.11 then cover the shipped architecture, the trainer and data, and the serving measurements behind K and the draft temperature.

E.1 Block drafting and metrics

One forward pass of the drafter fills a block of B=8 positions: an *anchor* slot carrying the last committed token, followed by seven masked slots. The pass is conditioned on the frozen target's hidden state at the anchor. The target carries its residual stream in four parallel branches (hyper-connections, HC); the drafter reads the final residual before these branches collapse, which we abbreviate GR (gated residual). The target then verifies the first K proposals by rejection sampling [22, 23] at the request temperature, so the drafter changes speed but not the output distribution. Figure 13 shows the shipped structure.

The baseline is the target's native MTP layer [63, 64], one trained decoder layer whose input fusion combines a token embedding with GR. SGLang's NEXTN mode applies it autoregressively, one draft token per step; we call this the *NEXTN draft*.

Metrics. For slot j let p_j be the target's distribution at T=0.7 over the full vocabulary, C_j the drafter's 16 candidates and q_j its proposal distribution restricted to C_j. Then

$$a_j = \sum_{v \in C_j} \min(p_j(v), q_j(v))$$

is the probability that rejection sampling accepts slot j. On held-out trajectories, teacher-forced on the data prefix, we report $\mathbf{E} = \sum_{k=1}^{7} \prod_{j \le k} a_j$, the expected number of accepted drafts per block (0–7, no bonus token), averaged uniformly over anchors; **E@**K truncates the sum at K. **Renewal** is the same expectation with anchors visited as serving visits them: each cycle starts after the previous cycle's accepted drafts and correction token. In serving we report **AL**, tokens committed per verify cycle including the bonus token, as counted by the server. Held-out sets changed between stages, so E values are comparable only within one panel of Tables 20 and 21.

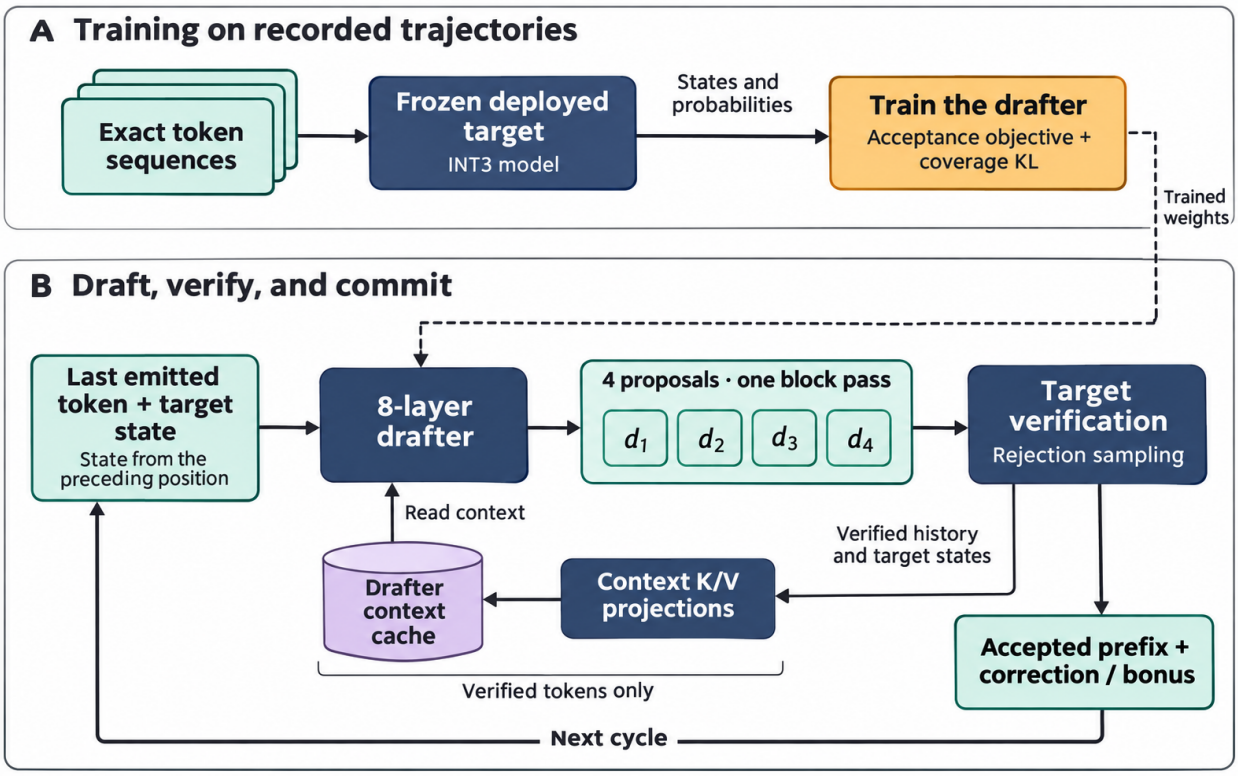


Figure 13: Training and decoding with the deployed speculative drafter. (A) Recorded token sequences are replayed through the frozen deployed target to obtain states and probabilities for the acceptance objective and coverage loss. (B) One block pass supplies four proposals for target verification. Only verified history updates the drafter context cache: token embeddings and predecessor target states are fused and projected into each drafter layer's keys and values, rather than copying the target attention cache. The correction or bonus token becomes the next anchor, paired with the predecessor state from verification; it has not yet entered the verified context cache.

E.2 Constraints and lineage

One drafter had to serve both the RTX PRO 6000 (ARC-AGI-3) and the four L4s (ARC-AGI-2). The hidden size is fixed at 2,560 because the drafter shares the target's embedding and output head; the L4 per-card memory budget rules out wider trunks that fit the RTX PRO 6000; and the author fixed the depth at five layers for the cost study that led to plan B. Five stages led to the shipped drafter, and the tables use their names. *MTP3-DFlash* stacked three copies of the native MTP layer and accepted 1.25× more tokens than the native MTP head (Section 8). *MTP5* used five layers and the slot alignment of Appendix E.6. *Dense10240* replaced the MoE FFN by a fixed dense FFN and has three runs: *v3*, the first production run, whose trainer truncated gradients (Appendix E.10); *v4*, which continues v3 with the corrected trainer; and *v5*, a control trained from random initialization. The *D line* was re-initialized from the native MTP layer followed by four target layers and trained on features from the deployed INT3 target. *Plan B* removed the hyper-connections and added target-KV injection, and *L8* deepened plan B to eight layers by copying. The L8 checkpoint at step 25,562 is the shipped drafter.

E.3 Initialization from the native MTP layer

Problem. Qwen3.8-Flash-Next ships a trained MTP layer whose input fusion already reads GR. A drafter that keeps this interface starts from a representation aligned with the target; one that does not must learn it.

Design. Both initializations in the lineage, MTP3-DFlash and the D line, start from the MTP layer's weights. The D line, from which the shipped drafter descends, uses the layer once, with its input fusion (Figure 13), and follows it with layers of the target (Appendix E.4).

Evidence. Without any training, the single MTP layer run as a NEXTN draft on the INT3 target reaches AL 3.08–3.10, and its first draft position is accepted with probability 0.84 (Table 20a). On one held-out set, a random-init drafter (v5) trained for one full pass with the fixed trainer finished at 3.721, 1.5% below the 3.776 of the drafter initialized from the trained MTP5 checkpoint (runs v3, then v4), and lower on all four corpora (Table 20c). That comparison also contains the v3 gradient truncation and its fix (Appendix E.10), so it does not isolate initialization, and it does not establish whether the truncated phase trapped the initialized run.

E.4 Combining the MTP layer with target layers

Problem. The MTP layer is one decoder layer, and the drafter trunk has five. We measured, without training, which added layers the MTP layer tolerates as a NEXTN draft on the INT3 target (Table 20a).

Design. The D line places four target layers after the MTP layer instead of further copies of it. Only the target's full-attention layers are candidates, because the drafter's KV pool holds full attention only; the D line uses layers 35, 39, 43 and 47.

Evidence. Stacking bitwise copies of the MTP layer fails: AL fell from 3.08 to 2.30 (two layers) and 1.62 (three), and the first draft position fell with it (0.84→0.66→0.44). The error is not accumulation along the draft chain; the layer was trained to be applied once. Placing *target* full-attention layers after the MTP layer costs far less: with target layer 43 (T43) after the MTP

layer AL was 3.03, and with T47 it was 2.97. The order matters: T47 placed before the MTP layer gave 2.63. Four copies of T47 after the MTP layer gave 2.02. The cost tracks how much each layer rewrites the residual stream: relative change $\|\text{out} - \text{in}\|/\|\text{in}\|$ was 1.6–2.0 for the MTP layer, 0.06–0.09 for T43 and 0.17–0.27 for T47.

Trained from this initialization, the D line started at E 0.626 (only slot 0 is functional, Appendix E.6) and reached 3.058 after 1,000 steps and 3.954 after 34,404 (Table 20d). The random-init run v5, with the same objective, was at 0, 2.165 and 3.721 at its end (different held-out sets; direction only).

The same measurements justified SWA 1024: replacing the MTP layer's sparse attention by a 1,024-token sliding window cost 0.05 AL, and even a window of one kept 74%, mostly lost at later draft positions.

E.5 From MoE to a dense FFN

Problem. The MTP layer routes each token to 10 of 512 experts. On the RTX PRO 6000 the five-layer MoE drafter (320 experts, NVFP4) held 5.08 GiB of weights. A first dense FFN built from 10 experts per layer (width 6,400, FP8) held 0.95 GiB. The freed memory went to the hot set of the HiSparse host-memory KV cache (Section 8), which grew by 43% in blocks, and each draft step ran 23% faster at 110 concurrent requests. Without retraining, however, its acceptance fell from 2.15 to 1.76, below the 2.08 needed to break even.

Design. The adopted conversion, Dense10240, uses 16 experts per layer. For each layer we sampled 2% of drafter rows in training proportions (proposal:clean 5.3:1), selected the 16 experts with the largest router mass, concatenated their gate/up rows and down columns into one 10,240-wide SwiGLU, and fitted per-expert coefficients α_e by least squares to the routed output; the shared expert was kept. We later split α_e as $\sqrt{\alpha_e}$ into the up and down projections: with α entirely in down, that matrix was 13× smaller than the others, so Adam moved it relatively faster.

Evidence (Table 20b). Starting from the trained MTP5 drafter, the dense init kept 79% of E (1.970 vs 2.484); least squares beat uniform α by 3.4% and won on all 18 held-out segments. The 16 experts explained only 35–41% of routed variance in layers 0–3. Training recovered the conversion loss within 804 steps (2.03→2.80, above the MoE source's 2.48). For the untrained native MTP as a NEXTN draft (Table 20a), dense conversion cost 18% AL (3.10→2.54), and widths from 5,120 to 20,480 gave 2.48–2.56: the loss comes from fixing the expert set per layer, not from capacity.

E.6 Aligning the first step with MTP

Problem. The native MTP's first draft position is its strongest (about 0.83 acceptance). An initialization from that layer helps only if the drafter's first slot performs the same computation.

Design. Two changes made slot 0 the NEXTN step. First, the token at position p is paired with the target state $H[p-1]$, as in DFlash and NEXTN, which also leaves one target forward per cycle. Second, slot j predicts x_{a+j+1}: the anchor slot carries the real token and emits the first proposal instead of serving only as input. Our 154 existing tests also passed with labels shifted by one position, which is how the first misalignment had survived 8,381 training steps; a causal test now fails on it.

Evidence. After converting the dense MTP+target-layer draft into the block drafter without training, slot 0 accepted 0.66 against 0.60 for the same weights' NEXTN first step (0.67 vs 0.63 for a second layer order; CIs disjoint; both use the real q in rejection sampling). We did not test the suspected cause, BF16 versus FP8 kernels. The mask slots initially received a copy of the anchor's GR; removing it left slot 0 unchanged (0.662 vs 0.661) but dropped the second position from 0.10 to 0.004. The D line was nevertheless trained with the mask-slot GR removed, so these slots learn their inputs from the mask embedding and attention; its held-out first-position acceptance rose from 0.600 to 0.841 in 1,000 steps. Whether the copied GR would have trained better was not compared.

E.7 An acceptance-oriented objective

Problem. Serving samples both q and p at the request temperature, and all captured trajectories were generated at T=0.7, but the original losses—hard cross-entropy, a position-weighted D-PACE cross-entropy, a selector cross-entropy, and a KL term on the first two slots—were all at T=1 and trained the selector on hard labels with detached hidden states.

Design. Following a design document supplied by the author, the objective optimizes the acceptance probability itself. With teacher p_j from the target's residual at $a{+}j$, drafter unary distribution $r_j = \mathrm{softmax}(U_j/T)$, candidates $C_j = \text{top-16}(U_j)$ and selector distribution q_j on C_j:

$$a_j = \sum_{v \in C_j} \min\big(p_j(v), q_j(v)\big), \qquad \mathcal{L}_{\mathrm{WTV}} = 1 - \frac{1}{7}\sum_{k=1}^{7}\prod_{j \le k} a_j, \tag{4}$$

$$\mathcal{L} = \frac{1}{N_{\mathrm{blk}}}\sum_{\text{blocks}} \mathcal{L}_{\mathrm{WTV}} + \lambda_{\mathrm{cov}}\frac{1}{N_{\mathrm{pos}}}\sum_{\text{slots}} \mathrm{KL}(p_j \,\|\, r_j), \qquad \lambda_{\mathrm{cov}} = 0.3,\ T = 0.7. \tag{5}$$

$\mathcal{L}_{\mathrm{WTV}}$ is one minus the mean survival probability, so later slots are weighted by the acceptance of their prefix; D-PACE's extra prefix weighting was therefore dropped, together with hard cross-entropy and the early KL. The full-vocabulary coverage KL is the only signal that reaches tokens outside the candidate set. Gradients flow *jointly* through the candidate logits and the selector's hidden projection into the backbone. The slot-0 q and $p(C)$ match the serving code bit for bit, and a_0 matches a Monte Carlo estimate of the serving acceptance within 4σ.

Evidence (Table 21a,b). At the 804-step pilot horizon the new objective beat D-PACE by only about 0.02 E (2.731 vs 2.711), about one seed standard deviation, on the quantity it optimizes. The clear differences were coverage KL (2.38 vs 2.95) and the later slots (last-slot a 0.380 vs 0.361). On the dense initialization with two seeds the E values were indistinguishable (2.804 vs 2.791) while the KL stayed lower (2.30–2.34 vs 2.83–2.96). λ_{cov}=1 was worse because the WTV gradient on the backbone was only 1–9% of the KL gradient at the start. The rise in E from 2.29 to 3.58 during production run v3 is the training progress of that one run over 22,000 steps; it is not the effect of the objective.

E.8 Off-policy data and on-policy training

Problem. The drafter trains teacher-forced on recorded trajectories, with target features recomputed by replaying each sequence through a capture server (Section 10). Serving differs in three ways: the conditioning features come from the deployed quantized target, the prefix is what the drafter

Table 20: Initialization variants. AL: serving tokens per verify cycle of an *untrained* native-MTP NEXTN draft (INT3 target, T=1, 64 prompts of 4–28k tokens, three draft steps). E: offline held-out expected accepted drafts per block at T=0.7 (0–7). MTP is the native MTP layer (320 experts); v3–v5 are the Dense10240 runs of Appendix E.2; HC is hyper-connections. Each panel uses its own held-out set; compare rows only within a panel.

Variant	AL (untrained)	E, step 0	E, trained
(a) Native MTP as NEXTN draft, no training			
MTP, MoE, one layer	3.08–3.10	–	–
Same layer copied, 2 / 3 layers	2.30 / 1.62	–	–
MTP, then target layer 43 / 47	3.03 / 2.97	–	–
MTP, then target layer 47 ×4	2.02	–	–
Target layer 47, then MTP	2.63	–	–
MTP, sliding window 1,024 / 1	3.03 / 2.28	–	–
MTP dense 10240, least-squares / uniform α	2.54 / 2.57	–	–
MTP dense 5120 / 20480	2.48 / 2.55	–	–
(b) Block-drafter inits, 18 held-out segments, all 586k anchors			
Trained MTP5 drafter, MoE (source)	–	2.484	–
MTP5 → dense 10240, least-squares α	–	1.970	–
MTP5 → dense 10240, uniform α	–	1.905	–
(c) One pass of 2.44B anchors, 1,058 held-out segments, macro			
MTP5 dense least-squares init: v3 (truncated gradient), then v4 (fixed)	–	–	3.776
Random init (v5)	–	0.000	3.721
(d) D line, INT3 capture, 1,073 held-out segments, macro			
MTP + target layers 35, 39, 43, 47, dense; 34,404 steps	–	0.626	3.954
(e) Architecture conversions, 105 held-out jobs			
HC drafter (D-line extension, step 9,425)	–	–	3.618
Plan B: plain residual + recompute	–	1.622	–
Plan B: plain residual + KV injection; 8,251 steps	–	0.569	3.531
L8, repeated layers zero-initialized	–	3.531	–
L8, layers copied (0,1,2,3,1,2,3,4); 25,562 steps (shipped)	–	3.446	3.703

and verifier produced, and anchors are visited in cycle order. Uniformly weighted E exceeded the cycle-weighted expectation by 7–18%, which is why we added the renewal metric. We applied three corrections; the first two are in the shipped drafter.

Matching the target. Runs v3–v5 captured features from an NVFP4 target; from the D line on, capture used the deployed gEAN320 INT3 W3A8 package, so the drafter is conditioned on the residuals it sees in serving. We did not isolate the effect of this change. The L8 run also added 508M anchors from the INT3 target and 778M from benchmark trajectories, which can raise acceptance on those tasks but, under rejection sampling, cannot change outputs.

Prefix gate v3. Training trajectories contain tokens the deployed target would rarely sample; supervision after such a token trains states serving seldom reaches. For each data token we compute the tail mass $u_t = \sum_{v:\, p_t(v) \le p_t(x_t)} p_t(v)$ under the full-vocabulary target at T=0.7, set $c_t = \min(1, u_t/\alpha)$ with $\alpha = 0.01$, and weight slot j by $w_j = c_a \prod_{i<j} c_{a+i+1}$, detached and applied to

Table 21: Objective and training variants. E: offline held-out expected accepted drafts per block at T=0.7. KL: coverage KL($p\|r$) per slot. Serving: accepted drafts per verify cycle (no bonus), T=0.7, 2,000 held-out turns from 130 sessions, two runs per drafter. D-PACE is the previous position-weighted cross-entropy; WTV is the acceptance term of Appendix E.7; v3 and v4 are the Dense10240 runs of Appendix E.2. Panels use different held-out sets.

Variant	E	KL	Serving
(a) Pilot, proxy init, 804 steps			
D-PACE + early KL (previous)	2.711	2.95	–
WTV + coverage, $\lambda_{\text{cov}} = 1$	2.639	2.40	–
WTV + coverage, $\lambda_{\text{cov}} = 0.3$ (adopted)	2.731	2.38	–
WTV + coverage, $\lambda_{\text{cov}} = 0.1$	2.735	2.46	–
WTV + coverage, $\lambda_{\text{cov}} = 0.03$	2.728	2.65	–
(b) Dense least-squares init, 804 steps, mean of 2 seeds			
D-PACE + early KL	2.791	2.83–2.96	–
WTV + 0.3 coverage	2.804	2.30–2.34	–
(c) Production runs, 1,058 held-out segments, macro			
v3 at step 22,917 (truncated gradient)	3.578	1.783	–
v4: v3 + 13,941 steps, fixed gradient	3.776	1.650	–
(d) Draft on-policy distillation from v4 (KL on held-out TITO-ledger segments)			
v4	–	1.637	2.870
Reached-state TV, 2 rounds	–	1.739	2.910
Reached-state TV, 8 rounds	–	1.838	2.912
Importance-weighted loss, 4 rounds	–	–	2.906
Offline, fresh v4 continuations, 4 rounds	–	–	2.899

both $\mathcal{L}_{\text{WTV}}$ and the coverage KL. The author adopted it for the D line together with a warmup–stable–decay schedule (decay over the last 20%) and a coverage weight annealed from 0.3 to 0.05; none of the three was ablated separately. In the D-line run the mean gate weight was 0.975 and about 1% of slots fell below 0.1. The shipped L8 run kept the gate ($\alpha = 0.01$, T=0.7), with decay over the last 30% and the coverage weight fixed at 0.05.

Draft on-policy distillation. Following the second part of the author's design document and on-policy distillation [16, 17], we ran the drafter in real speculative decoding on training prompts, recorded each verify cycle, and trained on the *reached* states: accepted drafts plus the first rejection, with a total-variation loss. An importance-weighted variant is unbiased for the expected accepted length. All rounds started from the v4 drafter. Measured in serving on 2,000 held-out turns (Table 21d), the best drafter added 0.041 accepted drafts per cycle (+1.4%, 95% CI [+0.021, +0.063]), and the gain saturated by the second round. The importance loss tied reached-state TV (−0.002), and offline training on fresh v4 continuations of the same prompts reached +1.0% by round four. The residual gap explains why: reached states lie on the verified sequence, which offline training also sees, and the dominant loss is within-candidate mismatch $p(C) - a = 0.168$, larger than the out-of-candidate mass $1 - p(C) = 0.127$ and reduced by only 0.002 over eight rounds; even slot 0 has 0.11. Pure on-policy training also raised coverage KL by 12%. The D line was re-initialized after these rounds, and neither its training recipe nor those of plan B and L8 contain on-policy rounds.

E.9 Architecture changes for deployment cost

Problem. The D-line drafter kept the MTP layer's four-branch hyper-connections (HC) and re-ran committed tokens through the trunk. Its trunk took 5.68 ms per cycle on the RTX PRO 6000 at 110 requests.

Plan B. Of four measured variants the author chose plan B: remove HC, keep the attention shape, fold the shared expert into the dense FFN (width 10,880), and inject target context K/V so committed tokens skip the trunk. The trunk took 3.49 ms (−39%); a variant that also rebuilt attention reached 2.91 ms but could not reuse the attention weights. The conversion averaged the HC branches and folded the shared-expert gate and mean residual gates into the projections. Zero-step E fell from 3.618 to 0.569 (1.622 with committed tokens still recomputed instead of injected; Table 20e), because the branch gates differ and deep-layer context K/V cannot be expressed from the shallow fused features. After 8,251 steps it reached 3.531, 2.4% below the HC drafter.

Deepening by layer copy. The author then expanded plan B from five to eight layers in the order 0,1,2,3,1,2,3,4. Zero-initializing the output projections of the repeated layers reproduced plan B exactly (E 3.531); the chosen plain copy started at 3.446 (−2.4%), recovered to within 0.5% of plan B in 1,000 steps and passed it after 3,000–4,000. After one pass over 2.25B anchors (25,562 steps, 31.2 h) it reached E 3.703 and renewal 3.060, 2.3% and 3.4% above the HC drafter; E@4 rose only 1.6%, since the lead grows with K (+1.6, +1.9, +2.1, +2.4% for $K = 4$–7). The held-out set is slightly optimistic: other segments of the same task groups were trained on, at the author's instruction. This checkpoint is the shipped drafter.

E.10 Trainer correctness and data

Gradient truncation. The trainer shares one "clean" pass over the context among all blocks. That pass ran without gradients and was reused for a median of 43 steps, so the backbone lost 52–59% of its gradient at initialization and 74% by step 22,000. Forward outputs matched serving and the reference implementation shared the truncation, so equivalence tests passed (Section 12.3). The fix recomputes the clean pass with gradients every step and checks every parameter against full autograd (worst relative error 0.65–1.2%, versus 73–78% for the old code). Continuing v3 from step 22,917 with the fix raised macro E from 3.578 to 3.776 over 13,941 steps (Table 21c); the Adam and learning-rate reset are confounded with the fix.

Data and anchors. The production plan combined TITO ledgers, two distillation corpora and Nemotron speculative-decoding data: 3.66B tokens, 2.43B anchors, and 1,058 held-out segments. The three corpora and the capture plans are released (Appendix C.2); the ledgers are not. That split, used through v5, grouped ARC tasks by variant and left 149 of 158 held-out parent tasks with siblings in training, so panel (c) of Table 20 measures in-family generalization; later plans group by parent task. Only sampled tokens are supervised: compacted agentic segments repeat earlier assistant turns verbatim, and masking these 679M tokens reduced supervised tokens from 2.52B to 1.85B. An anchor and its successor must both be generated tokens.

E.11 From offline acceptance to serving

The shipped settings, K=4 and draft temperature 0.7, were chosen by serving measurements. On the RTX PRO 6000 with 110 games, acceptance peaked at the deployed 0.7 (2.902); greedy drafts lost 6.3% (2.720) and 4.7% throughput, and temperatures 0.5–1.0 were within throughput noise. K=5 accepted 10.8% more per cycle than K=4, but the cycle took 16% longer and steady decode fell 5.5%, so we kept K=4. The resulting L4×4 speedups of 1.70–2.04× and their context-length caveat are in Section 8.

Offline gains shrink in serving. The eight-layer drafter's +2.3% E became +1.3–1.8% AL on the B200 (INT3 target, K=4, 240 replayed turns at 64 concurrent, T=0.7; paired bootstrap CIs exclude zero), and because verification takes about 38 of 41 ms per cycle, only +0.5–1% throughput, or +3.3% with the fused FP8 drafter path. 238 of the 240 turns were in its training split. Likewise, the D line led its predecessor by 0.18 E offline (on different held-out sets), but its step-18k and 20k checkpoints were 1.3% lower in single RTX PRO 6000 serving runs at the competition settings (2.857–2.864 vs 2.897, within single-run noise); the seven-slot lead sat mostly in slots 5–7, which K=4 never uses (E@4 at step 20k 2.669 vs 2.725, each on its own held-out set).